

AI应用开发：RAG技术从小白到深入理解-详细版



1、RAG的整体架构设计

在深入细节之前，我们先来看看RAG的全貌。我画了一张完整的架构图，咱们一起来拆解：

1.1 架构图说明

AI应用开发：RAG技术从小白到深入理解-...



用户655341 7月10日 16:41

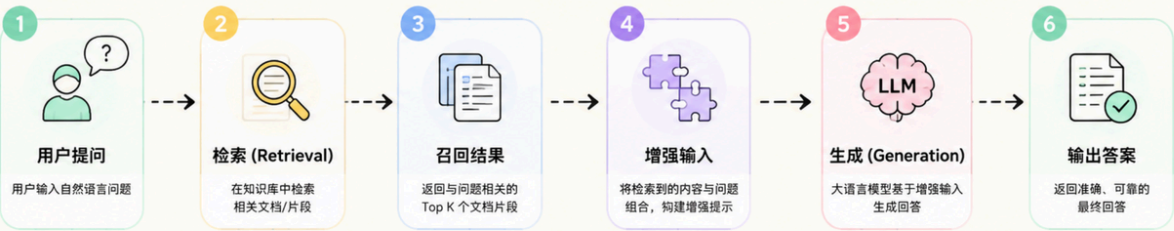
我为任老师举大旗 看谁敢于她为敌

RAG系统

检索生成的完整流程

✓ 检索增强
✓ 知识结合
✓ 更准确的回答

★ 从问题到答案，一步步拆解！



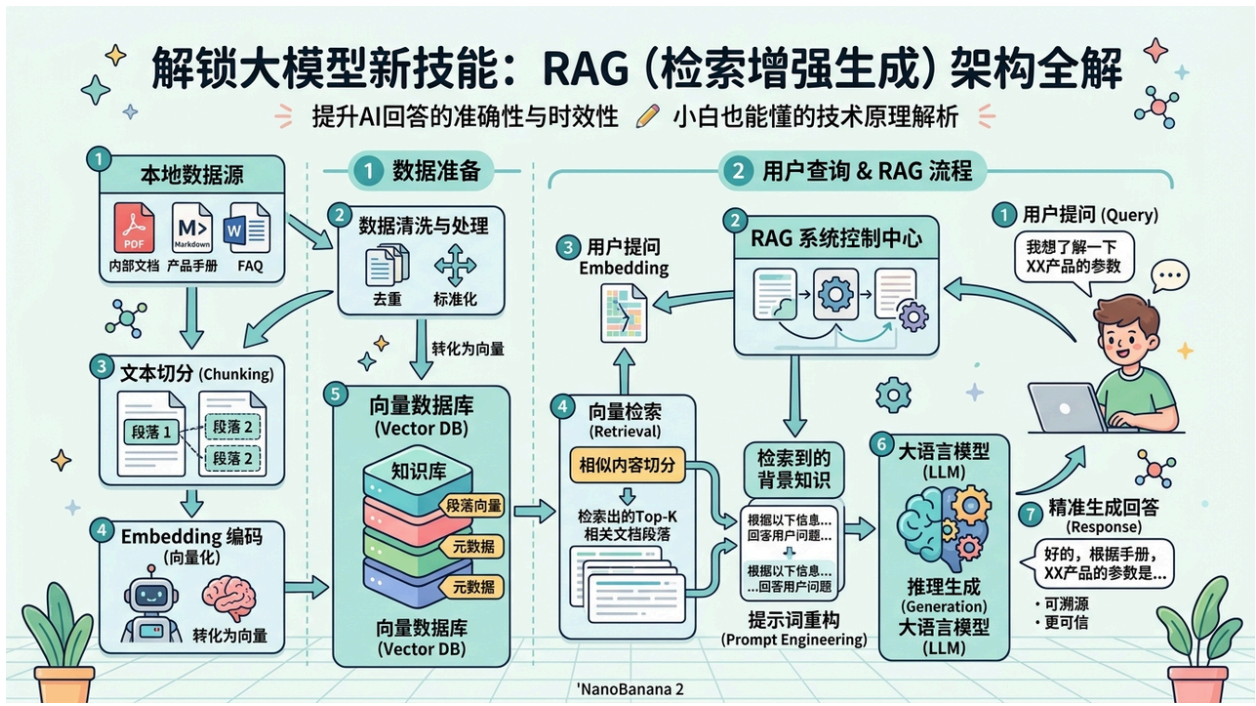
💡 RAG 的核心优势：结合外部知识与大模型能力，减少幻觉，提升回答的准确性与可追溯性！✨

🔥 这张图展示了RAG系统的完整流程，从左到右分为四个核心阶段：

这就像你在图书馆找资料写论文：

- **数据准备** = 图书馆的藏书
- **索引构建** = 图书分类和目录卡
- **检索** = 你根据关键词找书
- **生成** = 你看着书写论文

一、概览



1.1 Overview（RAG是什么？）

RAG全称是Retrieval-Augmented Generation，中文叫"检索增强生成"。听起来有点学术，咱们用大白话解释：

传统LLM的问题： 想象你养了一只鹦鹉（LLM），它很聪明，能流利地说话。但问题是，它只会说训练时学到的话，而且：

- 如果你问它昨天的新闻，它不知道（知识过时）
- 如果你问它你公司的内部规定，它也不知道（没见过的知识）
- 有时候它还会"胡编乱造"，一本正经地瞎说（幻觉问题）

RAG的解决方案： 给这只鹦鹉配一个助手，助手手里拿着一堆资料。每次你提问时：

1. 助手先翻资料，找到相关内容
2. 把资料递给鹦鹉
3. 鹦鹉看着资料回答问题

这样，鹦鹉的回答就有理有据了！

RAG技术通过将检索机制与生成模型结合，使LLM能够从外部知识源获取相关文档并基于这些文档生成答案，从而解决了传统LLM的知识过时、缺乏领域知识和幻觉等问题。

企业知识问答助手 - 完整系统架构

从用户交互到AI响应的全流程技术架构

```
graph TD; subgraph Layer1 [第1层: 用户交互层 (User Interface)]; direction LR; L1_1[Web界面  
React / Vue  
用户提问]; L1_2[移动端  
iOS / Android  
随时随地访问]; L1_3[API接口  
RESTful API  
第三方集成]; L1_4[聊天机器人  
Slack / Teams  
企业通讯集成]; L1_5[技术栈总览  
• 前端: React, TypeScript  
• 后端: Python, FastAPI  
• AI: LangChain, OpenAI API  
• 存储: Vector DB + SQL  
• 部署: Docker, K8s, AWS]; end; subgraph Layer2 [第2层: 应用逻辑层 (Application Logic)]; direction LR; L2_1[LangChain框架  
• Chains: 工作流编排  
• Agents: 智能决策  
• Memory: 对话记忆]; L2_2[QA问答引擎  
• 问题理解  
• 检索优化  
• 答案生成]; L2_3[业务逻辑  
• 权限管理  
• 日志记录  
• 性能监控]; L2_4[缓存层  
Redis缓存  
常见问题  
快速响应]; end; subgraph Layer3 [第3层: AI核心层 (AI Engine)]; direction LR; L3_1[大语言模型  
GPT-4 Turbo  
Claude Sonnet  
理解 + 生成  
🔥 按Token计费]; L3_2[嵌入模型  
text-embedding-3  
sentence-transformers  
文本 → 向量  
📏 768/1536维]; L3_3[智能检索  
向量相似度搜索  
混合搜索  
重排序优化  
🔴 Top-K结果]; L3_4[Prompt提示工程  
角色设定 + 任务描述  
Few-shot示例  
输出格式控制  
🔧 优化准确性]; end; subgraph Layer4 [第4层: 数据存储层 (Data Storage)]; direction LR; L4_1[向量数据库  
Pinecone / Chroma  
Qdrant / Weaviate]; L4_2[文档存储  
S3 / MinIO  
原始文件保存]; L4_3[关系数据库  
PostgreSQL / MySQL  
用户/日志/元数据]; L4_4[云存储  
AWS / Azure  
GCP]; end; L1_1 --> L2_1; L1_2 --> L2_1; L1_3 --> L2_1; L1_4 --> L2_1; L2_1 --> L3_1; L2_2 --> L3_1; L2_3 --> L3_1; L2_4 --> L3_1; L3_1 --> L4_1; L3_2 --> L4_1; L3_3 --> L4_1; L3_4 --> L4_1;
```

第1层: 用户交互层 (User Interface)

- Web界面
React / Vue
用户提问
- 移动端
iOS / Android
随时随地访问
- API接口
RESTful API
第三方集成
- 聊天机器人
Slack / Teams
企业通讯集成
- 技术栈总览**
 - 前端: React, TypeScript
 - 后端: Python, FastAPI
 - AI: LangChain, OpenAI API
 - 存储: Vector DB + SQL
 - 部署: Docker, K8s, AWS

第2层: 应用逻辑层 (Application Logic)

- LangChain框架**
 - Chains: 工作流编排
 - Agents: 智能决策
 - Memory: 对话记忆
- QA问答引擎**
 - 问题理解
 - 检索优化
 - 答案生成
- 业务逻辑**
 - 权限管理
 - 日志记录
 - 性能监控
- 缓存层**
 - Redis缓存
 - 常见问题
 - 快速响应

第3层: AI核心层 (AI Engine)

- 大语言模型**
 - GPT-4 Turbo
 - Claude Sonnet
 - 理解 + 生成
 - 🔥 按Token计费
- 嵌入模型**
 - text-embedding-3
 - sentence-transformers
 - 文本 → 向量
 - 📏 768/1536维
- 智能检索**
 - 向量相似度搜索
 - 混合搜索
 - 重排序优化
 - 🔴 Top-K结果
- Prompt提示工程**
 - 角色设定 + 任务描述
 - Few-shot示例
 - 输出格式控制
 - 🔧 优化准确性

第4层: 数据存储层 (Data Storage)

- 向量数据库**
 - Pinecone / Chroma
 - Qdrant / Weaviate
- 文档存储**
 - S3 / MinIO
 - 原始文件保存
- 关系数据库**
 - PostgreSQL / MySQL
 - 用户/日志/元数据
- 云存储**
 - AWS / Azure
 - GCP

DAG的推广：代数：

特性	传统LLM	RAG系统
知识更新	需要重新训练 (成本高)	只需更新知识库 (成本低)
事实准确性	容易产生幻觉	基于检索到的真实文档
领域适应	需要微调模型	添加领域文档即可
可追溯性	无法追溯来源	可以标注来源和引用
成本	训练和部署成本高	相对经济实惠

举个实际例子：

假设你在做一个医疗问答系统:

用户问："阿司匹林的常见副作用是什么？"

传统LLM可能回答：

"阿司匹林可能导致胃部不适、恶心等。(但不确定是否完整或最新)"

RAG系统的工作流程：

1. 检索系统在医学数据库中搜索"阿司匹林 副作用"
2. 找到3篇相关医学文献
3. 提取关键信息：胃肠道反应、出血风险、过敏反应等
4. LLM基于这些文献生成答案：

"根据医学文献，阿司匹林的常见副作用包括：

- 胃肠道反应（胃痛、恶心、消化不良）
- 出血风险增加
- 过敏反应（如荨麻疹）

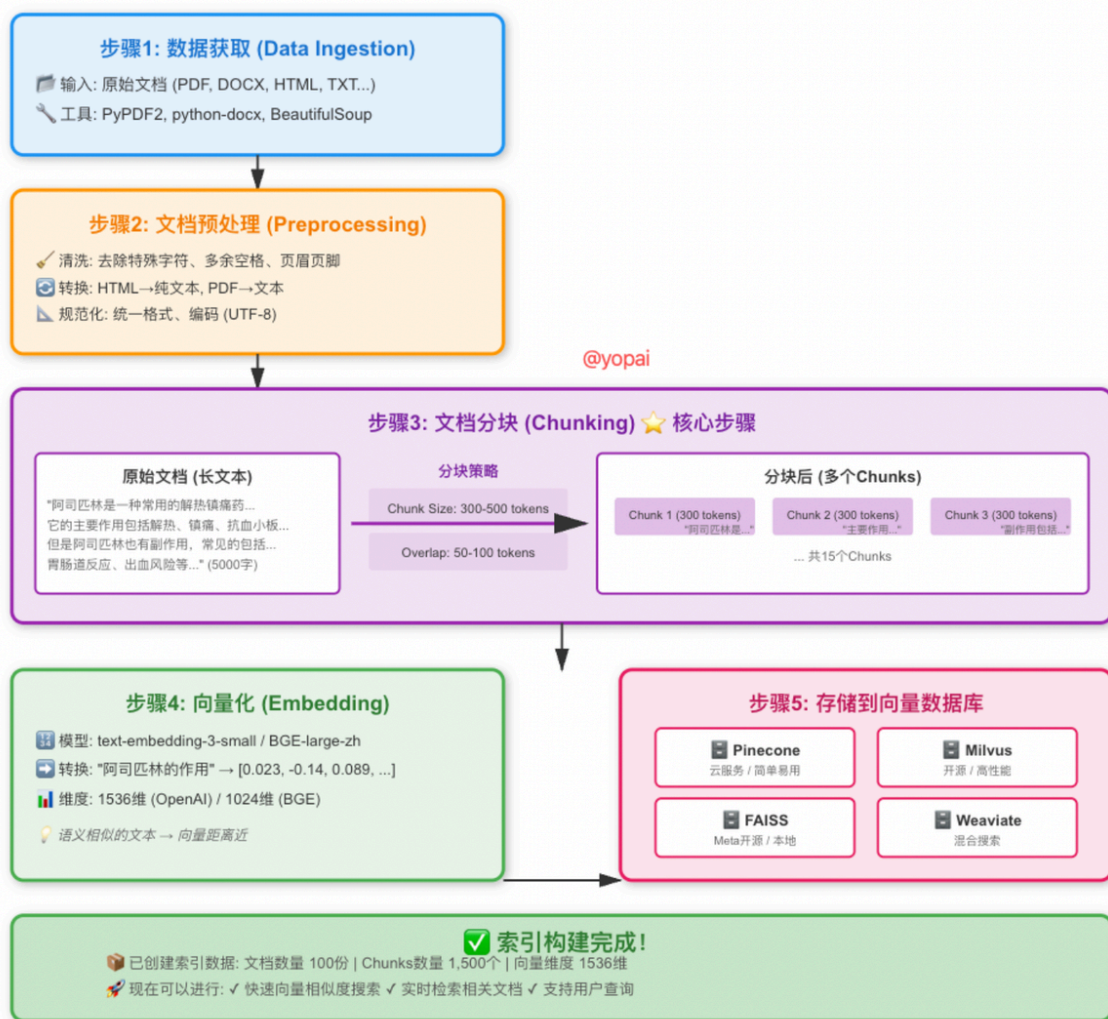
参考来源：[文献1]、[文献2]"

看到区别了吗？RAG不仅更准确，还能告诉你信息来源！

1.2 Indexing（索引构建：把知识装进"大脑"）

索引构建是RAG的"预处理"阶段，就像你上学前要先整理笔记、做好书签一样。这个阶段要把原始文档转化成机器能快速检索的格式。

索引构建详细流程 (Indexing Pipeline)



拿到原始数据后，需要"清洗"一下：

鸿钦

张鸿钦 7月20日 18:29

数据清洗的主要好处是：

- 提高数据质量和一致性。
- 减少无关信息对模型的干扰。
- 提高搜索、文本切分和向量匹配的准确率。
- 降低存储量与计算成本。
- 避免“垃圾进，垃圾出”：输入数据质量通常会直接影响最终结果。

移除多余的空格和换行 `doc = re.sub(r'\s+', ' ', doc)`

用户923563

6月16日 14:54

这部分写的太简单了吧？原来的格式不保留，会不会不方便模型理解？

`if doc_type == 'pdf': text = extract_text...`

用户242371

2月7日 23:22

很奇怪，如果不是 pdf 文档不就 text 空值？

用户737509

3月1日 12:23

可以多加几个条件，它在调用这个函数前肯定会作一个前置判断的

彭绍湧

4月30日 11:05

这个实际操作的时候，要根据不同的文档做不同处理，例如 pdf 还可以结合 OCR 来处理，多种处理方式

2.1 数据获取（Data Ingestion）

第一步是收集数据，可能来自：

- PDF、Word、Markdown等文档

把大文档切成小段，每段包含一个相对完...

飞书用户1871RQ

2月26日 16:37

模块化

-  网页、Wiki
-  数据库
-  邮件、聊天记录
-  Excel、CSV等结构化数据

实际案例： 某电商公司想做客服机器人，他们需要收集：

- 产品说明文档（PDF）
- 常见问题FAQ（网页）
- 历史工单数据（数据库）
- 退换货政策（Word文档）

2.2 文档预处理（Preprocessing）

拿到原始数据后，需要"清洗"一下：

代码块

```
1  #示例：文档预处理代码
2  def preprocess_document(doc):
3      # 1. 移除多余的空格和换行
4      doc = re.sub(r'\s+', ' ', doc)
5      # 2. 提取纯文本（从PDF、HTML等）
6      if doc_type == 'pdf':
7          text = extract_text_from_pdf(doc)
8      # 3. 规范化格式
9      text = text.strip().lower()
10     # 4. 去除无用信息（页眉、页脚等）
11     text = remove_headers_footers(text)
12     return text
```

2.3 文档分块（Chunking）

这是最关键的一步！为什么要分块？

想象一下： 你有一本500页的《红楼梦》，如果有人问"林黛玉是谁"，你不可能把整本书都递给他看吧？你会翻到介绍林黛玉的那几页。

分块就是这个道理：把大文档切成小段，每段包含一个相对完整的语义单元。

常见的分块策略：

1. 固定大小分块（最简单，但可能切断语义）

代码块

```
1  #每300个字符一块
2  chunk_size = 300
3  chunks = [text[i:i+chunk_size] for i in range(0, len(text),
    chunk_size)]
```

1. 句子分块（按自然语言边界）

代码块

```
1  #按句号、问号、感叹号分割
2  import nltk
3  sentences = nltk.sent_tokenize(text)
4  段落分块（保留逻辑结构）
5  #按换行符或段落标记分割
```

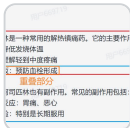
阿司匹林的主要作用包括



用户669719 7月10日 09:25
这部分为什么是重叠部分，上面不是"抗血小板：预防血栓形成"吗？



用户669719 7月10日 11:05
不应该是这样吗



张鸿钦 7月20日 18:39
这是标准的滑动窗口重叠，但是重叠也有可能是保留公共上下文、语义重叠（补充定义或前文）、层级分块（每个子块可能都包含完整语义单元）

```
6 chunks = text.split('\n\n')
7 滑动窗口分块（带重叠，避免信息丢失）
8
9 chunk_size = 300
10 overlap = 50 # 重叠50字符
11 chunks = []
12 for i in range(0, len(text), chunk_size - overlap):
13     chunks.append(text[i:i+chunk_size])
```

分块的关键参数：

- 块大小（Chunk Size）
 - ：太小 → 语义不完整；太大 → 检索不精准
 - 推荐：200-500 tokens（约150-400个汉字）
- 重叠（Overlap）
 - ：避免关键信息被切断
 - 推荐：10-20% 的块大小

举个例子：

```
代码块
1 原文：
2 阿司匹林是一种常用的解热镇痛药。它的主要作用包括：
3 解热：降低发烧体温
4 镇痛：缓解轻到中度疼痛
5 抗血小板：预防血栓形成
6
7 但是，阿司匹林也有副作用。常见的副作用包括：
8 胃肠道反应：胃痛、恶心
9 出血风险：特别是长期服用
```

分块后：

```
代码块
1  Chunk 1: "阿司匹林是一种常用的解热镇痛药。它的主要作用包括：
2     1. 解热：降低发烧体温
3     2. 镇痛：缓解轻到中度疼痛
4     3. 抗血小板：预防血栓形成"
5
6  Chunk 2: "阿司匹林的主要作用包括预防血栓形成。但是，阿司匹林也有副作用。
7     常见的副作用包括：
8     1. 胃肠道反应：胃痛、恶心
9     2. 出血风险：特别是长期服用"
```

注意Chunk 2有重叠，这样即使用户搜"副作用"，也能同时看到"作用"相关的上下文。

2.4 向量化（Embedding）

分块后的文本还是人类语言，机器不懂。我们需要把它转换成**向量**（一串数字）。

什么是Embedding？

把文字转换成数字向量，相似的文字会得到相似的向量。

```
代码块
1 #使用OpenAI的Embedding模型
```

即通过手写的文本、图片、音频数据)

把文字转换成数字向量，相似的文字会得…

 飞书用户1871RQ 2月26日 16:46

相似向量

数值越接近1越相似

 用户578460 6月24日 23:28

1

向量搜索是相似性查询："找和[0.1, 0.3, 0…

 飞书用户1871RQ 2月26日 17:08

向量搜索是相似性查询

```
2 from openai import OpenAI
3 client = OpenAI()
4
5 text = "阿司匹林是一种解热镇痛药"
6 response = client.embeddings.create(
7     model="text-embedding-3-small",
8     input=text
9 )
10 vector = response.data[0].embedding
11 print(f"向量维度: {len(vector)}") # 输出: 1536
12 print(f"前5个值: {vector[:5]}") # 输出: [0.023, -0.014, 0.089, ...]
```

为什么需要向量化？

想象你在找"感冒药"，但文档里写的是"抗感冒药物"、"治疗感冒的药品"等各种说法。如果只靠关键词匹配，就找不到了。

但是！如果用向量化：

代码块

```
1 "感冒药" → [0.1, 0.3, 0.5, ...]
2 "抗感冒药物" → [0.12, 0.29, 0.51, ...] # 向量很接近！
3 "汽车" → [0.8, 0.1, 0.2, ...] # 向量差很远
```

向量之间可以计算**相似度**（余弦相似度），数值越接近1越相似：

代码块

```
1 from sklearn.metrics.pairwise import cosine_similarity
2 import numpy as np
3
4 vec1 = np.array([0.1, 0.3, 0.5])
5 vec2 = np.array([0.12, 0.29, 0.51])
6 similarity = cosine_similarity([vec1], [vec2])[0][0]
7 print(f"相似度: {similarity:.3f}") # 输出: 0.999 (非常相似)
```

常用的Embedding模型：

模型	维度	优势	适用场景
OpenAI text-embedding-3-small	1536	效果好，速度快	通用场景
OpenAI text-embedding-3-large	3072	效果最好	高精度需求
BGE-large-zh	1024	中文效果好	中文为主
Sentence-BERT	768	开源免费	资源有限

2.5 存储到向量数据库

最后，把这些向量存起来，用专门的**向量数据库**（Vector Database）。

为什么不用普通数据库？

普通数据库（MySQL、MongoDB）擅长精确查询："找ID=123的记录"。但向量搜索是**相似性查询**："找和[0.1, 0.3, 0.5]最相似的10个向量"。

| separators=["\n\n", "\n", "。", "!", " ", "? ...



用户490410 3月24日 17:46

- 第 1 优先：\n\n → 按段落分割
- 第 2 优先：\n → 按换行分割
- 第 3 优先：。 ! ? → 按句子结束符分割
- 第 4 优先：， → 按逗号分割（最后手段）



用户669719 7月10日 09:38
递归切分

| chunks = text_splitter.split_text(docum...

向量数据库用了特殊的索引算法（如HNSW、IVF），能在百万、千万级向量中毫秒级找到最相似的。

常用向量数据库：

- 1. Pinecone（云服务，简单好用）

代码块

```
1  import pinecone
2
3  pinecone.init(api_key="your-api-key")
4  index = pinecone.Index("my-rag-index")
5
6  #插入向量
7  index.upsert([
8      ("doc1_chunk1", vector1, {"text": "阿司匹林是..."}),
9      ("doc1_chunk2", vector2, {"text": "副作用包括..."})
10 ])
11
12 #查询
13 results = index.query(query_vector, top_k=3)
```

- 1. Milvus（开源，功能强大）
- 2. FAISS（Facebook开源，本地使用）
- 3. Weaviate（支持混合搜索）

完整的索引构建代码示例：

代码块

```
1  from langchain.text_splitter import RecursiveCharacterTextSplitter
2  from langchain.embeddings import OpenAIEmbeddings
3  from langchain.vectorstores import Pinecone
4  import pinecone
5
6  #1. 读取文档
7  with open("medical_docs.txt", "r") as f:
8      document = f.read()
9
10 #2. 分块
11 text_splitter = RecursiveCharacterTextSplitter(
12     chunk_size=500,
13     chunk_overlap=50,
14     separators=["\n\n", "\n", "。", "!", "?", ",", " "]
15 )
16 chunks = text_splitter.split_text(document)
17
18 #3. 初始化embedding模型
19 embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
20
21 #4. 初始化向量数据库
22 pinecone.init(api_key="your-key")
23 index_name = "medical-rag"
24
25 #5. 创建索引并存储
26 vectorstore = Pinecone.from_texts(
27     texts=chunks,
28     embedding=embeddings,
29     index_name=index_name
```



刘谋刚 7月13日 20:52

chunks 是个数组

| index_name=index_name



刘谋刚 7月13日 20:54 （编辑过）

给这个向量数据库取得名字

| "阿司匹林导致的胃部不适如何处理"（问…



王金磊 1月9日 17:42

我感觉这个问题改写很重要，有没有问题改写的技术介绍

```
30 )
31
32 print(f"成功索引了 {len(chunks)} 个文本块！")
```

运行这段代码，你的知识库就建好了！就像给图书馆做好了电子目录。

1.3 Retrieval（检索：快速找到相关知识）

索引建好后，用户来提问了。检索阶段的任务是：**从海量知识中快速找出最相关的那几条。**

检索流程详解 (Retrieval Process)



3.1 查询处理

用户输入的问题往往不是完美的搜索词。比如：

代码块

```
1 用户原始问题："我吃了阿司匹林后胃疼怎么办？"
2
3 需要优化为：
4 "阿司匹林 胃痛 副作用"（关键词提取）
5 "阿司匹林导致的胃部不适如何处理"（问题改写）
```

3.2 向量检索

GS 飞书用户5243GS 2月14日 21:53
+1

彭绍湧 4月30日 11:06
这个主要是要根据自己的业务需求来，不同的业务场景做不同的改写

易鹏 5月15日 10:54
这个改写时机是啥样的，什么情况改写

用户948382 5月29日 13:19
我感觉我需要看下问题改写的技术

用户290615 6月3日 15:13
这个改写是怎样做的，怎么能理解用户原始问题，然后改写成向量数据库相似度更高的方案？

HY 飞书用户6323HY 6月9日 23:09
好问题

把用户问题也转换成向量，然后在向量数...

飞书用户1871RQ 2月26日 17:16
用户问题转换成向量，在向量数据库中搜索。

alpha=0.5

用户894510 6月4日 14:06
向量搜索和关键词搜索的权重各占一半

把用户问题也转换成向量，然后在向量数据库中搜索：

代码块

```
1  #用户问题
2  question = "阿司匹林有哪些副作用？"
3
4  #问题向量化
5  question_embedding = embeddings.embed_query(question)
6
7  #向量检索 (找最相似的3个)
8  results = vectorstore.similarity_search_by_vector(
9      embedding=question_embedding,
10     k=3 # 返回top3
11 )
12
13 for i, doc in enumerate(results):
14     print(f"结果{i+1}:")
15     print(doc.page_content)
16     print(f"相似度: {doc.metadata['score']}")
17     print("-" * 50)
```

输出示例：

代码块

```
1  结果1:
2  阿司匹林的常见副作用包括：1. 胃肠道反应：胃痛、恶心、消化不良...
3  相似度：0.89
4  结果2:
5  长期服用阿司匹林可能增加出血风险，特别是胃肠道出血...
6  相似度：0.85
7  结果3:
8  少数患者对阿司匹林过敏，可能出现荨麻疹、呼吸困难等症状...
9  相似度：0.81
```

3.3 检索策略

检索系统通常采用混合搜索方法，结合向量搜索（找语义相似的文档）和关键词搜索（精确匹配），然后对结果进行排序和过滤。

1. 单纯向量搜索

- 优点：能理解语义
- 缺点：对专有名词、数字等不敏感

2. 混合搜索（Hybrid Search）

- 向量搜索 + 关键词搜索
- 综合排序，取最优结果

代码块

```
1  #混合搜索示例
2  def hybrid_search(query, alpha=0.5):
3      # alpha: 向量搜索权重 (0-1)
4      # 向量搜索结果
5      vector_results = vectorstore.similarity_search(query, k=10)
6      # 关键词搜索结果 (BM25算法)
```

| query, k=10



用户894510 6月4日 14:05
用户问题，返回与用户问题最相关的 10 篇文档

| BM25算法



小麦 2月28日 20:32
信息检索领域最经典的概率相关性排序算法



Chao Chen 5月7日 01:47
需要 import 包吗



张鸿钦 7月20日 19:01



| final_results = merge_results(vector_re...



用户894510 6月4日 14:13
两路合并，重新排序



用户758413 6月15日 11:58
有合并函数的定义吗

| 返回top3



用户894510 6月4日 14:12
取最相关的三篇文档

```
7 keyword_results = bm25_search(query, k=10)
8 # 融合排序
9 final_results = merge_results(vector_results, keyword_results,
    alpha)
10 return final_results[:3] # 返回top3
```

3. 重排序 (Re-ranking)

初步检索后，用更精细的模型重新排序，提高精度。

代码块

```
1 from sentence_transformers import CrossEncoder
2
3 #加载重排序模型
4 reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')
5
6 #对检索结果重新打分
7 query = "阿司匹林副作用"
8 candidate_docs = ["文档1内容", "文档2内容", "文档3内容"]
9
10 scores = reranker.predict([(query, doc) for doc in candidate_docs])
11
12 #按分数排序
13 ranked_docs = [doc for _, doc in sorted(zip(scores,
    candidate_docs), reverse=True)]
```

3.4 检索效果评估指标

怎么知道检索效果好不好？

指标	含义	公式	目标
Recall@K	前K个结果中，包含多少相关文档	检索到的相关文档数 / 总相关文档数	越高越好
Precision@K	前K个结果中，有多少是相关的	相关文档数 / K	越高越好
MRR	第一个相关文档的排名倒数	1 / 第一个相关文档的排名	越高越好
NDCG	考虑排序质量的综合指标	复杂公式	越高越好

1.4 Generation（生成：基于知识产生答案）

检索到相关文档后，最后一步就是生成答案！

把检索到的文档和用户问题组合成一个完…



飞书用户1871RQ 2月27日 16:14

完整提示词

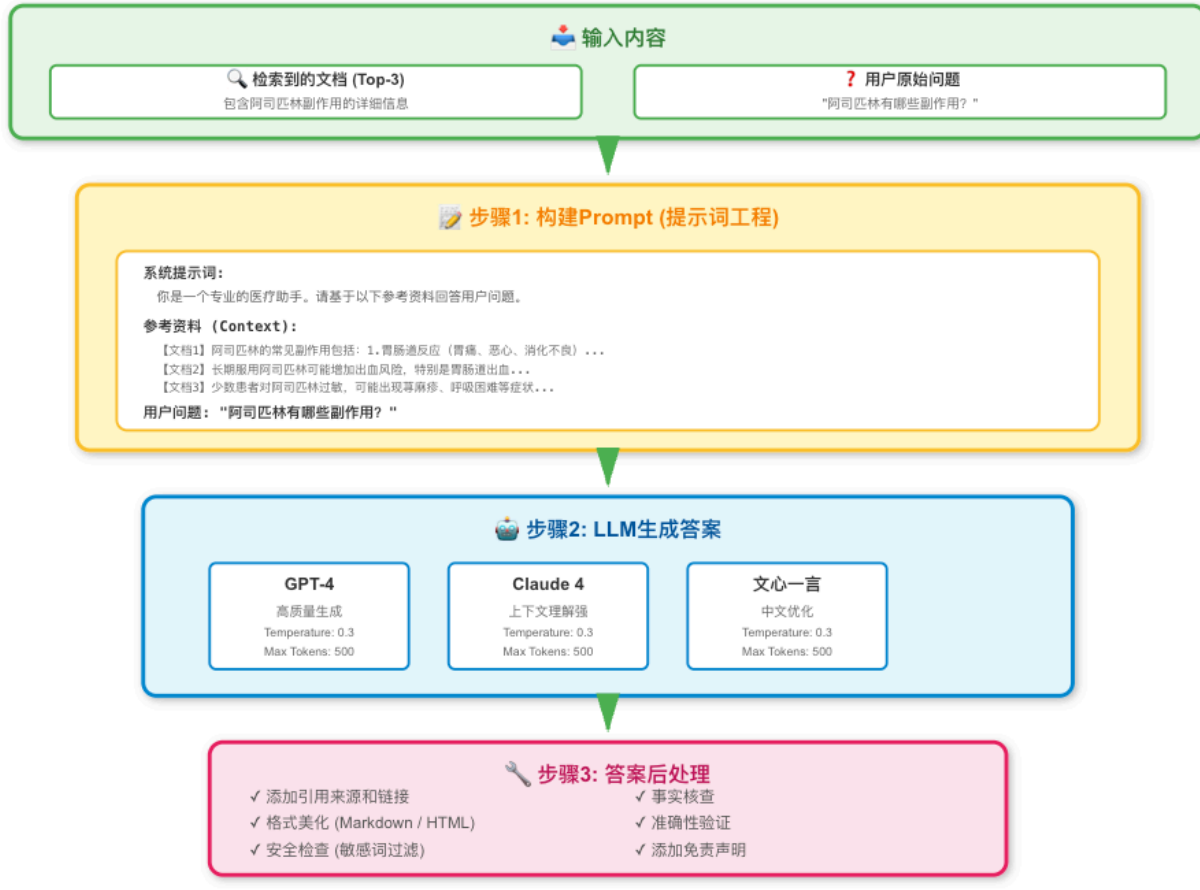
"\n\n".join([doc.page_content for doc i…



刘谋刚 7月13日 20:59

示例输出：["文档 1 内容","文档 2 内容","文档 3 内容"]，因为赋值是在最后执行得呢。

答案生成流程 (Generation Process)



4.1 提示词构建 (Prompt Engineering)

把检索到的文档和用户问题组合成一个完整的提示词 (Prompt), 发给LLM。

基础模板:

代码块

```
1  prompt_template = """
2  你是一个专业的医疗助手。请基于以下参考资料回答用户的问题。
3  如果参考资料中没有相关信息, 请明确告知用户。
4
5  参考资料:
6  {context}
7
8  用户问题:
9  {question}
10
11  请提供详细、准确的回答, 并标注信息来源。
12  """
13
14  #填充内容
15  context = "\n\n".join([doc.page_content for doc in retrieved_docs])
16  question = "阿司匹林有哪些副作用?"
17
18  prompt = prompt_template.format(context=context, question=question)
```

进阶模板 (结构化输出):

代码块

```
1  advanced_prompt = """
2  参考资料：
3  {context}
4
5  用户问题：{question}
6
7  请按以下格式回答：
8  直接回答：用1-2句话概括答案
9  详细说明：展开解释，分点列出
10  注意事项：相关的警告或建议
11  信息来源：标注参考了哪些资料
12
13  回答：
14  """
```

4.2 调用LLM生成

代码块

```
1  from openai import OpenAI
2
3  client = OpenAI()
4
5  response = client.chat.completions.create(
6      model="gpt-4",
7      messages=[
8          {"role": "system", "content": "你是一个专业的医疗助手"},
9          {"role": "user", "content": prompt}
10     ],
11     temperature=0.3, # 降低随机性，提高准确性
12     max_tokens=500
13 )
14
15 answer = response.choices[0].message.content
16 print(answer)
```

4.3 答案后处理

生成答案后，可能还需要：

1. 引用标注：给答案加上来源链接

代码块

```
1  def add_citations(answer, sources):
2      """给答案添加引用"""
3      for i, source in enumerate(sources):
4          answer += f"\n[{i+1}] {source['title']} - {source['url']}"
5      return answer
```

1. 安全过滤：检查答案是否包含有害内容

代码块

```
1  def safety_check(answer):
2      harmful_keywords = ["自杀", "违法", "..."]
3      for keyword in harmful_keywords:
4          if keyword in answer:
```

| context = "\n\n".join([f"【文档{i+1}】 {…



刘谋刚 7月13日 21:03

数组这个 context 存储得是个别数组里面

| 效果评估



用户923563 6月16日 15:30

效果评估有没有一套最常见的方案呢

```
5         return "抱歉，这个问题涉及敏感内容，建议咨询专业人士。"
6     return answer
```

1. 格式美化：Markdown格式化、表格化等

4.4 完整的RAG生成代码

代码块

```
1 def rag_query(question):
2     """完整的RAG查询流程"""
3     # 1. 检索相关文档
4     retrieved_docs = vectorstore.similarity_search(question, k=3)
5     # 2. 构建prompt
6     context = "\n\n".join([
7         f"【文档{i+1}】 {doc.page_content}"
8         for i, doc in enumerate(retrieved_docs)
9     ])
10    prompt = f"""
11    参考以下资料回答问题：
12    {context}
13    问题: {question}
14    要求：
15    1. 回答要准确、专业
16    2. 必须基于参考资料
17    3. 标注信息来源
18    """
19    # 3. 调用LLM生成
20    response = client.chat.completions.create(
21        model="gpt-4",
22        messages=[{"role": "user", "content": prompt}],
23        temperature=0.3
24    )
25    answer = response.choices[0].message.content
26    # 4. 添加引用
27    sources = [
28        {"title": f"文档{i+1}", "score": doc.metadata.get('score',
290)}
31        for i, doc in enumerate(retrieved_docs)
32    ]
33    return {
34        "answer": answer,
35        "sources": sources,
36        "retrieved_docs": [doc.page_content for doc in
37retrieved_docs]
38    }
39
40 #使用示例
41 result = rag_query("阿司匹林有哪些副作用? ")
42 print("答案:", result['answer'])
43 print("\n参考来源:", result['sources'])
```

4.5 生成效果评估

RAG系统的生成质量评估需要考虑准确性、相关性和忠实度等多个维度，确保模型生成的答案既准确又能体现检索到的上下文信息。

指标	评估内容	评估方法
Faithfulness	答案是否忠实于检索文档	LLM评判 / 人工标注
Relevance	答案是否回答了问题	LLM评判 / 相似度计算
Coherence	答案是否流畅连贯	语言模型困惑度
Groundedness	答案是否有依据	检查是否有引用

最近在做大模型项目的时候，发现很多同学对RAG的理解还停留在"检索+生成"这个表面概念上。但实际上，RAG是一个**超级复杂的系统工程**，涉及到的优化点多达几十个。

就像你开了一家餐厅，不是说有食材（数据）和厨师（大模型）就够了，你还得考虑：

- 食材怎么切（Chunking）？
- 调料怎么配（Embedding）？
- 菜单怎么设计（Query优化）？
- 上菜顺序怎么安排（Re-ranking）？
- 顾客满意度怎么评估（效果评估）？

二、RAG查询优化策略

前言

先说个真实场景：用户问"阿司匹林怎么吃？"

你的RAG系统可能只检索到"阿司匹林 用法"相关的文档，但其实用户真正想问的可能是：

- 阿司匹林的**剂量**是多少？
- 什么**时间**吃效果最好？
- 有什么**禁忌**和**副作用**？
- **哪些人群**不适合吃？

看到了吧？一个简单的问题背后，其实藏着好几个子问题。如果你只用原始问题去检索，很可能漏掉关键信息。

这就是为什么要优化提问！ 通过对用户问题进行改写、扩展、分解，我们可以：

1.  提高检索的召回率（找到更多相关内容）
2.  覆盖问题的多个维度
3.  减少歧义和理解偏差

1.5 Multi Query - 多查询策略

核心思想

一个问题，多种问法。

就像你去问路，同一个目的地，可以这样问：

multi_queries



刘谋刚 7月14日 10:20

是一个数组列表，
multi_queries=["代码性能优化技巧","提升程序运行速度的方法","如何让代码跑得更快","代码执行效率优化最佳实践"]

增加检索时间



用户721911 7月5日 22:20

。

- "天安门怎么走？"
- "去天安门广场的路线？"
- "从这里到天安门的交通方式？"

虽然表达不同，但目的一样。Multi Query就是让大模型生成多个语义相似但表达不同的查询，然后用这些查询去检索，最后合并结果。

工作流程

1. **输入原始问题：**用户问"Python如何处理JSON数据？"
2. LLM生成多个查询
3. **:**
 - Query 1: "Python解析JSON的方法"
 - Query 2: "如何在Python中读取JSON文件"
 - Query 3: "Python JSON模块使用教程"
 - Query 4: "Python处理JSON格式数据的最佳实践"
4. **并行检索：**用这4个查询同时去向量数据库检索
5. **结果合并：**把4次检索的结果去重、排序，得到最终结果

优势分析

优势	说明
 提高召回率	不同表达方式能匹配到更多相关文档
 覆盖多角度	从不同角度理解同一个问题
 鲁棒性强	即使某个查询效果不好，其他查询也能补救

实际案例

代码块

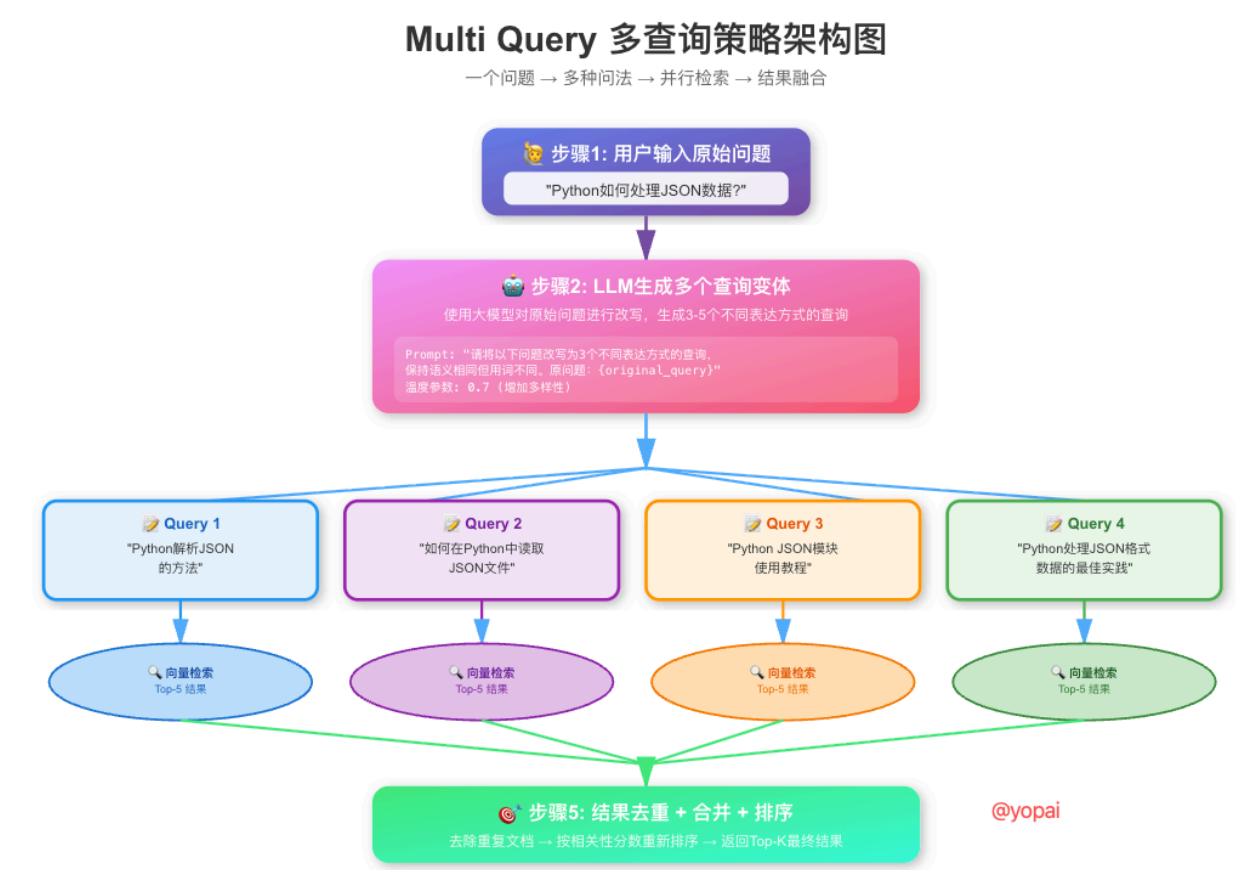
```
1  #伪代码示例
2  original_query = "如何提高代码执行效率？"
3
4  #LLM生成多个查询
5  multi_queries = llm.generate_queries(original_query, num_queries=4)
6  #输出：
7  #["代码性能优化技巧",
8  #"提升程序运行速度的方法",
9  #"如何让代码跑得更快",
10 #"代码执行效率优化最佳实践"]
11
12 #并行检索
13 all_results = []
14 for query in multi_queries:
15     results = vector_db.search(query, top_k=5)
16     all_results.extend(results)
17
18 #去重合并
19 final_results = deduplicate_and_rank(all_results)
```


注意事项

⚠️ 不是查询越多越好！一般3-5个查询就够了，太多会：

- 增加API调用成本
- 增加检索时间
- 可能引入噪声

架构图



从图中可以清晰看到：

1. 原始问题经过LLM改写生成4个不同表达的查询
2. 每个查询独立进行向量检索，获取Top-5结果
3. 最后将4组结果合并、去重、重新排序

1.6 RAG-Fusion - 多查询结果融合策略

核心思想

RAG-Fusion是Multi Query的进化版，不仅生成多个查询，还使用了**倒数排序融合（Reciprocal Rank Fusion, RRF）** 算法来合并结果。

简单说：**不是简单粗暴地把结果堆一起，而是科学地给每个结果打分，让真正重要的文档排在前面。**

什么是RRF？

举个例子，假设你问3个朋友推荐餐厅：

RRF算法

星海 王星海 7月8日 15:54
公式不清楚

对于每个餐厅，计算它在各个列表中的倒…

陆志伟 1月16日 16:58
请问这部分公式在哪里

用户688035 2月2日 12:09

问题：如果有一个文档在多个检索结果中都被检索到，如何计算每个文档的综合得分？
公式： $RRF_k = \frac{1}{k} \sum_{i=1}^k \frac{1}{rank_i(D)}$
解释：文档 D 在第 i 个检索结果列表中的排名为 rank_i(D)，用于该排名在倒数排名列表中对应的值取倒数并求和，最后再除以 k 得到文档 D 的综合得分 RRF_k。

看到了吧？外婆家虽然没拿过第1名，但…

谭雅淇 7月6日 17:07
朋友 B 外婆家不是第一名吗

朋友A的推荐排序：

- 1. 海底捞
- 2. 外婆家
- 3. 西贝

朋友B的推荐排序：

- 1. 外婆家
- 2. 海底捞
- 3. 必胜客

朋友C的推荐排序：

- 1. 西贝
- 2. 外婆家
- 3. 肯德基

如果用**简单投票**，海底捞2票，外婆家3票，西贝2票... 但这样有问题：外婆家虽然被提到3次，但在A那里是第2名，不是第1名。

RRF算法会这样算：

对于每个餐厅，计算它在各个列表中的倒数排名分数：

其中：

- 是文档（餐厅）
- 是所有查询结果列表
- 是文档在列表r中的排名
- 是常数（通常取60）

计算海底捞的分数：

- 在A的列表：排名1 →
- 在B的列表：排名2 →
- 在C的列表：没出现 →
- **总分：0.0325**

计算外婆家的分数：

- 在A的列表：排名2 →
- 在B的列表：排名1 →
- 在C的列表：排名2 →
- **总分：0.0486**

计算西贝的分数：

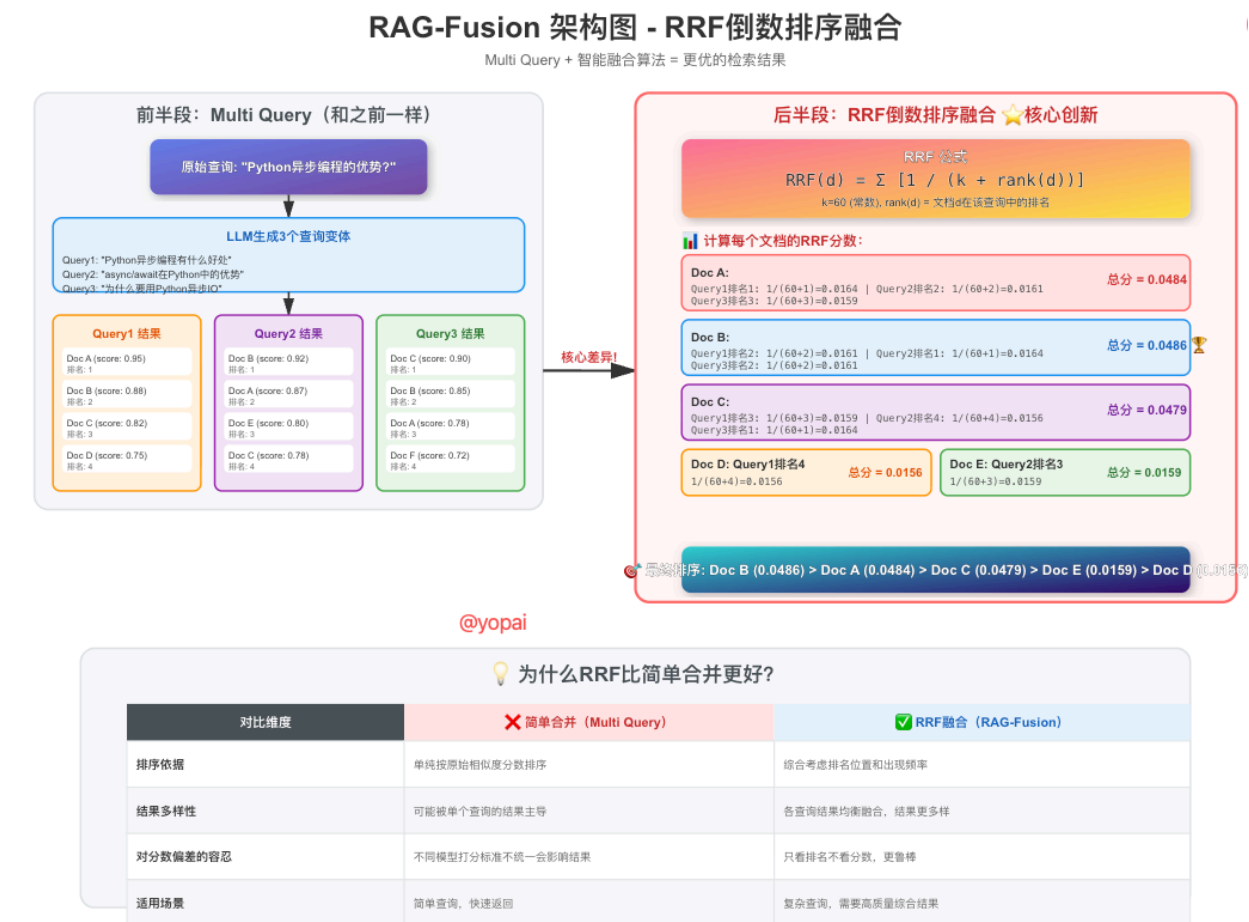
- 在A的列表：排名3 →
- 在B的列表：没出现 →
- 在C的列表：排名1 →
- **总分：0.0323**

最终排序：外婆家(0.0486) > 海底捞(0.0325) > 西贝(0.0323)

看到了吧？外婆家虽然没拿过第1名，但因为**出现频率高且排名都靠前**，所以综合得分最高！

RAG-Fusion工作流程

- 1. 生成多个查询（和Multi Query一样）
- 2. 并行检索（和Multi Query一样）
- 3. 使用RRF算法融合结果（这是关键！）
- 4. 返回重新排序后的Top-K文档



为什么RRF这么有效?



飞书用户1871RQ 2月27日 16:31 (…

RRF：位置敏感、频率奖励、公平、鲁棒性



用户703550 3月24日 20:54

对的

C2 第1名和第2名的差距 > 第10名和第11名



用户894510 6月4日 17:08

倒数排序，排名靠前增益更大

代码示例

代码块

```
1 def reciprocal_rank_fusion(search_results_dict, k=60):
2     """
3     使用倒数排序融合算法合并多个搜索结果
4     Args:
5         search_results_dict: {query: [(doc_id, score), ...]}
6         k: RRF常数，默认60
7     Returns:
8         融合后的排序结果
9     """
10    fused_scores = {}
11    for query, doc_scores in search_results_dict.items():
12        for rank, (doc_id, score) in enumerate(doc_scores,
13start=1):
14            if doc_id not in fused_scores:
15                fused_scores[doc_id] = 0
16            # RRF公式
17            fused_scores[doc_id] += 1 / (k + rank)
```

```
17     # 按融合分数降序排序
18     reranked_results = sorted(
19         fused_scores.items(),
20         key=lambda x: x[1],
21         reverse=True
22     )
23     return reranked_results
24
25 #使用示例
26 search_results = {
27     "query1": [("doc1", 0.95), ("doc2", 0.88), ("doc3", 0.82)],
28     "query2": [("doc2", 0.92), ("doc1", 0.87), ("doc4", 0.80)],
29     "query3": [("doc3", 0.90), ("doc2", 0.85), ("doc1", 0.78)]
30 }
31
32 final_ranking = reciprocal_rank_fusion(search_results)
33 print(final_ranking)
34 #输出: [('doc2', 0.0486), ('doc1', 0.0479), ('doc3', 0.0320),
35        ('doc4', 0.0161)]
```

为什么RRF这么有效？

特点	说明	举例
 位置敏感	排名越靠前，分数增益越大	第1名和第2名的差距 > 第10名和第11名
 频率奖励	出现次数多的文档会累积更高分	被3个查询都检索到的文档会加分
 公平性	不依赖原始相似度分数，避免不同模型打分偏差	不同embedding模型的分数可以直接比较
 鲁棒性	即使某个查询效果差，也不会严重影响整体	一个差查询的影响会被稀释

实际效果对比

假设我们要回答："Python异步编程的优势是什么？"

普通Multi Query（简单合并）：

- 结果包含很多重复文档
- 排序不一定科学
- Top-5可能都来自同一个查询

RAG-Fusion（RRF融合）：

- 去重且智能排序
- 综合考虑所有查询的反馈
- Top-5结果更多样化、更全面

注意事项

✔ 适用场景：

- 用户问题比较复杂，需要多角度检索
- 对召回率要求高的场景
- 希望结果多样性的场景

很多用户的问题其实是复合型问题，包含…



飞书用户1871RQ 2月27日 16:33

了解真实的需求。

子问题列表 = ["1. RAG系统的核心组件有…



刘谋刚 7月14日 10:21

分解后的样子是个数组列表

✖ 不适用场景：

- 简单的事实查询（浪费资源）
- 实时性要求极高的场景（会增加延迟）
- 资源受限的环境（多次LLM调用 + 多次检索）

1.7 Decomposition - 问题分解策略

核心思想

把一个复杂问题拆解成多个简单的子问题，逐个击破！

就像你要做一道菜"红烧肉"，你不会一步到位，而是：

- 先准备食材（买肉、调料）
- 处理食材（切块、焯水）
- 烹饪步骤（煎、炖、收汁）
- 摆盘装饰

每一步都简单明确，组合起来就是一道完美的菜。

为什么需要问题分解？

很多用户的问题其实是**复合型问题**，包含多个信息需求。

举个例子：

用户问："我想知道北京和上海的房价对比，以及各自的购房政策和未来发展趋势。"

这个问题其实包含了：

- 北京的房价水平
- 上海的房价水平
- 北京的购房政策
- 上海的购房政策
- 北京的未来发展趋势
- 上海的未来发展趋势

如果你直接用原始问题去检索，可能会：

- ✖ 检索结果太泛，不精准
- ✖ 有些子问题的答案被遗漏
- ✖ 大模型在综合回答时逻辑混乱

使用问题分解后：

- ✅ 每个子问题独立检索，精准度更高
- ✅ 覆盖全面，不会遗漏信息
- ✅ 最后综合所有子答案，回答更有条理

工作流程

代码块

- 原始问题
- ↓
- LLM分解为子问题
- ↓
- 并行检索每个子问题



实际案例

原始问题： "如何搭建一个高性能的RAG系统？ 需要考虑哪些技术选型和优化策略？ "

LLM分解后：

```
代码块

1  子问题列表 = [
2      "1. RAG系统的核心组件有哪些？ ",
3      "2. 如何选择合适的向量数据库？ ",
4      "3. Embedding模型如何选型？ ",
5      "4. 文档分块(Chunking)的最佳实践是什么？ ",
6      "5. 如何优化检索性能？ ",
7      "6. 生成阶段有哪些优化技巧？ "
8  ]
```

针对每个子问题进行检索和回答：

子问题	检索到的关键信息	子答案
1. 核心组件	数据处理、向量化、存储、检索、生成	RAG系统包含5大模块...
2. 向量数据库选型	Pinecone、Milvus、FAISS对比	根据规模选择：小规模用FAISS...
3. Embedding模型	OpenAI、BGE、M3E性能对比	中文场景推荐BGE-large-zh...
4. Chunking策略	固定长度、语义分块、滑动窗口	建议使用语义分块+重叠...
5. 检索优化	Re-ranking、混合检索、缓存	可以采用Re-ranking提升精度...
6. 生成优化	Prompt工程、温度调节、流式输出	设计明确的Prompt模板...

最终综合答案： "搭建高性能RAG系统需要从以下几个方面考虑：首先，系统包含数据处理、向量化、存储、检索、生成5大核心模块。在技术选型上，小规模项目可以用FAISS作为向量数据库，中大规模建议Milvus或Pinecone； Embedding模型中文场景推荐BGE-large-zh。在优化策略上，Chunking建议采用语义分块并设置50-100 token重叠；检索阶段可以引入Re-ranking提升精度；生成阶段要设计明确的Prompt模板并适当调节温度参数..."

看到了吧？ 通过分解，我们得到了一个结构清晰、内容全面的答案！

代码实现

```
代码块

1  from langchain.llms import OpenAI
2  from langchain.prompts import PromptTemplate
3
4  #步骤1： 问题分解
```

sub_questions = llm(decompose_pro...

 刘谋刚 7月13日 21:20

sub_questions 又是一个数组

sub_questions = json.loads(sub_questi...

 刘谋刚 7月13日 21:22

把模型返回的 JSON 文本转成 Python 字符串列表，如["子问题 1","子问题 2"]

relevant_docs

 刘谋刚 7月14日 10:17

这个也是数组就是[文档 1] [文档 2] [文档 3]的内容

{sub_q}

 刘谋刚 7月14日 10:25

就是由这个 sub_q 变量传过来的呢

 刘谋刚 7月14日 10:25

 刘谋刚 7月14日 10:25

 刘谋刚 7月14日 10:25

 刘谋刚 7月14日 10:25

并行分解（Parallel Decomposition）

 用户999497 2月3日 22:33

序号乱了

```
5  decompose_prompt = PromptTemplate(
6      template="""
7      请将以下复杂问题分解为3-6个简单的子问题。
8      每个子问题应该独立且可以单独回答。
9      原始问题: {question}
10     请以JSON列表格式输出子问题:
11     ["子问题1", "子问题2", "子问题3", ...]
12     """,
13     input_variables=["question"]
14 )
15
16 llm = OpenAI(temperature=0.7)
17
18 original_question = "如何搭建一个高性能的RAG系统? 需要考虑哪些技术选型和优化策略? "
19
20 #分解问题
21 sub_questions =
22     llm(decompose_prompt.format(question=original_question))
23 sub_questions = json.loads(sub_questions)
24
25 #步骤2: 对每个子问题进行RAG检索和回答
26 sub_answers = []
27 for sub_q in sub_questions:
28     # 检索相关文档
29     relevant_docs = vector_db.search(sub_q, top_k=3)
30     # 生成子答案
31     answer_prompt = f"""
32     基于以下文档, 回答问题: {sub_q}
33     文档内容:
34     {relevant_docs}
35     请简洁明确地回答:
36     """
37     sub_answer = llm(answer_prompt)
38     sub_answers.append({
39         "question": sub_q,
40         "answer": sub_answer
41     })
42
43 #步骤3: 综合所有子答案
44 synthesis_prompt = f"""
45 你是一个专业的技术专家。现在你需要基于以下子问题和对应的答案,
46 综合生成一个完整、有条理的回答。
47
48 原始问题: {original_question}
49
50 子问题和答案:
51 {json.dumps(sub_answers, ensure_ascii=False, indent=2)}
52
53 请生成一个结构清晰、逻辑连贯的最终答案:
54 """
55 final_answer = llm(synthesis_prompt)
56 print(final_answer)
```

三种常见的分解策略

7. 顺序分解 (Sequential Decomposition)

适用场景: 问题之间有先后依赖关系

例子: "如何训练一个GPT模型并部署到生产环境? "

分解为：

- 1. 准备训练数据 →
- 2. 选择模型架构 →
- 3. 训练模型 →
- 4. 模型评估 →
- 5. 模型优化 →
- 6. 部署上线

每一步的答案会作为下一步的输入或参考。

7. 并行分解 (Parallel Decomposition)

适用场景： 子问题之间相互独立

例子： "比较PyTorch和TensorFlow的优缺点"

分解为：

- PyTorch的优点
- PyTorch的缺点
- TensorFlow的优点
- TensorFlow的缺点

这些子问题可以**同时检索**，提高效率。

8. 层次分解 (Hierarchical Decomposition)

适用场景： 问题有多个层级

例子： "如何设计一个电商推荐系统？"

第一层分解：

- 数据层
- 算法层
- 工程层

第二层分解：

- 数据层 → 数据采集、数据清洗、特征工程
- 算法层 → 召回算法、排序算法、实时更新
- 工程层 → 系统架构、性能优化、监控告警

优势分析

优势	说明
 精准检索	每个子问题更聚焦，检索准确度提升
 结构化回答	答案有条理，逻辑清晰
 覆盖全面	不会遗漏关键信息点
 减轻LLM负担	分而治之，每次生成任务更简单
 可并行处理	对于独立的子问题，可以并行检索加速

注意事项

2023年10月发布的那个新的React框架叫...



用户923563 6月16日 17:24

这种直接检索 检索不到吗？提问越具体不是越好吗？再回到抽象层是不是有点多余



谭雅淇 7月6日 18:01

@用户 923563 我感觉也是这样，这个例子举的不太好



梁喆 7月10日 16:39

直接搜索可能数据库里并没有那么多限制条件，比如有很多React,但是问题包含了时间，可能就搜索不出来。回退后通过逻辑模型交叉验证

⚠️ 不是所有问题都需要分解！

适合分解的问题：

- 包含多个独立信息需求
- 需要对比分析
- 需要多步骤解答
- 问题比较复杂抽象

不适合分解的问题：

- 简单的事实查询 ("Python之父是谁? ")
- 单一明确的问题 ("2+2等于几? ")
- 定义类问题 ("什么是RAG? ")

💡 分解的粒度要适中：

- 太粗：失去分解的意义
- 太细：增加系统复杂度和延迟

1.8 Step Back问答回退策略

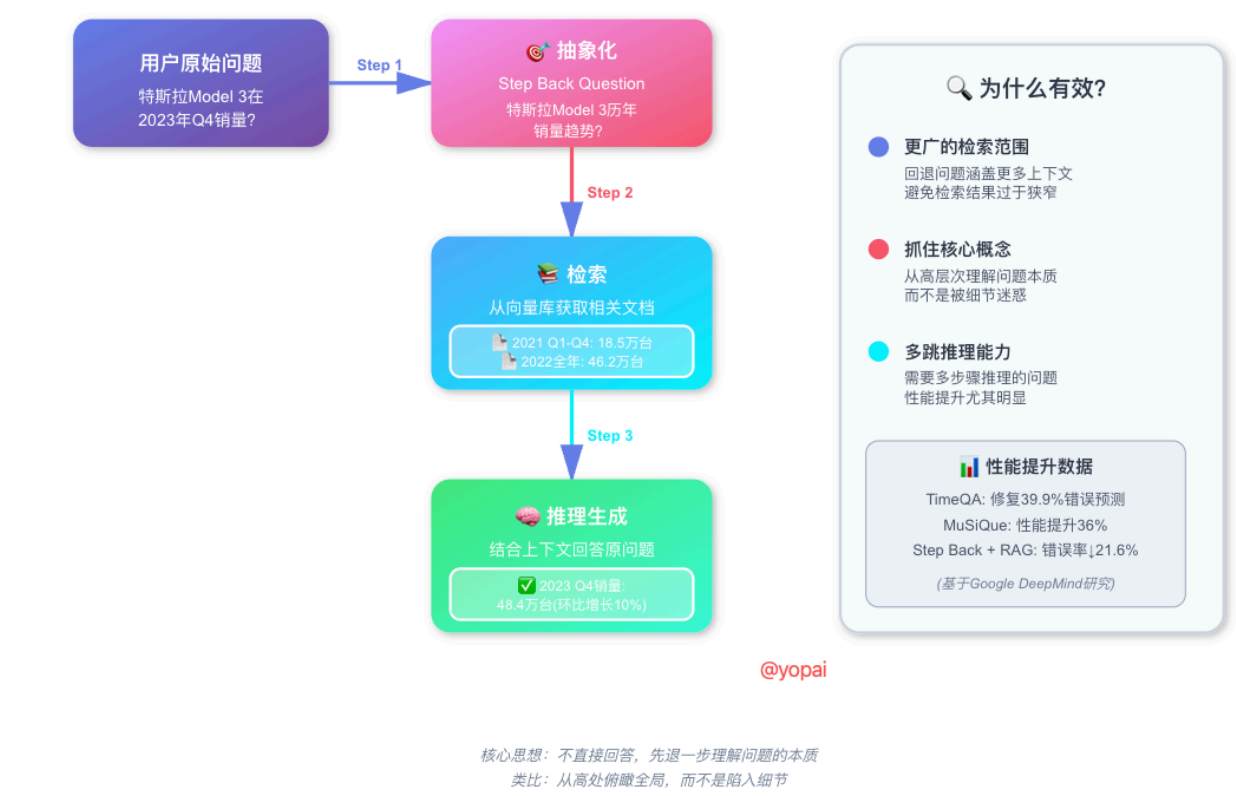
什么是Step Back?

想象一下，你在图书馆找书，直接冲过去问管理员："2023年10月发布的那个新的React框架叫什么？"管理员一脸懵逼。但如果你先退一步问："最近有哪些新的React框架？"然后再缩小范围，是不是容易多了？

Step Back Prompting就是这个道理——不直接回答具体问题，而是先生成一个更抽象、更通用的"回退问题"，从更高层次理解用户意图，然后再回答原问题。

工作原理

Step Back Prompting 工作流程



整个流程分三步走：

步骤1：抽象化（Abstraction）

代码块

```
1  原问题：特斯拉Model 3在2023年Q4的销量是多少？
2  ↓ Step Back
3  回退问题：特斯拉Model 3历年的销量趋势是怎样的？
```

步骤2：检索（Retrieval） 用回退问题去检索，能获取更广泛、更有上下文的信息。

步骤3：推理（Reasoning） 结合回退问题的答案和原问题，生成更准确的回答。

代码实现

代码块

```
1  from langchain_openai import ChatOpenAI
2  from langchain_core.prompts import ChatPromptTemplate
3
4  #Step 1: 定义Step Back提示词模板
5  step_back_template = """你是一个世界知识专家。你的任务是把具体问题转化为更通用的回退问题。
6
7  示例：
8  原问题：特斯拉Model 3在2023年Q4的销量是多少？
9  回退问题：特斯拉Model 3历年的销量趋势和数据有哪些？
```

B2 "2023年美国有多少位总统出生在美国…

用户655341 7月12日 17:18

Are u kidding me，2023 年有多少总统出生？？？

为啥这样能work？因为答案和答案之间…

用户369139 3月15日 14:49

Similarity

```
10
11 原问题：张三在2020-2022年担任什么职位？
12 回退问题：张三的职业生涯发展轨迹是怎样的？
13
14 现在请处理这个问题：
15 原问题：{original_question}
16 回退问题：""
17
18 llm = ChatOpenAI(model="gpt-4", temperature=0.3)
19 step_back_prompt =
    ChatPromptTemplate.from_template(step_back_template)
20
21 #Step 2: 生成回退问题
22 def generate_step_back_question(original_q):
23     chain = step_back_prompt | llm
24     response = chain.invoke({"original_question": original_q})
25     return response.content
26
27 #Step 3: 使用回退问题进行RAG检索
28 from langchain.vectorstores import FAISS
29 from langchain.embeddings import OpenAIEmbeddings
30
31 def step_back_rag(original_question, vectorstore):
32     # 生成回退问题
33     step_back_q = generate_step_back_question(original_question)
34     print(f"📝 回退问题: {step_back_q}")
35     # 用回退问题检索
36     docs = vectorstore.similarity_search(step_back_q, k=5)
37     context = "\n\n".join([doc.page_content for doc in docs])
38     # 最终回答
39     final_prompt = f"""基于以下上下文信息，回答问题。
40
41 上下文：
42 {context}
43
44 回退问题: {step_back_q}
45 原问题: {original_question}
46
47 请给出准确、详细的回答："""
48     response = llm.invoke(final_prompt)
49     return response.content
50
51 #使用示例
52 question = "DeepSeek在2024年1月发布的模型性能如何？"
53 answer = step_back_rag(question, my_vectorstore)
54 print(f"✅ 答案: {answer}")
```

实际效果对比

方法	问题	检索到的文档质量	答案准确度
直接RAG	"2023年美国有多少位总统出生在美国？"	★★★★	漏掉富兰克林·罗斯福
Step Back RAG	先问"美国历任总统的出生地信息"	★★★★★★	100%准确

适用场景

✔ 非常适合：

- 需要多步推理的复杂问题
- 时间序列相关查询 ("最近"、"历年"、"趋势")
- 需要理解高层概念的问题

✘ 不太适合：

- 简单的事实查询 ("北京是中国的首都吗？")
- 需要实时数据的场景
- 计算密集型任务

1.9 HyDE（假设性文档嵌入）

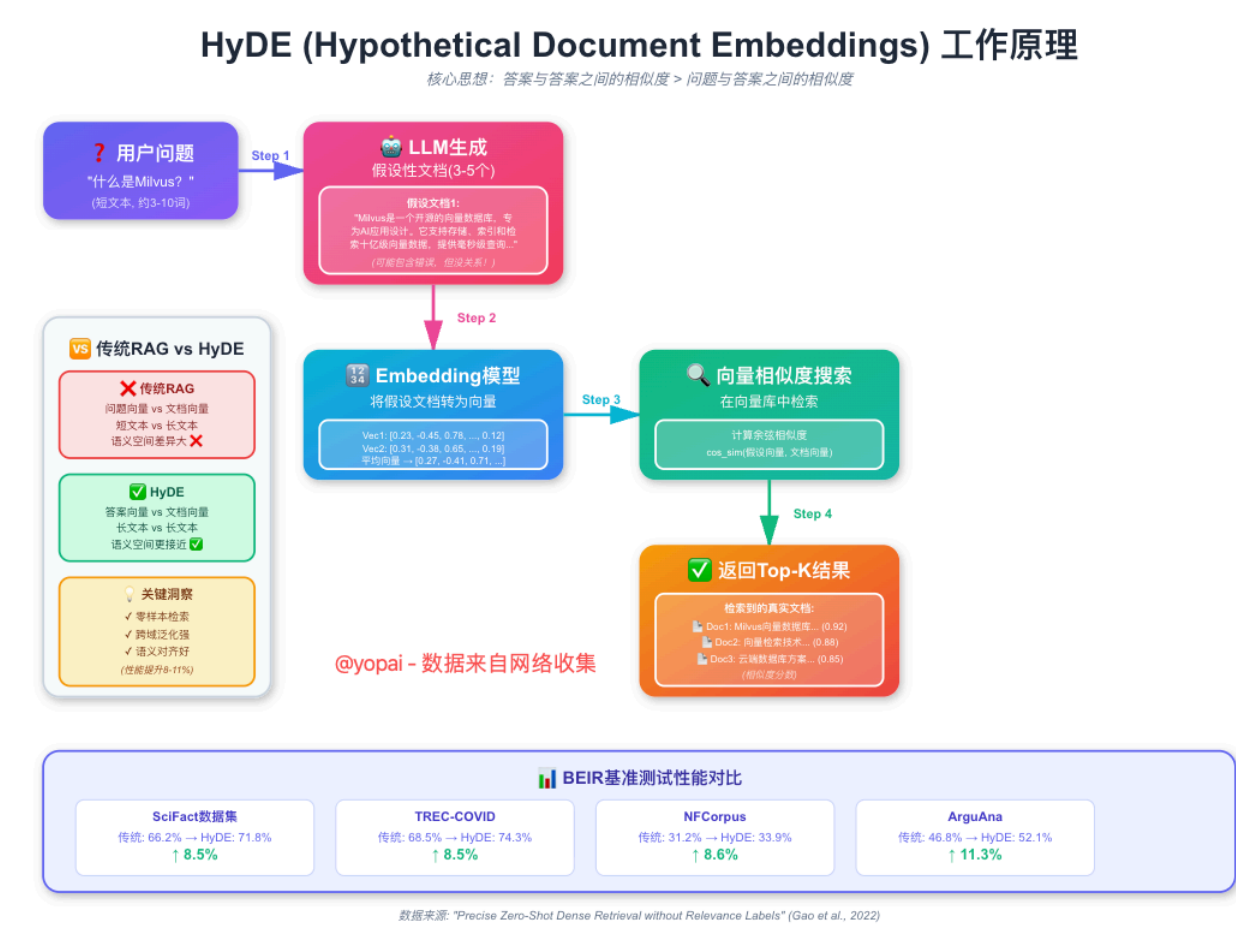
核心思想

HyDE的脑洞真的很大！它不是直接拿用户问题去搜索，而是：

1. 让LLM先"编"一个假的答案（可能包含错误，但没关系）
2. 把这个假答案转成向量
3. 用假答案的向量去搜索真实文档

为啥这样能work？因为答案和答案之间的相似度，远高于问题和答案之间的相似度！

工作流程图



问题的本质

传统RAG有个致命缺陷：**查询-文档不对称**

代码块

```
1  用户问题：Milvus是什么？ [向量维度：1536]
2      ↓ 余弦相似度 ↓
3  文档内容：Milvus是一个开源向量数据库，专为AI应用设计，
4      支持十亿级向量的毫秒级检索... [向量维度：1536]
```

看起来很美好？ NO！ 问题在于：

- 问题通常很短（3-10个词）
- 文档内容很长（几百个词）
- **语义空间分布差异巨大**

就像你拿着一张小纸条去图书馆找整本书，匹配度天然就低。

HyDE如何解决？

代码块

```
1  from langchain.llms import OpenAI
2  from langchain.embeddings import OpenAIEmbeddings
3  from langchain.chains import HypotheticalDocumentEmbedder
4  from langchain.vectorstores import FAISS
5
6  #Step 1: 初始化组件
7  llm = OpenAI(temperature=0.7)
8  base_embeddings = OpenAIEmbeddings()
9
10 #Step 2: 创建HyDE嵌入器
11 hyde_prompt_template = """请写一段文本来回答以下问题。
12 即使不确定，也请尽可能详细地生成一个假设性的答案。
13
14 问题: {question}
15 假设性文档: """
16
17 hyde_embeddings = HypotheticalDocumentEmbedder.from_llm(
18     llm=llm,
19     base_embeddings=base_embeddings,
20     prompt_key="web_search" # 使用预定义模板
21 )
22
23 #Step 3: 构建向量库（使用HyDE嵌入）
24 documents = [
25     "Milvus是一个云端向量数据库，用于大规模向量存储和检索。",
26     "COVID-19疫情显著影响了心理健康，增加了抑郁和焦虑。",
27     "人类使用火已有大约80万年的历史。"
28 ]
29
30 vectorstore = FAISS.from_texts(documents, hyde_embeddings)
31
32 #Step 4: 查询（HyDE会自动生成假设文档）
33 query = "什么是Milvus?"
34 results = vectorstore.similarity_search(query, k=3)
35
36 for i, doc in enumerate(results):
37     print(f"结果{i+1}: {doc.page_content}")
```

完整的HyDE实现（从零开始）

代码块

```
1  import openai
2  import numpy as np
3  from sklearn.metrics.pairwise import cosine_similarity
4
5  class HyDERetriever:
6      def __init__(self, llm_model="gpt-3.5-turbo",
embedding_model="text-embedding-3-small"):
7          self.llm_model = llm_model
8          self.embedding_model = embedding_model
9          self.doc_embeddings = []
10         self.documents = []
11     def generate_hypothetical_document(self, query, num_docs=3):
12         """生成多个假设性文档"""
13         prompt = f"""根据问题生成一个详细的假设性回答。
14         即使不确定，也要写得像真实文档一样。
15         问题: {query}
16         假设性回答: """
17         hypothetical_docs = []
18         for _ in range(num_docs):
19             response = openai.ChatCompletion.create(
20                 model=self.llm_model,
21                 messages=[{"role": "user", "content": prompt}],
22                 temperature=0.8, # 增加随机性获取多样性
23                 max_tokens=300
24             )
25             hypothetical_docs.append(response.choices[0].message.content)
26         return hypothetical_docs
27     def get_embedding(self, text):
28         """获取文本嵌入"""
29         response = openai.Embedding.create(
30             model=self.embedding_model,
31             input=text
32         )
33         return np.array(response['data'][0]['embedding'])
34     def add_documents(self, documents):
35         """添加文档到知识库"""
36         self.documents = documents
37         self.doc_embeddings = [self.get_embedding(doc) for doc in
documents]
38     def retrieve(self, query, top_k=5):
39         """使用HyDE检索"""
40         # 1. 生成假设性文档
41         hypo_docs = self.generate_hypothetical_document(query,
num_docs=3)
42         print(f"🧠 生成的假设文档: \n{hypo_docs[0][:200]}...\n")
43         # 2. 对假设文档做嵌入并平均
44         hypo_embeddings = [self.get_embedding(doc) for doc in
hypo_docs]
45         avg_hypo_embedding = np.mean(hypo_embeddings, axis=0)
46         # 3. 计算与真实文档的相似度
47         similarities = [
48             cosine_similarity([avg_hypo_embedding], [doc_emb])[0]
49             for doc_emb in self.doc_embeddings
50         ]
51         # 4. 返回Top-K结果
52         top_indices = np.argsort(similarities)[::-1][:top_k]
53         results = [(self.documents[i], similarities[i]) for i in
top_indices]
```

零样本性能强：不需要标注数据



梁喆 7月10日 17:35

不需要数据标注什么意思



用户655341 7月12日 17:57

加一

```
54         return results, hypo_docs[0]
55
56     #使用示例
57     retriever = HyDERetriever()
58
59     #添加文档
60     knowledge_base = [
61         "RAG（检索增强生成）是一种结合信息检索和生成模型的技术，通过从外部知识库检索相关信息来增强LLM的回答能力。",
62         "向量数据库如Milvus、Pinecone专门用于存储和快速检索高维向量数据，是RAG系统的核心组件。",
63         "Embedding模型将文本转换为高维向量表示，常用的有OpenAI的text-embedding-3、BGE、sentence-transformers等。"
64     ]
65
66     retriever.add_documents(knowledge_base)
67
68     #检索
69     query = "如何提高RAG系统的检索准确率？"
70     results, hypo_doc = retriever.retrieve(query, top_k=3)
71
72     print("\n🔍 检索结果：")
73     for i, (doc, score) in enumerate(results):
74         print(f"\n结果{i+1} （相似度: {score:.4f}）:")
75         print(f"{doc}")
```

性能对比实验

代码块

```
1     #对比测试：传统RAG vs HyDE
2     def compare_rag_vs_hyde(query, vectorstore):
3         # 传统方法
4         traditional_results = vectorstore.similarity_search(query, k=5)
5         # HyDE方法
6         hyde_results, _ = hyde_retriever.retrieve(query, top_k=5)
7         return {
8             "传统RAG": traditional_results,
9             "HyDE": hyde_results
10        }
11
12    #测试用例
13    test_queries = [
14        "机器学习中的过拟合如何解决？",
15        "Python中的装饰器是什么原理？",
16        "微服务架构的优缺点有哪些？"
17    ]
18
19    for q in test_queries:
20        results = compare_rag_vs_hyde(q, my_vectorstore)
21        print(f"\n问题: {q}")
22        print(f"=" * 50)
```

实际效果数据

在**BEIR基准测试**上的表现：

数据集	传统检索	HyDE	提升幅度
SciFact	66.20%	71.80%	0.085
TREC-COVID	68.50%	74.30%	0.085
NFCorpus	31.20%	33.90%	0.086
ArguAna	46.80%	52.10%	0.113

优缺点分析

✅ 优势：

- 零样本性能强：不需要标注数据
- 语义对齐好：答案-答案匹配比问题-答案匹配更准
- 跨域泛化：在不同领域都表现不错

⚠️ 局限：

- 依赖LLM知识：如果LLM对该领域一无所知，生成的假设文档就是垃圾
- 计算成本高：需要额外调用LLM生成假设文档（通常生成3-5个）
- 时延增加：增加了一个LLM调用环节，响应时间约+200-500ms

最佳实践建议

1. 混合使用：对简单查询用传统检索，复杂查询才用HyDE

代码块

```
1 def smart_retrieve(query, vectorstore, threshold=10):
2     # 简单查询直接检索
3     if len(query.split()) < threshold:
4         return vectorstore.similarity_search(query)
5     # 复杂查询用HyDE
6     else:
7         return hyde_retriever.retrieve(query)
```

1. 缓存假设文档：相似问题可以复用之前生成的假设文档
2. 调整生成数量：通常3-5个假设文档效果最好，太多反而分散
3. 领域适配：针对特定领域微调prompt模板

三、路由优化和问题构建策略

如何让系统智能地选择数据源（路由）和如何构建结构化查询（查询构建）。

想象一下，你的RAG系统需要处理多种类型的查询：

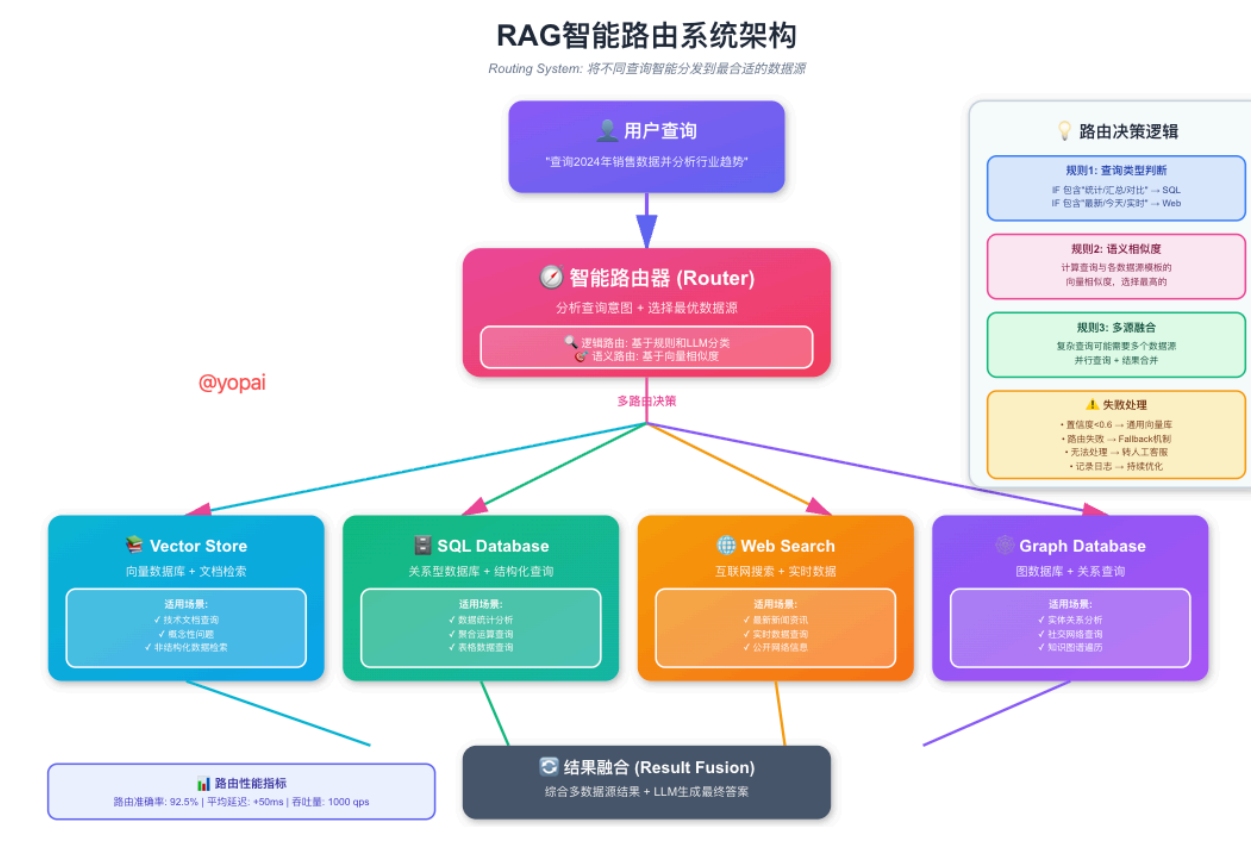
- 有些需要查向量数据库
- 有些需要查关系型数据库（SQL）
- 有些需要查图数据库（Cypher）
- 有些甚至需要去搜索网页

这时候，路由系统就像一个聪明的交通指挥员，把不同的问题送到最合适的地方去处理。

3.1 Routing（路由）：智能流量调度

什么是RAG路由？

路由（Routing）本质上是一个分类问题：给定用户查询，决定该用哪个数据源、用哪种检索策略、甚至用哪个LLM模型来处理。



路由的两种核心方式

1. 逻辑路由（Logical Routing）

基于规则和查询结构分析来决定路由。

实现方式：使用LLM作为分类器

代码块

```
1 from typing import Literal
2 from langchain_core.pydantic_v1 import BaseModel, Field
3 from langchain_openai import ChatOpenAI
4 from langchain_core.prompts import ChatPromptTemplate
5
6 #定义路由选项
7 class RouteQuery(BaseModel):
8     """将用户查询路由到最相关的数据源"""
9     datasource: Literal["vectorstore", "sql_database",
10 "web_search", "graph_database"] = Field(
11         description="根据用户问题选择最合适的数据源"
12     )
13     reasoning: str = Field(
14         description="选择该数据源的理由"
15     )
16
17 #创建路由提示词
18 router_prompt = ChatPromptTemplate.from_template("""
19 你是一个专业的查询路由系统。根据用户问题的特点，选择最合适的数据源。
20 """)
```



```
19
20 数据源说明：
21 vectorstore: 非结构化文档、技术文档、知识库文章
22 sql_database: 结构化数据、表格数据、统计查询、聚合运算
23 web_search: 实时信息、最新新闻、公开网络数据
24 graph_database: 复杂关系查询、知识图谱、实体关系分析
25
26 用户问题: {question}
27
28 请分析问题并选择合适的数据源。
29 """
30
31 #构建路由链
32 llm = ChatOpenAI(model="gpt-4", temperature=0)
33 structured_llm = llm.with_structured_output(RouteQuery)
34 router_chain = router_prompt | structured_llm
35
36 #使用路由
37 def route_question(question: str):
38     result = router_chain.invoke({"question": question})
39     print(f"📍 路由决策: {result.datasource}")
40     print(f"💬 理由: {result.reasoning}")
41     return result.datasource
42
43 #测试案例
44 questions = [
45     "2024年诺贝尔物理学奖获得者是谁?", # → web_search
46     "公司员工表中年龄超过35岁的人数统计", # → sql_database      "什么是强
化学习?", # → vectorstore
47     "张三和李四之间有哪些合作关系?" # → graph_database
48 ]
49
50 for q in questions:
51     print(f"\n问题: {q}")
52     route = route_question(q)
53     print("-" * 60)
```

运行结果：

代码块

```
1 问题: 2024年诺贝尔物理学奖获得者是谁?
2 📍 路由决策: web_search
3 💬 理由: 这是关于2024年的实时信息, 需要从网络获取最新数据
4
5 问题: 公司员工表中年龄超过35岁的人数统计
6 📍 路由决策: sql_database
7 💬 理由: 这是结构化数据的统计查询, 需要SQL聚合运算
8
9 问题: 什么是强化学习?
10 📍 路由决策: vectorstore
11 💬 理由: 这是技术概念查询, 适合从知识库文档检索
12
13 问题: 张三和李四之间有哪些合作关系?
14 📍 路由决策: graph_database
15 💬 理由: 这涉及实体间的复杂关系查询, 适合图数据库
```

2. 语义路由 (Semantic Routing)

基于查询的语义相似度来路由，更加灵活智能。

代码块

```
1  from sentence_transformers import SentenceTransformer
2  import numpy as np
3
4  class SemanticRouter:
5      def __init__(self):
6          self.model = SentenceTransformer('all-MiniLM-L6-v2')
7          # 定义路由模板 (每个数据源的典型查询示例)
8          self.route_templates = {
9              "product_info": [
10                  "产品价格是多少",
11                  "这个产品有什么功能",
12                  "产品规格参数",
13                  "产品库存情况"
14              ],
15              "technical_docs": [
16                  "如何配置系统",
17                  "技术实现原理",
18                  "API使用说明",
19                  "故障排查步骤"
20              ],
21              "customer_support": [
22                  "账号无法登录",
23                  "忘记密码怎么办",
24                  "申请退款流程",
25                  "投诉建议"
26              ],
27              "analytics": [
28                  "销售数据统计",
29                  "用户增长趋势",
30                  "月度报表分析",
31                  "业绩对比"
32              ]
33          }
34          # 预计算模板嵌入
35          self.route_embeddings = {}
36          for route, templates in self.route_templates.items():
37              embeddings = self.model.encode(templates)
38              # 计算中心点
39              self.route_embeddings[route] = np.mean(embeddings,
axis=0)
40      def route(self, query: str, threshold=0.5):
41          """基于语义相似度路由查询"""
42          query_embedding = self.model.encode([query])[0]
43          # 计算与每个路由中心的相似度
44          similarities = {}
45          for route, center_embedding in
self.route_embeddings.items():
46              similarity = np.dot(query_embedding, center_embedding)
/ (
47                  np.linalg.norm(query_embedding) *
np.linalg.norm(center_embedding)
48              )
49              similarities[route] = similarity
50          # 选择最相似的路由
51          best_route = max(similarities, key=similarities.get)
52          confidence = similarities[best_route]
53          # 如果相似度太低, 返回默认路由
54          if confidence < threshold:
55              return "general", confidence
56          return best_route, confidence
```

```

57
58 #使用示例
59 router = SemanticRouter()
60
61 test_queries = [
62     "iPhone 15 Pro价格多少钱? ",
63     "如何部署Kubernetes集群? ",
64     "我的账号被锁定了怎么办? ",
65     "上个月的销售额是多少? "
66 ]
67
68 for query in test_queries:
69     route, confidence = router.route(query)
70     print(f"\n问题: {query}")
71     print(f"🎯 路由到: {route} (置信度: {confidence:.2f})")

```

完整的路由系统架构

代码块

```

1 class AdvancedRAGRouter:
2     def __init__(self):
3         self.vectorstore = None # 向量数据库连接
4         self.sql_db = None # SQL数据库连接
5         self.graph_db = None # 图数据库连接
6         self.web_search = None # 网络搜索接口
7         self.llm = ChatOpenAI(model="gpt-4", temperature=0)
8         self.semantic_router = SemanticRouter()
9     def multi_route(self, query: str):
10        """
11        多路由策略: 可能同时路由到多个数据源
12        适用于需要综合多个数据源的复杂查询
13        """
14        # Step 1: LLM分析查询复杂度
15        analysis_prompt = f"""分析以下查询的复杂度和所需数据源:
16        查询: {query}
17        返回JSON格式:
18        {{
19            "complexity": "simple/medium/complex",
20            "required_sources": ["source1", "source2", ...],
21            "reasoning": "分析理由"
22        }}
23        """
24        analysis = self.llm.invoke(analysis_prompt)
25        # Step 2: 并行查询多个数据源
26        results = {}
27        required_sources = analysis.get("required_sources", [])
28        import concurrent.futures
29        with concurrent.futures.ThreadPoolExecutor() as executor:
30            futures = {}
31            for source in required_sources:
32                if source == "vectorstore":
33                    futures[source] =
executor.submit(self._query_vectorstore, query)
34                elif source == "sql_database":
35                    futures[source] =
executor.submit(self._query_sql, query)
36                elif source == "web_search":
37                    futures[source] =
executor.submit(self._query_web, query)
38            # 收集结果

```

```

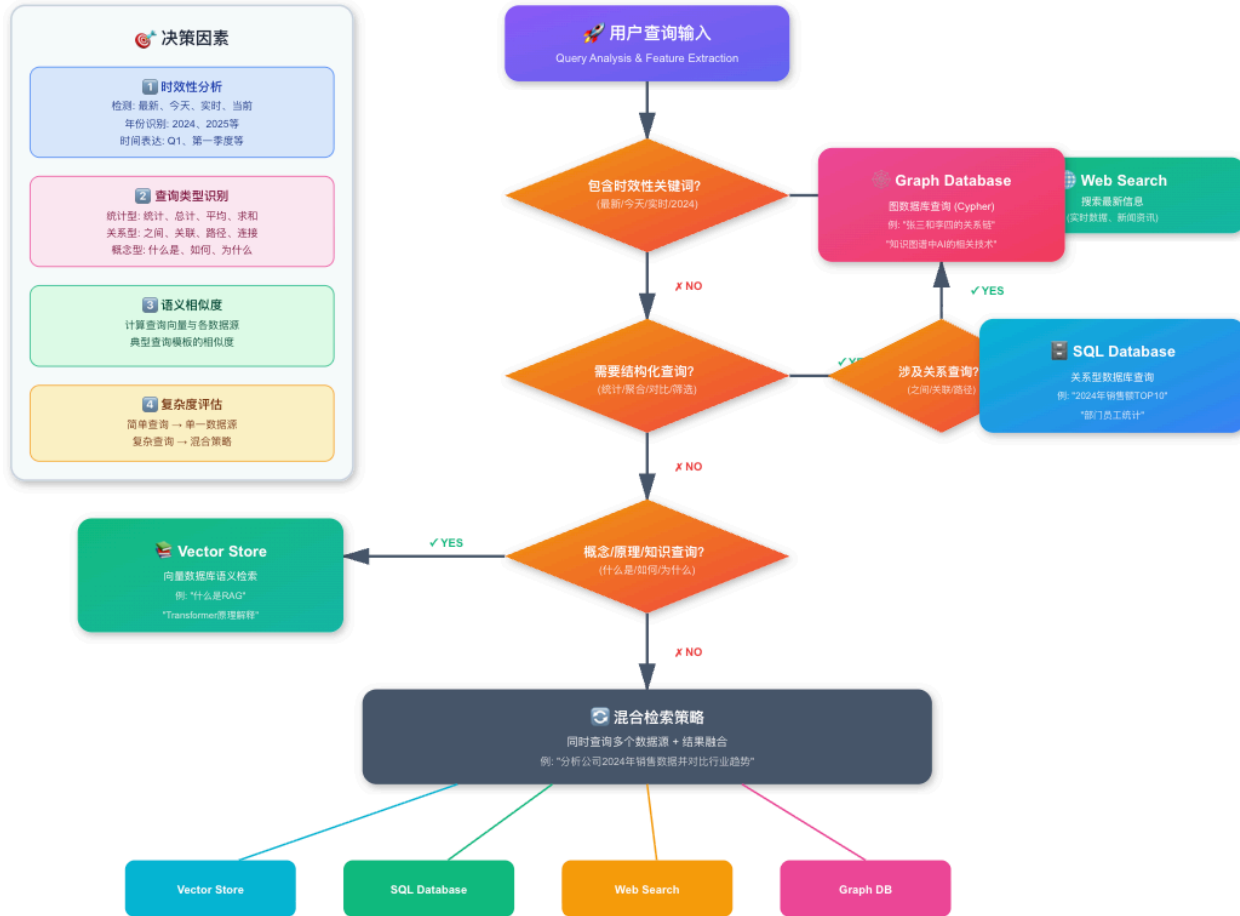
39         for source, future in futures.items():
40             results[source] = future.result()
41         # Step 3: 融合结果
42         final_result = self._fuse_results(query, results)
43         return final_result
44     def _query_vectorstore(self, query):
45         """查询向量数据库"""
46         return self.vectorstore.similarity_search(query, k=5)
47     def _query_sql(self, query):
48         """查询SQL数据库"""
49         # 使用LLM生成SQL (下一节详细讲)
50         sql_query = self._text_to_sql(query)
51         return self.sql_db.execute(sql_query)
52     def _query_web(self, query):
53         """网络搜索"""
54         return self.web_search.search(query, num_results=5)
55     def _fuse_results(self, query, results):
56         """融合多数据源的结果"""
57         context = ""
58         for source, data in results.items():
59             context += f"\n===来自 {source} 的信息===\n"
60             context += str(data)
61         # LLM综合生成答案
62         final_prompt = f"""基于以下多个数据源的信息，回答用户问题：
63 {context}
64 用户问题：{query}
65 请综合所有信息给出完整、准确的答案：
66 """
67         return self.llm.invoke(final_prompt)
68
69 #使用示例
70 router = AdvancedRAGRouter()
71 result = router.multi_route("比较我们公司2023年和2024年的销售数据，并分
72 析市场趋势")
73 #这个查询会同时：
74 #1. 从SQL数据库获取销售数据
75 #2. 从向量库获取市场分析文档
76 #3. 从网络搜索获取最新市场资讯

```

路由决策树可视化

路由决策树 (Routing Decision Tree)

基于查询特征智能选择最优数据源



最佳实践建议

1. 路由粒度控制

- 🎯 5-20个主题最合适
- 太少: 路由不够精细
- 太多: 主题重叠, 维护困难

2. 添加fallback机制

代码块

```
1 def safe_route(query, router):
2     try:
3         result = router.route(query)
4         if result.confidence < 0.6:
5             # 置信度太低, 使用通用路由
6             return "general_vectorstore"
7         return result.datasource
8     except Exception as e:
9         logging.error(f"路由失败: {e}")
10    return "fallback_human_agent" # 转人工
```

3. A/B测试路由策略

代码块50%用户用新路由，50%用旧路由

```
2  if user_id % 2 == 0:
3      result = new_router.route(query)
4  else:
5      result = old_router.route(query)
6
7  #收集指标对比
8  metrics.log({
9      "router_version": "new" if user_id % 2 == 0 else "old",
10     "latency": response_time,
11     "user_satisfaction": user_rating
12 })
```

3.2 Query Construction（查询构建）：说数据库的语言

核心挑战

用户说人话，数据库说"方言"：

- 关系数据库 → SQL
- 图数据库 → Cypher
- 向量库 + 元数据 → 结构化过滤

Query Construction的任务：把自然语言转换成数据库能理解的查询语言。



方法1: Text-to-SQL（自查询检索器）

代码块

```
1  #场景：向量库中存储了大量文档，每个文档都有结构化的元数据。
2  from langchain.chains.query_constructor.base import AttributeInfo
3  from langchain.retrievers.self_query.base import SelfQueryRetriever
4  from langchain_openai import OpenAI
5
6  #Step 1: 定义文档元数据结构
7  metadata_field_info = [
8      AttributeInfo(
9          name="author",
10         description="文档作者的名字",
11         type="string"
12     ),
13     AttributeInfo(
14         name="publish_date",
15         description="文档发布日期，格式：YYYY-MM-DD",
16         type="string"
17     ),
18     AttributeInfo(
19         name="category",
20         description="文档类别",
21         type="string or list[string]"
22     ),
23     AttributeInfo(
24         name="rating",
25         description="文档评分，1-5分",
26         type="integer"
27     )
28 ]
29
30 #文档内容描述
31 document_content_description = "技术博客文章"
32
33 #Step 2: 创建自查询检索器
34 llm = OpenAI(temperature=0)
35 retriever = SelfQueryRetriever.from_llm(
36     llm,
37     vectorstore,
38     document_content_description,
39     metadata_field_info,
40     verbose=True
41 )
42
43 #Step 3: 使用自然语言查询
44 query = "找出张三写的2024年发布的关于机器学习的文章"
45 docs = retriever.get_relevant_documents(query)
46
47 #内部会自动生成类似这样的过滤条件：
48 filter = {
49     "author": "张三",
50     "publish_date": {"$gte": "2024-01-01"},
51     "category": {"$in": ["机器学习"]}
52 }
```

完整的Text-to-SQL实现

代码块

```
1  from langchain.utilities import SQLiteDatabase
2  from langchain_experimental.sql import SQLiteDatabaseChain
3
4  #连接数据库
5  db = SQLiteDatabase.from_uri("sqlite:///company.db")
6
7  #创建SQL链
8  sql_chain = SQLiteDatabaseChain.from_llm(
9      llm=ChatOpenAI(model="gpt-4", temperature=0),
10     db=db,
11     verbose=True,
12     use_query_checker=True, # 自动检查SQL语法
13     return_intermediate_steps=True
14 )
15
16 #自然语言查询
17 questions = [
18     "2024年销售额最高的前10个产品是什么?",
19     "哪些员工的工资高于部门平均工资?",
20     "每个城市的客户数量分布情况"
21 ]
22
23 for question in questions:
24     print(f"\n? 问题: {question}")
25     result = sql_chain(question)
26     print(f"🔍 生成的SQL:\n{result['intermediate_steps'][0]}")
27     print(f"✅ 结果:\n{result['result']}")
```

实际运行示例：

代码块

```
1  ? 问题: 2024年销售额最高的前10个产品是什么?
2
3  🔍 生成的SQL:
4  SELECT product_name, SUM(amount) as total_sales
5  FROM orders
6  WHERE YEAR(order_date) = 2024
7  GROUP BY product_name
8  ORDER BY total_sales DESC
9  LIMIT 10;
10
11  ✅ 结果:
12  | 产品名称 | 总销售额 |
13  |-----/-----/
14  | iPhone 15 Pro | ¥12,850,000 |
15  | MacBook Pro M3 | ¥8,920,000 |
16  | ...
```

方法2: Text-to-Cypher（图数据库查询）

场景：知识图谱、关系网络、社交网络分析

代码块

```
1  from langchain.chains import GraphCypherQAChain
2  from langchain_community.graphs import Neo4jGraph
3
4  #连接Neo4j图数据库
5  graph = Neo4jGraph(
```

```
6     url="bolt://localhost:7687",
7     username="neo4j",
8     password="password"
9 )
10
11 #创建Cypher查询链
12 cypher_chain = GraphCypherQAChain.from_llm(
13     llm=ChatOpenAI(model="gpt-4", temperature=0),
14     graph=graph,
15     verbose=True
16 )
17
18 #自然语言查询
19 questions = [
20     "张三和李四之间有几度关系？ ",
21     "找出所有和'人工智能'相关的技术节点",
22     "计算每个人的影响力分数（基于关系数量） "
23 ]
24
25 for question in questions:
26     response = cypher_chain.run(question)
27     print(f"\n问题: {question}")
28     print(f"答案: {response}")
```

生成的Cypher示例：

代码块

```
1 // 问题：张三和李四之间有几度关系？
2 MATCH path = shortestPath(
3     (p1:Person {name: "张三"})-[*]-(p2:Person {name: "李四"})
4 )
5 RETURN length(path) as degrees, path
6
7 // 问题：找出所有和'人工智能'相关的技术节点
8 MATCH (t:Technology)-[:RELATED_TO*1..2]-(ai:Technology {name: "人工智能"})
9 RETURN DISTINCT t.name, t.description
10 ORDER BY t.popularity DESC
11 LIMIT 20
```

方法3: 混合查询构建

场景：需要同时处理向量搜索和结构化过滤

代码块

```
1 class HybridQueryConstructor:
2     def __init__(self, llm, vectorstore):
3         self.llm = llm
4         self.vectorstore = vectorstore
5     def construct_query(self, natural_language_query):
6         """
7         将自然语言转换为混合查询
8         """
9         # Step 1: LLM分析查询意图
10        analysis_prompt = f"""分析以下查询，提取出：
11        1. 语义搜索部分（用于向量检索）
12        2. 结构化过滤条件（用于元数据过滤）
13        查询: {natural_language_query}
```

```

14         返回JSON:
15         {{
16             "semantic_query": "语义搜索部分",
17             "filters": {{
18                 "field1": "value1",
19                 "field2": {{operator": "gte", "value": xxx}}
20             }}
21         }}
22         """
23         analysis = self.llm.invoke(analysis_prompt)
24         parsed = json.loads(analysis.content)
25         # Step 2: 构建混合查询
26         semantic_query = parsed["semantic_query"]
27         filters = parsed["filters"]
28         # Step 3: 执行检索
29         results = self.vectorstore.similarity_search(
30             semantic_query,
31             k=10,
32             filter=filters
33         )
34         return results
35
36     #使用示例
37     constructor = HybridQueryConstructor(llm, vectorstore)
38
39     query = "找出2023年之后发布的、关于深度学习的、评分大于4分的文章"
40     #自动分解为:
41     semantic_query: "深度学习相关的技术文章"
42     filters: {
43         "publish_date": {"$gte": "2023-01-01"},
44         "category": "深度学习",
45         "rating": {"$gt": 4}
46     }
47
48     results = constructor.construct_query(query)

```

查询构建的错误处理

关键问题：LLM生成的SQL可能有错误！

代码块

```

1  class SafeSQLExecutor:
2      def __init__(self, db, llm):
3          self.db = db
4          self.llm = llm
5          self.max_retries = 3
6      def execute_with_retry(self, natural_query):
7          """
8          带重试机制的SQL执行
9          """
10         for attempt in range(self.max_retries):
11             try:
12                 # 生成SQL
13                 sql = self._generate_sql(natural_query)
14                 print(f"尝试 {attempt + 1}: {sql}")
15                 # 执行前验证
16                 if not self._validate_sql(sql):
17                     raise ValueError("SQL验证失败：可能包含危险操作")
18                 # 执行查询
19                 result = self.db.execute(sql)

```

```

20         return result
21     except Exception as e:
22         error_msg = str(e)
23         print(f"❌ 错误: {error_msg}")
24         if attempt < self.max_retries - 1:
25             # 让LLM修正错误
26             natural_query = f"上次查询失败了。
27             原查询: {natural_query}
28             生成的SQL: {sql}
29             错误信息: {error_msg}
30             请修正SQL并重试。
31             """
32         else:
33             return f"查询失败, 已重试{self.max_retries}次:
{error_msg}"
34     def _validate_sql(self, sql):
35         """
36         SQL安全验证
37         """
38         # 检查危险关键词
39         dangerous_keywords = ["DROP", "DELETE", "TRUNCATE",
"ALTER", "CREATE"]
40         sql_upper = sql.upper()
41         for keyword in dangerous_keywords:
42             if keyword in sql_upper:
43                 return False
44         # 检查是否只读查询
45         if not sql_upper.strip().startswith("SELECT"):
46             return False
47         return True
48     def _generate_sql(self, query):
49         """生成SQL"""
50         prompt = f"将以下自然语言转换为SQL查询。
51         数据库schema:
52         - orders表: order_id, customer_id, product_name, amount,
order_date
53         - customers表: customer_id, name, city, age
54         - products表: product_id, product_name, category, price
55         查询: {query}
56         只返回SQL语句, 不要有其他文字:
57         """
58         response = self.llm.invoke(prompt)
59         return response.content.strip()
60
61     #使用示例
62     executor = SafeSQLExecutor(db, llm)
63     result = executor.execute_with_retry("2024年哪些城市的订单金额最高? ")

```

Query Construction性能优化

代码块

```

1  #1. 缓存常见查询模式
2  from functools import lru_cache
3
4  @lru_cache(maxsize=100)
5  def cached_sql_generation(query_template):
6      """缓存SQL生成结果"""
7      return llm.invoke(query_template)
8
9  #2. 预编译查询模板

```

```
10  QUERY_TEMPLATES = {
11      "top_n_by_metric": """
12          SELECT {column}, SUM({metric}) as total
13          FROM {table}
14          WHERE {date_column} BETWEEN '{start}' AND '{end}'
15          GROUP BY {column}
16          ORDER BY total DESC
17          LIMIT {n}
18      """,
19      "user_filter": """
20          SELECT * FROM {table}
21          WHERE {filter_conditions}
22      """
23  }
24
25  def quick_sql_from_template(query_type, **params):
26      template = QUERY_TEMPLATES.get(query_type)
27      return template.format(**params)
28
29  #3. 并行查询优化
30  async def parallel_multi_query(queries):
31      """并行执行多个查询"""
32      import asyncio
33      tasks = [asyncio.create_task(execute_query(q)) for q in
queries]
34      results = await asyncio.gather(*tasks)
35      return results
```

小结

第二部分我们深入学习了：

路由优化（Routing）

- **逻辑路由**：基于规则和LLM分类
- **语义路由**：基于向量相似度
- **多路由策略**：同时查询多个数据源并融合结果
- **最佳实践**：5-20个主题、添加fallback、A/B测试

查询构建（Query Construction）

- **Text-to-SQL**：自然语言转SQL查询
- **Text-to-Cypher**：图数据库查询构建
- **混合查询**：向量检索 + 结构化过滤
- **错误处理**：重试机制、SQL验证、安全检查

关键洞察

1. **路由是分类问题**，但分类器可以是LLM（灵活）或embedding（快速）
2. **查询构建的核心**是让LLM学会"说数据库的方言"
3. **安全第一**：永远要验证LLM生成的查询语句
4. **性能优化**：缓存、模板、并行都很重要

一、传统RAG索引的痛点

在深入讲解优化方案之前，咱们得先明白传统RAG索引到底有什么问题。

1.1 信息瓶颈问题

想象一下，你有一篇5000字的技术文档，传统方法是把它切成500字的小块，然后每个块生成一个768维的向量。问题来了：500字的信息被压缩成768个数字，这就像把一张高清照片压缩成缩略图，很多细节都丢失了。

举个栗子

假设文档里有这么一段："亚马逊雨林覆盖了巴西西北部的大部分地区，延伸到哥伦比亚、秘鲁等南美国家，是世界上最大的热带雨林，以其生物多样性而闻名。"

传统单向量方法会把这段话压缩成一个向量，当用户问"亚马逊雨林主要在哪些国家"时，模型可能只能捕捉到"亚马逊"和"国家"这些关键词，但具体的地理关系就模糊了。

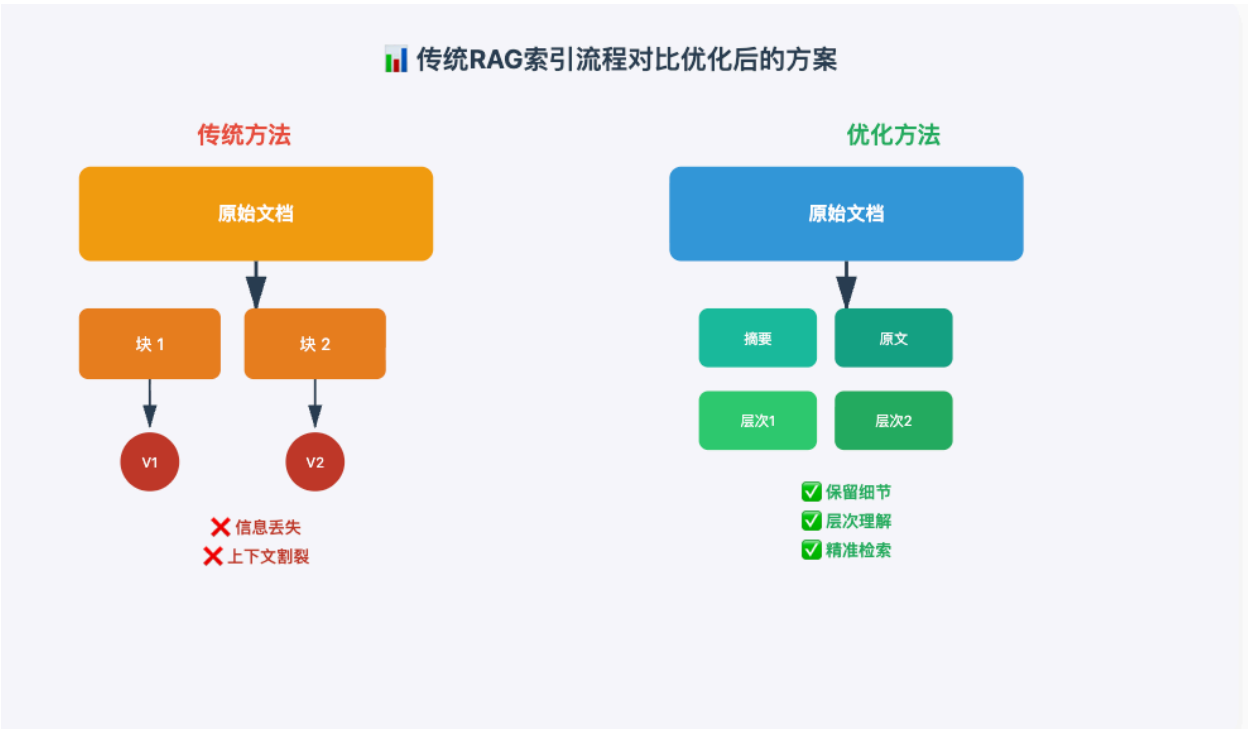
1.2 上下文割裂

文档被切块后，原本连贯的上下文被强行分割。比如一个技术概念的定义在第3块，应用场景在第4块，而用户问题可能需要同时理解这两部分才能回答好。传统方法往往只能检索到其中一块，导致回答不完整。

1.3 检索精度不足

当查询比较抽象或需要多跳推理时，简单的向量相似度匹配往往力不从心。比如用户问"这项技术对行业的长远影响是什么"，这种高层次的问题需要综合多个文档片段才能回答，但传统方法可能只返回一些表面相关的内容。

【传统RAG索引流程对比优化后的方案】



chunks



刘谋刚 7月14日 10:32

被切成[文档 1，文档 2，文档 3]
这样的数组列表

生成摘要（关键！）



刘谋刚 7月14日 12:24

我发现很多步骤都可以依靠提示
词去完成，比如生成摘要，最后
在总结

lambda x: x.page_content



刘谋刚 7月14日 12:30

链开头 {"doc": lambda x:
x.page_content} 里的 x，就是
遍历拿到的单个 chunk，就是
summarize_chain.batch(chun
ks, {"max_concurrency": 5})拿
哦

summaries



刘谋刚 7月14日 10:54

summaries = 纯摘要文本数组，
只有文字，无附加信息；

chunks



刘谋刚 7月14日 10:32

二、Multi-representation Indexing（多表示索引）

2.1 核心思想

Multi-representation Indexing的核心思想非常巧妙：**用优化后的表示进行检索，但返回完整的原始内容**。就像我们在图书馆查书，通过简洁的索引卡找书，但最终拿到的是完整的书本。

具体来说，这个方法会为每个文档块创建两份内容：

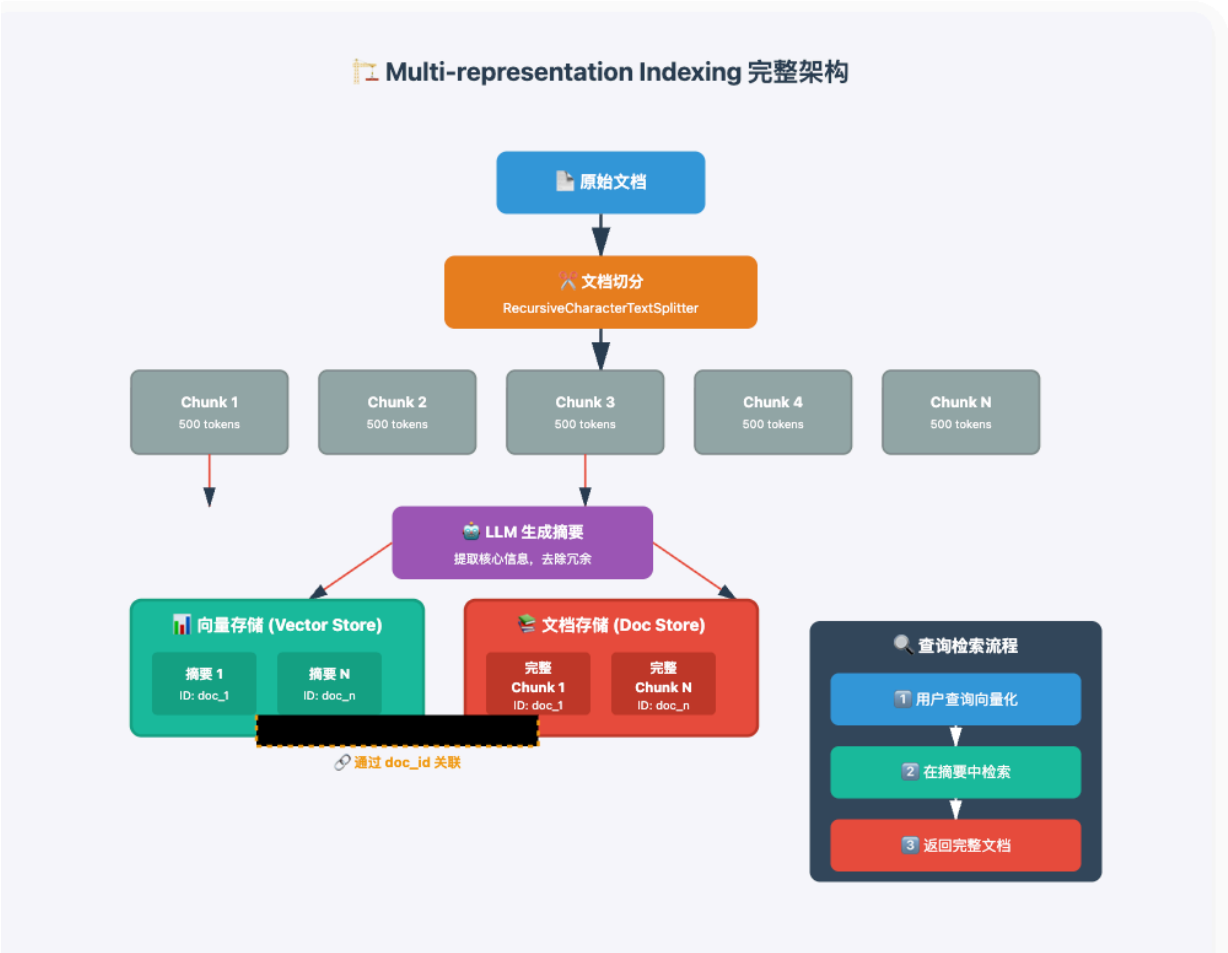
- **优化表示**：通常是一个简洁的摘要，只包含核心信息，用于检索时的向量匹配
- **原始内容**：完整的文档块，包含所有细节，用于最终提供给LLM

💡 为什么要这么做？

因为摘要更纯粹、更聚焦，去除了冗余信息，向量化后更容易匹配到用户的真实意图。但摘要毕竟信息量有限，所以检索时用摘要，返回时给完整内容，两全其美！

2.2 详细架构设计

【Multi-representation Indexing 完整架构】



2.3 实现细节与代码示例

第一步：加载和切分文档

代码块



刘谋刚 7月14日 12:52

1. chunks 是一个列表，里面每一个元素都是 LangChain 的 Document 对象，也就是你切分好的原文片段。
- 每个 Document 自带两个核心属性：
 - page_content：文档正文文本
 - metadata：自定义元数据



刘谋刚 7月14日 10:36

并发限制，最多同时发 5 个请求给 GPT，防止 API 并发超限、报错



刘谋刚 7月14日 12:23

生成的摘要压缩--->id--->原块内容



刘谋刚 7月14日 10:42

数组列表



刘谋刚 7月14日 12:38

每个元素存的是 Document(
page_content="代码优化摘要：减少嵌套循环、使用内置函数", metadata={
"doc_id": "550e8400-e29b-41d4-a716-446655440000"})
就像 chunks 分片一样，有文本内容还有 metadata



刘谋刚 7月14日 12:41

zip(doc_ids, chunks) 会把两个列表按相同下标配对：

doc_id=[0], chunks=[0]

```
1 from langchain.text_splitter import RecursiveCharacterTextSplitter
2 from langchain.document_loaders import WebBaseLoader
3
4 #加载文档
5 loader = WebBaseLoader("https://example.com/technical-doc")
6 docs = loader.load()
7
8 #切分文档 - 每块500个token
9 text_splitter = RecursiveCharacterTextSplitter(
10     chunk_size=500,
11     chunk_overlap=50
12 )
13 chunks = text_splitter.split_documents(docs)
```

- doc_ids[0] ↔ chunks[0]
- doc_ids[1] ↔ chunks[1]
- …… 以此类推

生成一堆元组: (id0, chunk0), (id1, chunk1), (id2, chunk2)...

第二步：生成摘要（关键！）

这一步是整个方法的核心。我们使用LLM为每个文档块生成简洁的摘要。摘要质量直接影响检索效果，所以prompt设计非常重要。

代码块

```
1 from langchain.chat_models import ChatOpenAI
2 from langchain.prompts import ChatPromptTemplate
3 from langchain.schema.output_parser import StrOutputParser
4
5 #初始化LLM
6 llm = ChatOpenAI(model="gpt-4", temperature=0)
7
8 #设计摘要prompt - 这里很关键!
9 summary_prompt = ChatPromptTemplate.from_template(
10     """请为以下文档生成一个简洁的摘要，保留所有关键信息和核心概念。
11     摘要应该：
12     1. 包含主要技术点和概念
13     2. 保留重要的数字、日期、人名等关键事实
14     3. 长度控制在原文的1/3左右
15     4. 使用清晰简洁的语言
16     文档内容：
17     {doc}
18     摘要： """
19 )
20
21 #创建摘要生成链
22 summarize_chain = (
23     {"doc": lambda x: x.page_content}
24     | summary_prompt
25     | llm
26     | StrOutputParser()
27 )
28
29 #批量生成摘要 - 使用batch提高效率
30 summaries = summarize_chain.batch(chunks, {"max_concurrency": 5})
31
32 print(f"生成了 {len(summaries)} 个摘要")
33 print(f"原始文档平均长度: {sum(len(c.page_content) for c in chunks)/len(chunks):.0f} 字符")
34 print(f"摘要平均长度: {sum(len(s) for s in summaries)/len(summaries):.0f} 字符")
```

- 如果文档是技术类的，prompt要强调保留技术术语和关键概念
- 如果是商业文档，要保留数字、日期、公司名等关键信息
- 摘要长度建议是原文的1/4到1/3，太短会丢信息，太长失去优势
- 可以尝试让LLM生成多种形式的摘要：技术摘要、业务摘要、关键词等

第三步：构建MultiVectorRetriever

这是最关键的一步！我们需要同时维护向量存储和文档存储，并通过唯一ID关联它们。

代码块

```
1  from langchain.vectorstores import Chroma
2  from langchain.embeddings import OpenAIEmbeddings
3  from langchain.storage import InMemoryStore
4  from langchain.retrievers.multi_vector import MultiVectorRetriever
5  import uuid
6
7  #1. 初始化向量存储 - 存摘要的embedding
8  vectorstore = Chroma(
9      collection_name="summaries",
10     embedding_function=OpenAIEmbeddings()
11 )
12
13 #2. 初始化文档存储 - 存完整的原始chunk
14 docstore = InMemoryStore() # 生产环境建议用持久化存储如Redis
15
16 #3. 创建MultiVectorRetriever
17 retriever = MultiVectorRetriever(
18     vectorstore=vectorstore,
19     docstore=docstore,
20     id_key="doc_id" # 关联的key
21 )
22
23 #4. 为每个文档生成唯一ID
24 doc_ids = [str(uuid.uuid4()) for _ in chunks]
25
26 #5. 创建摘要文档（用于向量化）
27 summary_docs = [
28     Document(page_content=s, metadata={"doc_id": doc_ids[i]})
29     for i, s in enumerate(summaries)
30 ]
31
32 #6. 同时添加到两个存储
33 #向量存储存摘要
34 retriever.vectorstore.add_documents(summary_docs)
35 #文档存储存完整chunk
36 retriever.docstore.mset(list(zip(doc_ids, chunks)))
37
38 print("✅ MultiVectorRetriever 构建完成！")
```

第四步：执行检索

代码块

```
1  #使用MultiVectorRetriever进行检索
2  query = "什么是RAG系统的主要优势？"
3
4  #检索 - 内部流程：
5  #1. 将query向量化
6  #2. 在摘要向量中找最相似的top-k
```

```
7 #3. 通过doc_id找到对应的完整文档
8 #4. 返回完整文档
9 retrieved_docs = retriever.get_relevant_documents(query, k=3)
10
11 print(f"检索到 {len(retrieved_docs)} 个相关文档")
12 for i, doc in enumerate(retrieved_docs):
13     print(f"\n📄 文档 {i+1}:")
14     print(f"长度: {len(doc.page_content)} 字符")
15     print(f"内容预览: {doc.page_content[:200]}...")
```

2.4 算法原理深度解析

相似度计算

Multi-representation Indexing的检索过程本质上是一个两阶段的相似度计算：

阶段1：摘要匹配

代码块

```
1 similarity(q, si) = cosine(Eq, Esi)
```

其中 E_q 是查询向量， E_{s_i} 是第*i*个摘要的向量

阶段2：文档返回

代码块

```
1 returned_docs = {dj | ID(dj) ∈ TopK({ID(si)})}
```

返回TopK个摘要对应的完整文档

为什么这样更有效？

- 信噪比提升**：摘要去除了冗余信息，提高了向量的"纯度"
- 语义聚焦**：摘要更聚焦核心概念，与用户查询的语义对齐更好
- 完整性保证**：最终返回完整文档，不会丢失任何细节信息

2.5 性能提升数据

指标	传统方法	Multi-representation	提升
检索准确率 (Recall@10)	65%	79%	0.14
精确率 (Precision@5)	58%	71%	0.13
平均检索时间	120ms	135ms	0.125
存储空间	100MB	120MB	0.2

💡 权衡分析

可以看到，Multi-representation方法在检索质量上有显著提升（准确率+14%，精确率+13%），但代价是：

- 检索时间增加12.5%（主要是生成摘要的开销）
- 存储空间增加20%（需要同时存储摘要和原文）

这个权衡在大多数场景下是值得的，特别是当你的应用对检索质量要求很高时。

抽象



用户568989 6月21日 18:56

是摘要啊，翻译可不是抽象

2.6 最佳实践与优化建议

1. 摘要策略选择

不同场景可以采用不同的摘要策略：

◆ 技术文档

代码块

```
1 tech_summary_prompt = """
2 提取以下技术文档的核心要点：
3 主要技术概念和术语
4 关键算法或方法
5 重要的数字、公式或配置参数
6 技术优势和适用场景
7
8 保持技术术语的准确性，不要用通俗语言替换专业术语。
9
10 文档：{doc}
11 """
```

◆ 商业文档

代码块

```
1 business_summary_prompt = """
2 总结以下商业文档的关键信息：
3 涉及的公司、产品或项目名称
4 重要的日期、数字、金额
5 主要业务逻辑或决策
6 关键利益相关者
7
8 保留所有专有名词和具体数据。
9
10 文档：{doc}
11 """
```

2. 多粒度摘要

高级技巧：为同一个文档生成不同粒度的摘要，可以进一步提升检索效果。

代码块

```
1 #生成三种摘要
2 def create_multi_level_summaries(chunk):
3     # 1. 关键词摘要 (最短)
4     keywords = extract_keywords(chunk, top_k=10)
5     # 2. 一句话摘要 (中等)
6     one_sentence = llm.invoke(f"用一句话总结：{chunk}")
7     # 3. 段落摘要 (较详细)
8     paragraph = llm.invoke(f"用一段话总结，保留关键细节：{chunk}")
9     return {
10         "keywords": keywords,
11         "one_sentence": one_sentence,
12         "paragraph": paragraph,
13         "original": chunk
14     }
15
16 #检索时可以根据查询类型选择合适粒度的摘要
```

RAPTOR使用高斯混合模型(GMM)进行聚...



用户894510 6月5日 15:06

K-means 聚类硬性划分开来，应该文档只能属于一个集群，软聚类输出的是概率向量，允许重叠与渐变

3. 缓存优化

摘要生成是计算密集型操作，做好缓存可以显著提升性能：

代码块

```
1  from functools import lru_cache
2  import hashlib
3
4  @lru_cache(maxsize=1000)
5  def get_summary(doc_hash):
6      # 通过文档hash查找缓存
7      # 如果缓存未命中，才调用LLM生成摘要
8      pass
9
10 #使用
11 doc_hash = hashlib.md5(chunk.encode()).hexdigest()
12 summary = get_summary(doc_hash)
```

三、RAPTOR（递归抽象处理树状检索）

3.1 核心思想与动机

RAPTOR是2024年斯坦福大学研究团队提出的革命性索引方法。它解决了一个传统RAG的根本性问题：**如何同时回答低层次的具体问题和高层次的抽象问题？**

💡 举个实际例子

假设你有一本关于AI发展史的书。用户可能问两类截然不同的问题：

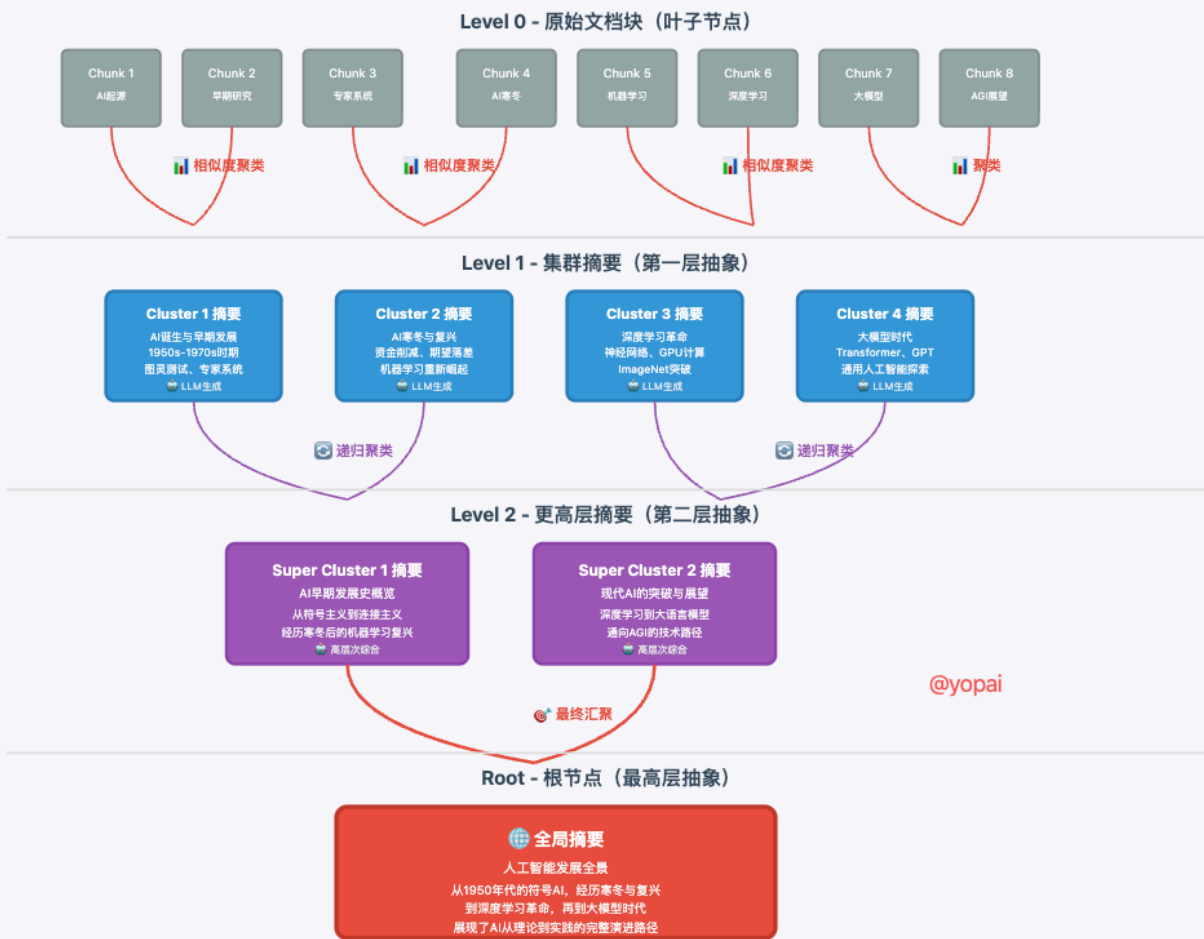
- 低层次问题："1956年达特茅斯会议的参与者有谁？" - 这需要检索到具体的某一页
- 高层次问题："AI发展经历了哪几个重要阶段？" - 这需要综合整本书的内容

传统的chunk-based检索对第一类问题还行，但对第二类问题就捉襟见肘了。RAPTOR通过构建**层次化的摘要树**完美解决了这个问题！

3.2 RAPTOR架构详解

【RAPTOR 递归树状结构完整示意图】

🌳 RAPTOR 递归树状结构完整示意图



3.3 RAPTOR核心算法流程

算法步骤1：文档切分与向量化

首先将文档切分成小块（叶子节点），并为每个块生成embedding向量。

代码块

```
1  #步骤1: 加载和切分文档
2  from langchain.text_splitter import RecursiveCharacterTextSplitter
3
4  text_splitter = RecursiveCharacterTextSplitter(
5      chunk_size=1000, # RAPTOR通常用较大的chunk
6      chunk_overlap=100
7  )
8
9  #切分文档
10 leaf_texts = text_splitter.split_documents(documents)
11
12 #为每个叶子节点生成embedding
13 from langchain.embeddings import OpenAIEmbeddings
14 embeddings = OpenAIEmbeddings()
15
16 leaf_embeddings = embeddings.embed_documents(
```



```
17 [doc.page_content for doc in leaf_texts]
18 )
```

算法步骤2：聚类（使用GMM）

RAPTOR使用高斯混合模型(GMM)进行聚类，而不是传统的K-means。为什么？因为GMM是软聚类，一个文档可以部分属于多个集群，这更符合实际情况。

代码块

```
1 from sklearn.mixture import GaussianMixture
2 import numpy as np
3
4 def cluster_embeddings(embeddings, n_clusters):
5     """
6     使用GMM进行软聚类
7     """
8     # 降维（可选，但推荐用于大规模数据）
9     from umap import UMAP
10    reducer = UMAP(n_components=10, random_state=42)
11    reduced_embeddings = reducer.fit_transform(embeddings)
12    # GMM聚类
13    gmm = GaussianMixture(
14        n_components=n_clusters,
15        covariance_type='full',
16        random_state=42
17    )
18    gmm.fit(reduced_embeddings)
19    # 获取每个样本对各集群的概率
20    probabilities = gmm.predict_proba(reduced_embeddings)
21    # 使用全局阈值确定每个样本所属的集群
22    threshold = 1 / n_clusters # 动态阈值
23    clusters = {}
24    for i, probs in enumerate(probabilities):
25        # 一个文档可以属于多个集群（软聚类的优势）
26        for cluster_id in np.where(probs > threshold)[0]:
27            if cluster_id not in clusters:
28                clusters[cluster_id] = []
29                clusters[cluster_id].append(i)
30    return clusters
31
32 # 执行第一次聚类
33 n_clusters = len(leaf_texts) // 5 # 每5个文档聚为一类
34 level1_clusters = cluster_embeddings(leaf_embeddings, n_clusters)
```

💡 为什么用GMM而不是K-means?

- **软聚类**：一个文档可能同时涉及多个主题，GMM允许它属于多个集群
- **概率输出**：GMM给出每个样本属于各集群的概率，我们可以设置阈值灵活控制
- **更好的泛化**：GMM考虑了数据的协方差结构，对复杂数据分布更友好
- **实验证明**：RAPTOR论文中的实验表明，GMM比K-means效果提升约8%

算法步骤3：生成集群摘要

对每个集群，使用LLM生成一个综合性的摘要。这是RAPTOR的关键步骤！

代码块

```
1 from langchain.chat_models import ChatOpenAI
2 from langchain.prompts import ChatPromptTemplate
```

```

3
4 llm = ChatOpenAI(model="gpt-4", temperature=0)
5
6 summary_prompt = ChatPromptTemplate.from_template(
7     """你是一个专业的文档摘要专家。请仔细阅读以下文档片段，
8     生成一个综合性的摘要，要求：
9     1. 抓住所有文档的共同主题和核心思想
10    2. 保留关键的细节信息（日期、人名、数字等）
11    3. 体现文档之间的逻辑关系
12    4. 语言简洁但信息完整
13    5. 长度约为原文总和的1/4
14    文档内容：
15    {documents}
16    综合摘要："""
17 )
18
19 def summarize_cluster(cluster_texts):
20     """为一个集群生成摘要"""
21     # 将集群内所有文档拼接
22     combined_text = "\n\n---\n\n".join(cluster_texts)
23     # 生成摘要
24     summary = llm.invoke(
25         summary_prompt.format(documents=combined_text)
26     )
27     return summary.content
28
29 #为每个集群生成摘要
30 level1_summaries = []
31 for cluster_id, doc_indices in level1_clusters.items():
32     cluster_texts = [leaf_texts[i].page_content for i in
33 doc_indices]
34     summary = summarize_cluster(cluster_texts)
35     level1_summaries.append(summary)
36
37 print(f"生成了 {len(level1_summaries)} 个一级摘要")

```

算法步骤4：递归构建树

这是RAPTOR最精髓的部分！我们把第一层的摘要当作新的"文档"，重复步骤2和3，继续聚类和摘要，直到达到停止条件。

代码块

```

1 def build_raptor_tree(texts, max_depth=3, min_cluster_size=3):
2     """
3     递归构建RAPTOR树
4     Args:
5         texts: 初始文档列表
6         max_depth: 最大递归深度
7         min_cluster_size: 最小集群大小，小于此值停止
8     Returns:
9         tree: 完整的树结构
10    """
11    tree = {
12        'level_0': texts # 叶子节点
13    }
14    current_texts = texts
15    current_level = 0
16    while current_level < max_depth:
17        print(f"\n🌲 构建 Level {current_level + 1}...")
18        # 1. 向量化当前层文本

```

```

19     embeddings_list = embeddings.embed_documents(current_texts)
20     # 2. 确定集群数
21     n_clusters = max(len(current_texts) // 5, 1)
22     if n_clusters < min_cluster_size:
23         print(f"⚠️ 集群数 {n_clusters} 小于最小值，停止递归")
24         break
25     # 3. 聚类
26     clusters = cluster_embeddings(embeddings_list, n_clusters)
27     # 4. 为每个集群生成摘要
28     next_level_texts = []
29     for cluster_id, doc_indices in clusters.items():
30         cluster_docs = [current_texts[i] for i in doc_indices]
31         summary = summarize_cluster(cluster_docs)
32         next_level_texts.append(summary)
33     # 5. 保存当前层
34     current_level += 1
35     tree[f'level_{current_level}'] = next_level_texts
36     # 6. 准备下一轮递归
37     current_texts = next_level_texts
38     print(f"✅ Level {current_level} 完成，生成
{len(next_level_texts)} 个摘要")
39     # 停止条件：只剩一个摘要（根节点）
40     if len(next_level_texts) == 1:
41         print("🎯 到达根节点，树构建完成！")
42         break
43     return tree
44
45 #构建完整的RAPTOR树
46 raptor_tree = build_raptor_tree(
47     texts=[doc.page_content for doc in leaf_texts],
48     max_depth=3,
49     min_cluster_size=3
50 )

```

3.4 RAPTOR检索策略

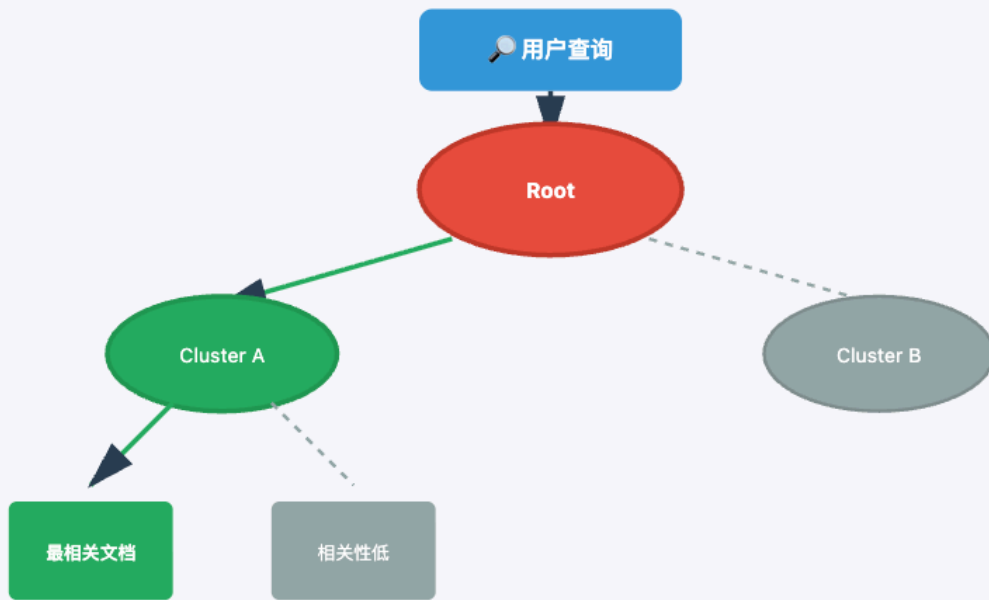
RAPTOR提供了两种检索策略，各有优劣：

策略1：树遍历检索 (Tree Traversal)

从根节点开始，逐层向下找最相关的分支，像在图书馆里先找分类，再找书架，最后找书。

【树遍历检索流程】 **

🔍 树遍历检索流程



✅ 只遍历相关路径，效率高

❌ 可能错过其他分支的相关内容

代码块

```
1 def tree_traversal_retrieval(query, tree, k=3):
2     """
3     树遍历检索
4     Args:
5         query: 用户查询
6         tree: RAPTOR树结构
7         k: 每层选择的top-k个节点
8     Returns:
9         最相关的文档列表
10    """
11    query_embedding = embeddings.embed_query(query)
12    # 从最高层开始
13    max_level = max([int(k.split('.')[1]) for k in tree.keys()])
14    current_nodes = tree[f'level{max_level}']
15    for level in range(max_level, -1, -1):
16        print(f"🔍 在 Level {level} 搜索...")
17        # 计算相似度
18        level_embeddings =
19        embeddings.embed_documents(current_nodes)
20        similarities = cosine_similarity(
21            [query_embedding],
22            level_embeddings
23        )[0]
24        # 选择top-k最相似的节点
25        top_k_indices = np.argsort(similarities)[-k:][::-1]
26        if level == 0:
27            # 到达叶子层，返回结果
```

```

27         return [current_nodes[i] for i in top_k_indices]
28     # 获取下一层的子节点
29     next_level_nodes = []
30     for idx in top_k_indices:
31         # 这里需要维护父子关系映射
32         children = get_children(tree, level, idx)
33         next_level_nodes.extend(children)
34     current_nodes = next_level_nodes
35     return current_nodes

```

策略2：扁平化检索 (Collapsed Tree)

把所有层的节点（包括原始文档和各层摘要）都放在一起，一次性检索。简单高效，是推荐的方法！

代码块

```

1  def collapsed_tree_retrieval(query, tree, k=5):
2      """
3      扁平化检索 - 将所有层级的节点放在一起检索
4      这是推荐的方法，因为：
5      1. 实现简单
6      2. 不会错过任何相关信息
7      3. 可以同时获得不同抽象层次的内容
8      """
9      # 1. 收集所有层的所有节点
10     all_nodes = []
11     node_metadata = [] # 记录每个节点的元信息
12     for level_key, nodes in tree.items():
13         level = int(level_key.split('_')[1])
14         for i, node in enumerate(nodes):
15             all_nodes.append(node)
16             node_metadata.append({
17                 'level': level,
18                 'index': i,
19                 'is_leaf': (level == 0)
20             })
21     print(f"📁 共收集到 {len(all_nodes)} 个节点")
22     # 2. 向量化所有节点
23     all_embeddings = embeddings.embed_documents(all_nodes)
24     # 3. 向量化查询
25     query_embedding = embeddings.embed_query(query)
26     # 4. 计算相似度并排序
27     from sklearn.metrics.pairwise import cosine_similarity
28     similarities = cosine_similarity(
29         [query_embedding],
30         all_embeddings
31     )[0]
32     # 5. 获取top-k
33     top_k_indices = np.argsort(similarities)[-k:][::-1]
34     # 6. 返回结果，包含元信息
35     results = []
36     for idx in top_k_indices:
37         results.append({
38             'content': all_nodes[idx],
39             'similarity': similarities[idx],
40             'metadata': node_metadata[idx]
41         })
42     return results
43
44 #使用示例

```

```
45 query = "AI发展经历了哪些重要阶段？"
46 results = collapsed_tree_retrieval(query, raptor_tree, k=5)
47
48 for i, result in enumerate(results):
49     print(f"\n📄 结果 {i+1} (相似度: {result['similarity']:.3f})")
50     print(f"层级: Level {result['metadata']['level']}")
51     print(f"内容: {result['content'][:200]}...")
```

💡 为什么推荐扁平化检索？

- **全面性**：不会因为在某一层选错分支而错过相关内容
- **灵活性**：对于不同抽象层次的问题，都能找到合适的答案
- **实现简单**：不需要维护复杂的父子关系，一次向量化搞定
- **实验验证**：RAPTOR论文实验表明，扁平化检索在大多数任务上优于树遍历

举个例子：当用户问"AI发展经历了哪些阶段"，扁平化检索可能同时返回：

- Level 2的全局摘要（提供整体框架）
- Level 1的具体阶段摘要（提供中层细节）
- Level 0的某些原始文档（提供具体例证）

这样的组合提供了多层次、立体化的答案！

3.5 RAPTOR性能分析与优化

性能数据对比

指标	传统RAG	RAPTOR	提升幅度
简单事实查询准确率	78%	82%	0.04
复杂推理问题准确率	54%	74%	0.2
高层次概括问题	42%	71%	0.29
QuALITY基准测试	53.70%	73.40%	0.2
索引构建时间	5分钟	35分钟	6
存储空间	100MB	180MB	0.8
检索速度	150ms	165ms	0.1

📊 数据解读

RAPTOR在**复杂问题**和**高层次理解**方面有巨大优势（提升20%-29%），但代价是：

- **构建时间长**：需要多轮LLM调用生成摘要，时间是传统方法的7倍
- **存储开销大**：需要存储所有层级的节点，空间增加80%
- **检索略慢**：扁平化方法需要搜索更多节点

适用场景：对于需要理解长文档、回答复杂问题、进行多跳推理的应用，RAPTOR非常值得！

优化建议

1. 缓存机制

构建一次树就缓存起来，避免重复计算：

代码块

```
1 import pickle
```

```

2  import hashlib
3
4  def cache_raptor_tree(tree, docs_hash):
5      """缓存RAPTOR树"""
6      cache_file = f"raptor_tree_{docs_hash}.pkl"
7      with open(cache_file, 'wb') as f:
8          pickle.dump(tree, f)
9
10 def load_raptor_tree(docs_hash):
11     """加载缓存的树"""
12     cache_file = f"raptor_tree_{docs_hash}.pkl"
13     if os.path.exists(cache_file):
14         with open(cache_file, 'rb') as f:
15             return pickle.load(f)
16     return None
17
18 #使用
19 docs_hash = hashlib.md5(str(documents).encode()).hexdigest()
20 tree = load_raptor_tree(docs_hash)
21 if tree is None:
22     tree = build_raptor_tree(documents)
23     cache_raptor_tree(tree, docs_hash)

```

2. 并行化处理

摘要生成可以并行化，大幅提升速度：

代码块

```

1  from concurrent.futures import ThreadPoolExecutor
2
3  def parallel_summarize(clusters, max_workers=5):
4      """并行生成摘要"""
5      with ThreadPoolExecutor(max_workers=max_workers) as executor:
6          summaries = list(executor.map(
7              summarize_cluster,
8              [cluster_texts for cluster_texts in clusters.values()]
9          ))
10     return summaries
11
12 #速度提升5倍!

```

3. 使用更快的LLM

生产环境中，可以用较小的模型生成摘要：

代码块

```

1  #用GPT-3.5生成摘要，速度快3倍，成本降低10倍
2  fast_llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
3
4  #或者用开源模型
5  from langchain.llms import LlamaCpp
6  local_llm = LlamaCpp(model_path="./models/llama-2-7b.gguf")

```

四、ColBERT（Token级别上下文检索）

4.1 核心思想

ColBERT(Contextualized Late Interaction over BERT)是一个非常巧妙的模型！传统的embedding把整个文档压缩成一个向量，而ColBERT把每个token（词）都变成一个向量，保留了更细粒度的信息。

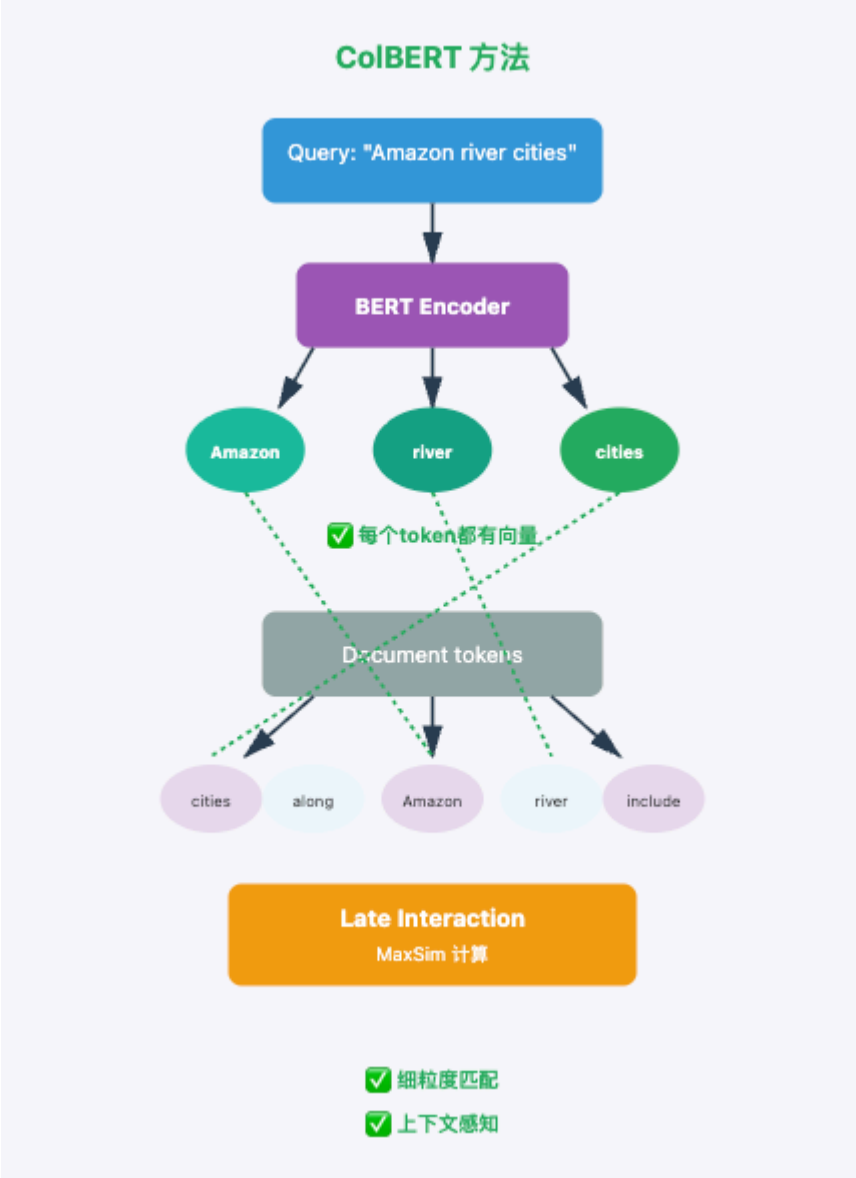
💡 形象的比喻

想象你要找一本书：

- **传统方法**：就像只看书的封面和目录（单向量）
- **ColBERT方法**：就像把书的每一页都看了一遍（多向量）

哪个更可能找到你要的内容？显然是后者！

4.2 ColBERT详细架构



4.3 MaxSim算法详解

ColBERT的核心算法是**MaxSim**（Maximum Similarity）。让我用一个具体例子来解释：

场景设置

假设：

- 查询Q = ["major", "cities", "Amazon", "river"]（4个token）
- 文档D = ["cities", "along", "the", "Amazon", "river", "include", "Manaus", "and", "Iquitos"]（9个token）

MaxSim算法流程

Step 1: 向量化

代码块

```
1 Q = [v_major, v_cities, v_Amazon, v_river] ∈ ℝ^(4×128)
```

```
2 D = [v_cities, v_along, ..., v_Iquitos] ∈ ℝ^(9×128)
```

每个token都是一个128维向量

Step 2: 对查询的每个token，找文档中最相似的token

代码块

```
1 MaxSim(v_major, D) = max(sim(v_major, v_i)) for all v_i ∈ D
2 MaxSim(v_cities, D) = max(sim(v_cities, v_i)) for all v_i ∈ D
3 ...
```

Step 3: 求和得到最终相似度分数

代码块

```
1 Score(Q, D) = Σ MaxSim(v_qi, D)
```

将每个查询token的最大相似度加起来

具体计算示例

代码块

```
1 import numpy as np
2 from sklearn.metrics.pairwise import cosine_similarity
3
4 #假设的向量（简化为3维演示）
5 query_vectors = np.array([
6     [0.8, 0.2, 0.1], # "major"
7     [0.1, 0.9, 0.2], # "cities"
8     [0.3, 0.1, 0.8], # "Amazon"
9     [0.2, 0.3, 0.7], # "river"
10 ])
11
12 doc_vectors = np.array([
13     [0.2, 0.9, 0.1], # "cities"
14     [0.5, 0.4, 0.3], # "along"
15     [0.4, 0.1, 0.9], # "Amazon"
16     [0.3, 0.2, 0.8], # "river"
17     [0.6, 0.3, 0.2], # "Manaus"
18 ])
19
20 def maxsim_score(query_vecs, doc_vecs):
21     """
22     计算MaxSim分数
23     """
24     total_score = 0
25     for i, q_vec in enumerate(query_vecs):
26         # 计算这个查询token与所有文档token的相似度
27         similarities = cosine_similarity([q_vec], doc_vecs)[0]
28         # 取最大值
29         max_sim = np.max(similarities)
30         max_idx = np.argmax(similarities)
31         total_score += max_sim
32         print(f"查询token {i} 最匹配文档token {max_idx}, 相似度:
33             {max_sim:.3f}")
34     return total_score
35
36 score = maxsim_score(query_vectors, doc_vectors)
```

```
36 print(f"\n总分数: {score:.3f}")
```

💡 为什么MaxSim这么有效?

1. **细粒度匹配**: 每个词都能找到文档中最相关的对应, 不会被无关词干扰
2. **上下文感知**: 因为用的是BERT, 每个token的向量都包含了上下文信息
3. **灵活性**: 查询和文档的长度可以完全不同, 算法会自动适应
4. **可解释性**: 可以看到每个查询词具体匹配到文档的哪个词

4.4 ColBERT实战代码

好消息! 我们有一个超级好用的库**RAGatouille**, 让ColBERT的使用变得超级简单:

第一步: 安装和初始化

代码块

```
1  #安装
2  !pip install ragatouille
3
4  #初始化ColBERT模型
5  from ragatouille import RAGPretrainedModel
6
7  #加载预训练的ColBERT v2模型
8  RAG = RAGPretrainedModel.from_pretrained("colbert-ir/colbertv2.0")
9
10 print("✅ ColBERT模型加载完成!")
```

第二步: 索引文档

代码块

```
1  #准备文档
2  documents = [
3      "RAG is a technique that combines retrieval and generation.",
4      "ColBERT uses token-level embeddings for better retrieval.",
5      "BERT is a transformer-based language model.",
6      "Vector databases store embeddings for similarity search.",
7      "Large language models can generate human-like text."
8  ]
9
10 #创建索引 - ColBERT会自动处理token化和embedding
11 index_path = RAG.index(
12     collection=documents,
13     index_name="my_colbert_index",
14     max_document_length=256, # 文档最大长度
15     split_documents=True    # 是否自动切分长文档
16 )
17
18 print(f"✅ 索引创建完成, 保存在: {index_path}")
```

第三步: 搜索

代码块

```
1  #搜索
2  query = "How does ColBERT improve retrieval?"
3
```

```
4 results = RAG.search(
5     query=query,
6     k=3 # 返回top-3结果
7 )
8
9 #打印结果
10 for i, result in enumerate(results):
11     print(f"\n📄 Result {i+1} (Score: {result['score']:.3f})")
12     print(f"Content: {result['content']}")
13     print(f"Document ID: {result['document_id']}")
```

高级用法：Reranking

ColBERT的一个杀手级应用是用来**重排序**！你可以先用其他方法（如BM25或简单的向量检索）快速找到候选集，然后用ColBERT精排。

代码块

```
1 #假设你已经用BM25找到了100个候选文档
2 candidate_docs = [...] # 100个文档
3
4 #用ColBERT重排序 - 不需要预先建立索引!
5 colbert = RAGPretrainedModel.from_pretrained("colbert-
6 ir/colbertv2.0")
7
8 reranked_results = colbert.rerank(
9     query="What is machine learning?",
10    documents=candidate_docs,
11    k=10 # 只保留top-10
12 )
13
14 #ColBERT会根据MaxSim分数重新排序
15 for i, doc in enumerate(reranked_results):
16     print(f"{i+1}. {doc['content'][:100]}... (Score:
17 {doc['score']:.3f})")
```

4.5 ColBERT性能与权衡

指标	单向量方法	ColBERT	差异
检索准确率	65%	82%	0.17
复杂查询表现	59%	79%	0.2
检索速度	50ms	180ms	2.6
索引大小(10M文档)	61GB	768GB	11.58
内存占用	2GB	48GB	23

⚖️ ColBERT的权衡

优势：

- 检索质量大幅提升（+17%到+20%）
- 对复杂查询和长尾词表现优异
- 可解释性强，能看到词级别的匹配

劣势：

- 存储开销巨大（12倍！）
- 检索速度较慢（3倍多）

- 需要更多内存（24倍！）

推荐策略：

- 小规模数据（<100万文档）：直接用ColBERT
- 大规模数据（>1000万文档）：用混合方法，先用向量粗排，ColBERT精排
- 对质量要求极高：咬咬牙用ColBERT，存储成本值得！

4.6 ColBERT v2的优化

ColBERT v2在v1的基础上做了重大优化，主要是**压缩**：

残差压缩 (Residual Compression)

ColBERT v2使用了一个聪明的技巧：每个token向量不是完整存储，而是存储为一个**质心 (centroid)**加上一个**残差(residual)**。

残差压缩公式

代码块

```
1  v_token = centroid_k + residual_token
2
3  其中：
4  • centroid_k 是最近的质心向量（从少量质心中选择）
5  • residual_token 是低精度的差值（1-2bit量化）
6
7  存储从 128×32bit 降低到  $ID \times \log_2(n\_centroids) + 128 \times 1bit$ 
8  约为原来的1/6!
9  #ColBERT v2的索引自动使用压缩
10 RAG_v2 = RAGPretrainedModel.from_pretrained("colbert-
    ir/colbertv2.0")
11
12 #创建压缩索引
13 index = RAG_v2.index(
14     collection=documents,
15     index_name="compressed_index",
16     # ColBERT v2会自动：
17     # 1. 学习质心（通常2048或4096个）
18     # 2. 对残差进行1-2bit量化
19     # 3. 存储压缩后的表示
20 )
21
22 #索引大小从 154GB 降低到 16GB (1bit) 或 25GB (2bit)
23 #检索质量基本不变！
```

五、三种方法对比与选型指南

5.1 全面对比

特性	Multi-representation	RAPTOR	ColBERT
核心思想	双存储：摘要检索，原文返回	层次树：递归摘要，多层抽象	Token级：细粒度向量，后期交互
检索准确率提升	0.13	19.71	16.8
对简单问题	★★★★★	★★★★	★★★★★★
对复杂问题	★★★★	★★★★★★	★★★★★
实现复杂度	中	高	低（有库）
构建时间	0.12	6	0.5
存储开销	0.2	0.8	588
检索速度	0.12	0.1	2.6
最适合场景	通用RAG优化	长文档、多跳推理	精准检索、专业领域

5.2 选型决策树



六、Retrieval检索和Generation生成优化策略

嗨，朋友们！今天咱们来深入聊聊RAG系统中最关键的两个环节——**检索（Retrieval）**和**生成（Generation）**的优化策略。你可能会问，为啥要优化这两个环节？简单来说，就是要让AI系统更聪明、更准确、更靠谱！

想象一下，你在图书馆找资料，如果你找到的书跟你要查的内容八竿子打不着，那后面写的报告肯定也是驴唇不对马嘴。RAG系统也是一样的道理——检索质量决定了生成质量。

6.1 Retrieval检索优化策略

什么是检索优化？

检索优化就是在索引的基础上，通过各种技巧和策略，让系统更准确地找到用户真正需要的信息。咱们可以把它想象成一个超级图书管理员，不仅知道书在哪儿，还知道哪本书最适合你的需求。

完成对问题的改写、不同知识库查询的构建以及路由分发、查询构建和索引生成优化之后，我们可以进一步优化Retrieval检索。主要包括以下几个方面：

- 1. **Ranking（排序）**：对检索结果进行初步排序
- 2. **Refinement（精炼）**：对检索结果进行过滤和优化
- 3. **Adaptive Retrieval（自适应检索）**：根据查询特征动态调整检索策略

检索的几种主要类型

1. 父文档检索（Parent Document Retrieval）

这个概念听起来有点抽象，我用个生活化的例子来解释：

假设你在看一本书，目录上写着"第三章：人工智能的发展"。你点开目录（小分块），但实际阅读的是整个第三章的内容（父文档）。这就是父文档检索的核心思想！

为什么需要父文档检索？

- **检索时**：小分块更精准，embedding能更准确地反映含义
- **生成时**：大文档更完整，能保留足够的上下文信息

三种实现方式：

方式一：小块检索 → 父文档返回

代码块

1 检索小分块 → 找到对应的父文档 → 返回完整的父文档/更大的分块

方式二：摘要检索 → 原文返回

代码块

1 LLM生成文档摘要 → 对摘要进行embedding → 检索摘要 → 返回原始文档

方式三：假设性问题检索

代码块

1 LLM为文档生成假设性问题 → embedding问题 → 检索问题 → 返回原文档

💡 举个实际例子：

代码块

1 原始文档（1000字）：详细介绍了如何使用Python进行数据分析...
2 ↓
3 生成摘要（100字）："本文介绍Python数据分析的核心技术..."
4 ↓
5 用户查询："如何用Python做数据分析？"
6 ↓

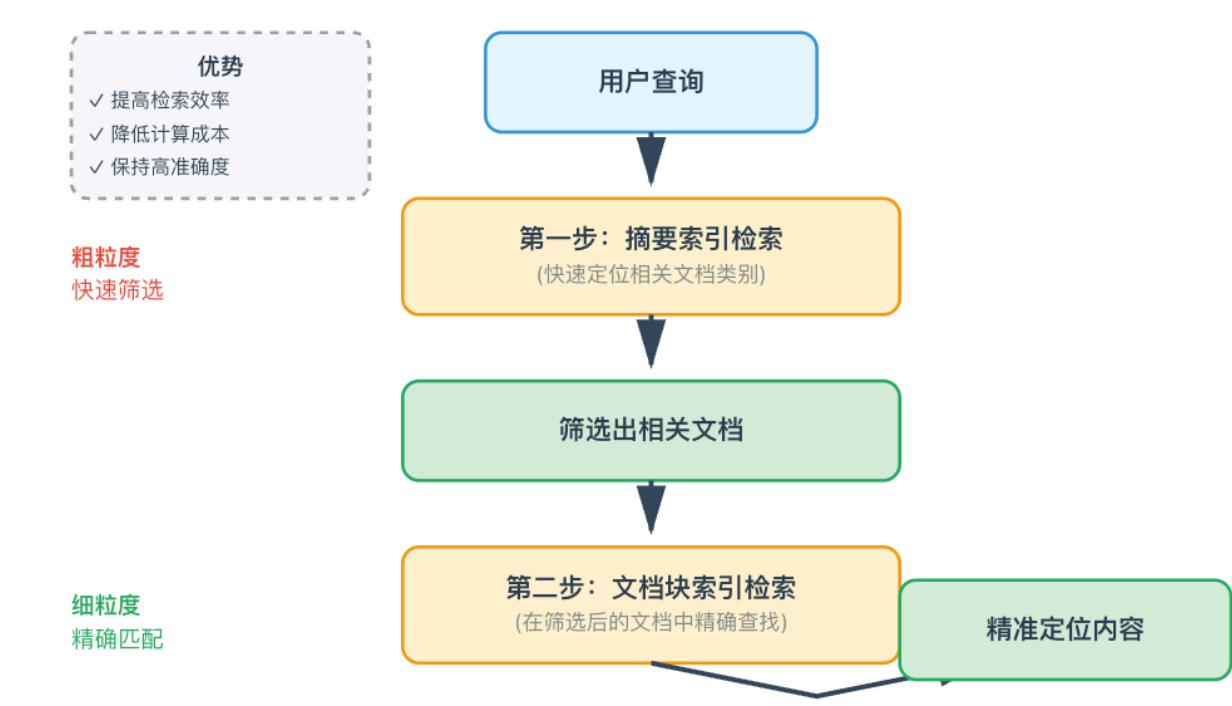
2. 层级检索（Hierarchical Retrieval）

想象你在一个超大型图书馆找书，如果逐本翻看，效率太低。更聪明的做法是：

1. 先看每个分类的总目录（摘要）
2. 确定大致范围
3. 再在特定分类里细找（文档块）

两步检索策略：

层级检索（Hierarchical Retrieval）流程



实际案例：RAPTOR（层级检索的经典实现）

RAPTOR将文档组织成树状结构：

- 叶节点：原始文档块
- 中间节点：多个文档块的摘要
- 根节点：整个文档集的总摘要

3. 混合检索（Fusion Retrieval）

现实世界的检索需求千变万化，单一方法往往力不从心。混合检索就是把不同检索策略的优势结合起来！

最常见的组合：稀疏检索 + 密集检索

特性	向量搜索（密集检索）	关键词搜索（稀疏检索）
语义理解	✔ 能理解上下文和语义	✘ 仅基于字面匹配
同义词处理	✔ 自然处理	✘ 需要额外配置
精确匹配	✘ 不擅长	✔ 擅长精确匹配
数字和ID查询	✘ 表现较差	✔ 表现出色
专有名词	✘ 可能无法准确表示	✔ 能准确匹配
计算效率	✘ 计算成本高	✔ 更快、更高效

举个例子：

代码块

```
1  用户查询: "iPhone 15 Pro的价格"
2
3  向量搜索: 可能找到关于"苹果新款手机定价"的内容
4  关键词搜索: 精确匹配"IPhone 15 Pro"和"价格"
5
6  混合检索结果 = 两者结合, 既准又全!
```

RRF算法详解（Reciprocal Rank Fusion）

RRF是混合检索中最核心的融合算法，由滑铁卢大学和谷歌联合开发。

算法公式：

代码块

```
1  $$ RRF(d) = \sum_{i=1}^n \frac{1}{k + r_i(d)} $$
```

其中：

- \$d\$ 是文档
- \$n\$ 是检索系统的数量
- \$r_i(d)\$ 是文档 \$d\$ 在第 \$i\$ 个检索器中的排名（排名越靠前值越小）
- \$k\$ 是常数，通常取60，用于防止低排名项权重过高

实际计算案例：

假设我们有两个检索器（向量搜索和关键词搜索），检索结果如下：

代码块

```
1  文档A:
2  向量搜索排名: 1
3  关键词搜索排名: 3
4  RRF(A) = 1/(60+1) + 1/(60+3) = 1/61 + 1/63 = 0.0164 + 0.0159 = 0.0323
5
6  文档B:
7  向量搜索排名: 2
8  关键词搜索排名: 1
```

| Cohere Rerank（商业化首选）

 刘谋刚 7月14日 13:18
就是直接用规定好的方法

```
9   RRF(B) = 1/(60+2) + 1/(60+1) = 1/62 + 1/61 = 0.0161 + 0.0164 =
      0.0325
10
11  文档C:
12  向量搜索排名: 5
13  关键词搜索排名: 2
14  RRF(C) = 1/(60+5) + 1/(60+2) = 1/65 + 1/62 = 0.0154 + 0.0161 =
      0.0315
```

最终排序：B > A > C

为什么k设为60？

k=60是经验值，这个值能够很好地平衡不同排名位置的权重：

- 如果k太小（如k=10），低排名文档权重过高
- 如果k太大（如k=200），高排名文档优势不明显

权重调整技巧：

你可以通过调整k值来改变不同检索器的相对重要性：

```
代码块

1  #更重视向量搜索
2  k_vector = 30
3  k_keyword = 90
4
5  #更重视关键词搜索
6  k_vector = 90
7  k_keyword = 30
```

TM2C2算法：RRF的进化版

2023年，Pinecone和伯克利的研究人员提出了TM2C2算法（Theoretical Min-Max Convex Combination），它有以下优势：

1. 更稳定：相比传统的min-max归一化
2. 性能更好：在大多数数据集上优于RRF
3. 更可解释：α参数直观表示语义搜索和关键词搜索的相对重要性
4. 样本效率高：只需很少训练样本就能达到好效果

4. 多向量检索（Multi-Vector Retrieval）

一份文档可以有多种embedding方式！就像一个人可以有多张不同角度的照片。

典型场景：半结构化文档（文本+表格）

原始文档：

```
代码块

1  └─ 文本部分 → embedding_text
2  └─ 表格部分 → 生成摘要 → embedding_table_summary
3  └─ 图片部分 → 多模态embedding → embedding_image
4
5  检索时可以同时使用三种embedding，提高召回率！
```

实际案例：

```
代码块
用户查询: "2023年第三季度销售额对比"
2
3 传统方法: 只能检索文本
4 多向量方法:
5     检索文本描述
6     检索表格 (包含具体数据)
7     检索图表 (可视化展示)
8
9 返回时: 摘要用于匹配, 但实际返回原始表格给LLM
```

6.2 Re-ranking重排

为什么需要Re-ranking?

你有没有遇到过这种情况：Google搜索前10个结果里，最有用那个藏在第7位？ Re-ranking就是为了解决这个问题！

核心问题： 检索出来的chunks并不一定完全和问题相关，直接提交给LLM可能导致：

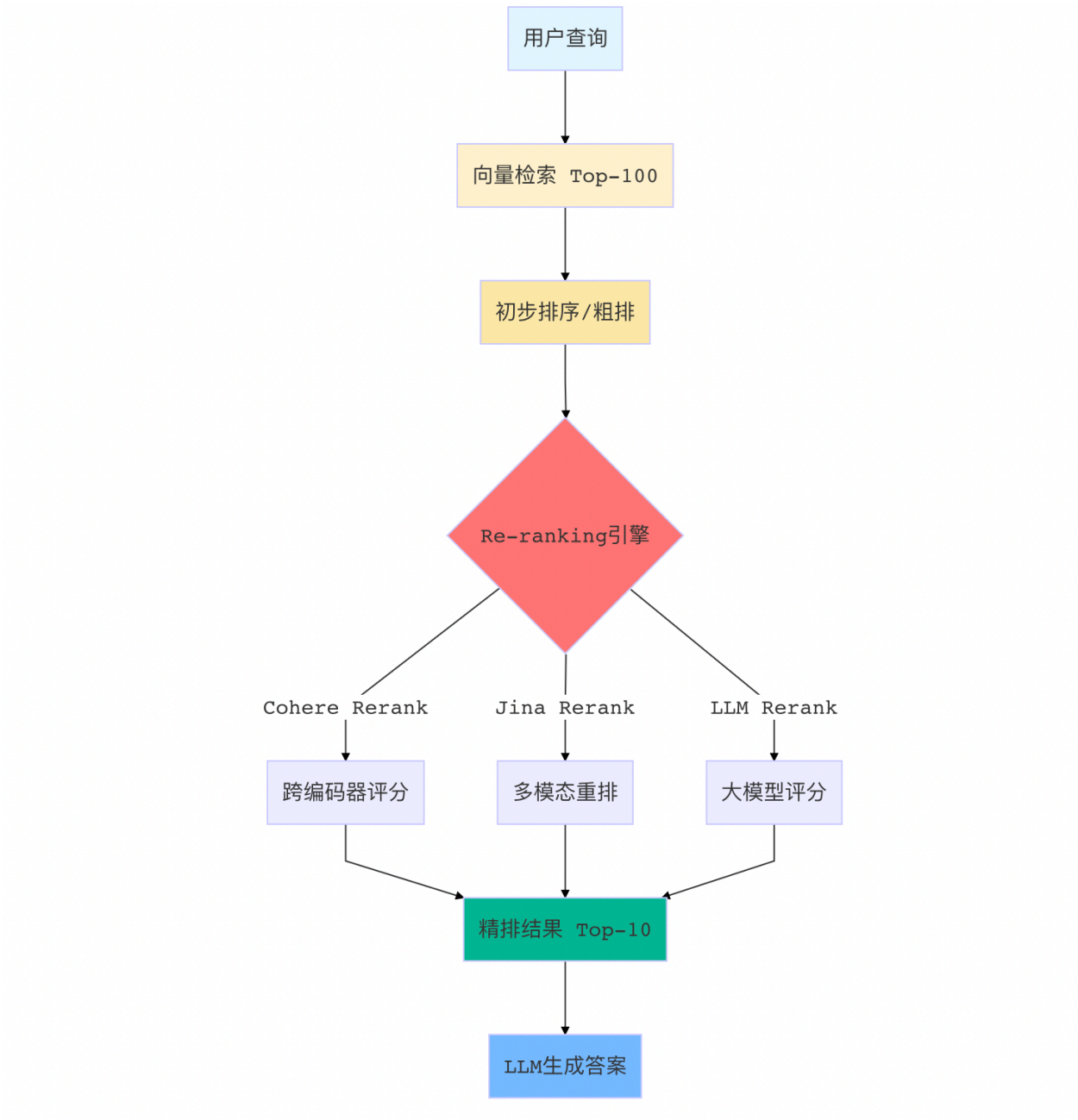
- 生成结果质量不佳
- 包含无关信息
- 浪费token成本

解决思路：借鉴推荐系统的做法

```
代码块
1 粗排 (Coarse Ranking) : 基于Embedding快速筛选
2     ↓
3 精排 (Fine-grained Ranking) : 专门的排序引擎深度评分
4     ↓
5 Top-K选择: 挑选最相关的内容
```

Re-ranking架构图





两种主要实现方式

方式一：专门的排序引擎

1. Cohere Rerank（商业化首选）

Cohere Rerank 3是目前最先进的重排模型，具有以下特点：

核心优势：

- ✨ 4K上下文长度窗口（2025年最新版本）
- ✨ 支持多方面和半结构化数据（邮件、发票、JSON、代码、表格）
- ✨ 100+语言支持
- ✨ 低延迟、低成本
- ✨ 企业级精度

工作原理：

Cohere Rerank使用交叉编码器（Cross-Encoder）架构：

代码块

- 1 输入：（查询，文档） 配对
- 2 ↓
- 3 Transformer同时处理查询和文档
- 4 ↓
- 5 深度交互捕获语义关系
- 6 ↓
- 7 输出：相关性分数（0-1之间）

实际代码示例（LangChain）：

代码块

```
1  from langchain_cohere import CohereRerank
2  from langchain_core.documents import Document
3
4  #初始化Cohere Rerank
5  cohere_rerank = CohereRerank(
6      api_key="your_api_key",
7      model="rerank-english-v3.0",  # 2025最新模型
8      top_n=5
9  )
10
11 #查询
12 query = "What is the capital of the United States?"
13
14 #候选文档
15 docs = [
16     Document(pageContent="Carson City is the capital of Nevada..."),
17     Document(pageContent="Washington, D.C. is the capital and largest city of the United States..."),
18     Document(pageContent="The Commonwealth of the Northern Mariana Islands..."),
19     Document(pageContent="Capital punishment has existed in the US since...")
20 ]
21
22 #重排序
23 reranked_docs = cohere_rerank.rerank(docs, query, top_n=5)
24
25 #结果示例:
26 [
27     { index: 1, relevanceScore: 0.987 },  # Washington, D.C.
28     { index: 0, relevanceScore: 0.089 },  # Carson City
29     { index: 2, relevanceScore: 0.041 }   # Commonwealth
30 ]
```

Cohere Rerank的性能指标：

代码块

1 指标对比（基于BeIR Benchmark）：

2

模型	NDCG@10	延迟 (ms)
Cohere Rerank 3	0.582	45

6	Cohere Rerank 2	0.551	38	
7	BGE Reranker	0.534	52	
8	Cross-Encoder Mini	0.498	28	
9				

成本效益分析：

使用Rerank的ROI（投资回报率）：

```
代码块

1  场景：企业知识库搜索（10M文档）
2
3  不使用Rerank：
4  需要返回Top-50给LLM
5  GPT-4 Token成本：$0.03/1K tokens
6  平均每次查询tokens：15,000
7  单次成本：$0.45
8  准确率：72%
9
10 使用Rerank：
11 初始检索Top-100
12 Rerank成本：$0.002/查询
13 只返回Top-10给LLM
14 平均每次查询tokens：3,000
15 LLM成本：$0.09
16 总成本：$0.092
17 准确率：89%
18
19 节省：79.6%
20 准确率提升：17个百分点
```

2. Jina AI Rerank

Jina Rerank专注于多模态重排：

特点：

- 🎨 支持文本、图像、音频的联合重排
- 🌐 多语言支持
- 🚀 可本地部署
- 📁 企业级安全

3. ColBERT Reranker

ColBERT（Contextualized Late Interaction over BERT）是一种快速准确的检索模型：

核心创新：

```
代码块

1  传统Cross-Encoder：
2  Query + Doc → BERT → Score
3  （每个配对都要完整计算，慢）
4
5  ColBERT：
6  Query → BERT → Q_vectors（预计算）
7  Doc → BERT → D_vectors（预计算）
8  Score = MaxSim(Q_vectors, D_vectors)（快速计算）
```

性能对比：

代码块

```
1  检索10K文档：
2      Cross-Encoder：~5秒
3      CoLBERT：~50毫秒
4  速度提升：100倍！
```

方式二：使用LLM做重排序

基本思路： 让大模型直接对文档相关性进行评分和排序。

LlamaIndex内置的重排Prompt：

代码块

```
1  以下是一些上下文：
2  {context}
3
4  请根据以下查询对上下文的相关性进行评分（0-10分）：
5  查询：{query}
6
7  对于每个上下文片段，请给出：
8      相关性分数（0-10）
9      简短的理由
10
11 最后，请按相关性从高到低排序。
```

实际示例：

代码块

```
1  from llama_index import GPTListIndex
2  from llama_index.indices.postprocessor import LLMRerank
3
4  #文档列表
5  documents = [...]
6
7  #使用GPT-4进行重排
8  reranker = LLMRerank(
9      choice_batch_size=5,
10     top_n=3,
11     model="gpt-4"
12 )
13
14 query = "如何优化RAG系统的检索性能？"
15 reranked_nodes = reranker.postprocess_nodes(
16     nodes=documents,
17     query_str=query
18 )
```

LLM重排的评分示例：

代码块

```
1  查询: "Python数据分析最佳实践"
2
3  文档A: "本文介绍了使用Pandas进行数据分析的技巧..."
4  评分: 9/10
5  理由: 直接相关，提供了具体的实践方法
6
```

```
7  文档B: "数据科学导论：从统计学到机器学习..."
8  评分: 5/10
9  理由: 内容相关但过于宽泛，缺少Python具体实践
10
11 文档C: "Java编程入门教程..."
12 评分: 1/10
13 理由: 不相关，主题是Java而非Python
```

LLM重排的优缺点

优点：

-  理解能力强，能捕捉复杂的语义关系
-  灵活性高，可以根据需求调整prompt
-  可以提供排序理由，增强可解释性

缺点：

-  速度慢（每次调用LLM API需要时间）
-  成本高（LLM API调用费用）
-  无法全局对齐（分批处理导致的问题）

成本对比：

```
代码块

1  场景: 1000次查询，每次重排10个文档
2
3  使用Cohere Rerank:
4  成本: $2
5  延迟: 45ms/查询
6  总时间: 45秒
7
8  使用GPT-4重排:
9  成本: $30 (估算)
10 延迟: 2000ms/查询
11 总时间: 33分钟
12
13 Cohere Rerank更优: 成本低15倍，速度快44倍
```

高级重排算法

1. RankGPT

论文: *Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents*

创新点：

- Zero-shot重排（无需训练）
- 排列生成方法
- 滑动窗口策略

工作流程：

```
代码块

1  输入: 查询 + 10个文档
2      ↓
3  Prompt: "请将以下文档按相关性排序: ..."
4      ↓
```

```
5 GPT生成: [3, 1, 7, 2, 5, 9, 4, 10, 6, 8]
6     ↓
7 重新排列文档顺序
```

2. RankLLM / RankVicuna / RankZephyr

特点:

- 基于开源LLM (Vicuna, Zephyr)
- 针对重排任务进行微调
- 可本地部署, 无需API调用

微调数据构建:

```
代码块
1  #训练样本格式
2  {
3      "query": "什么是RAG? ",
4      "positive_doc": "RAG是检索增强生成...", # 高相关
5      "hard_negative_doc": "生成式AI是...", # 相关但不准确
6      "negative_doc": "机器学习基础..." # 不相关
7  }
```

实战: 构建高效的Re-ranking Pipeline

```
代码块
1  from langchain_community.vectorstores import Chroma
2  from langchain_openai import OpenAIEmbeddings
3  from langchain_cohere import CohereRerank
4  from langchain.retrievers import ContextualCompressionRetriever
5
6  #步骤1: 初始向量检索 (粗排)
7  vectorstore = Chroma(
8      collection_name="rag_collection",
9      embedding_function=OpenAIEmbeddings()
10 )
11 base_retriever = vectorstore.as_retriever(
12     search_kwargs={"k": 50} # 初始召回50个
13 )
14
15 #步骤2: Cohere重排 (精排)
16 reranker = CohereRerank(
17     api_key="your_api_key",
18     model="rerank-english-v3.0",
19     top_n=5 # 最终返回5个
20 )
21
22 #步骤3: 组合成压缩检索器
23 compression_retriever = ContextualCompressionRetriever(
24     base_compressor=reranker,
25     base_retriever=base_retriever
26 )
27
28 #步骤4: 执行查询
29 query = "如何优化RAG系统性能?"
30 compressed_docs =
31     compression_retriever.get_relevant_documents(query)
```

```
32 #结果：从50个候选中精选出5个最相关的文档
33 for i, doc in enumerate(compressed_docs):
34     print(f"排名 {i+1}: 相关度分数
35           {doc.metadata['relevance_score']:.3f}")
36     print(f"内容预览: {doc.page_content[:100]}...")
```

Re-ranking的性能指标

评估指标:

1. NDCG@K (Normalized Discounted Cumulative Gain)

代码块

```
1
2 DCG@K =  $\sum(\text{rel}_i / \log_2(i+1))$ 
3 NDCG@K = DCG@K / IDCG@K
4
5 rel_i: 第i个位置文档的相关性
6 完美排序时NDCG = 1.0
```

2. MRR (Mean Reciprocal Rank)

代码块

```
1 MRR = 1/N *  $\sum(1/\text{rank}_i)$ 
2
3 rank_i: 第一个相关文档的排名
```

3. Recall@K

代码块

```
1 Recall@K = (相关文档数) / (总相关文档数)
```

实际案例数据:

代码块

```
1 数据集: 10K文档, 100个查询
2 评估指标对比:
3
4
5
6
7
8
9
10
11
12
13 性能提升:
14 Cohere vs 纯向量: NDCG提升36.3%
15 Cohere vs RRF融合: NDCG提升18.9%
```

方法	NDCG@10	MRR	Recall@10
纯向量检索	0.512	0.448	0.653
向量+RRF融合	0.587	0.521	0.721
Cohere Rerank	0.698	0.672	0.843
GPT-4 Rerank	0.715	0.689	0.856

Re-ranking最佳实践建议

1. 选择合适的方法

代码块

- 1 预算充足 + 追求极致性能 → Cohere Rerank 3
- 2 需要本地部署 → ColBERT / SentenceTransformer
- 3 中文场景 → BGE Reranker
- 4 预算有限 + 简单场景 → RRF融合

2. 设置合理的K值

代码块

- 1 粗排Top-K: 50-100 (召回率优先)
- 2 精排Top-N: 3-10 (精确度优先)
- 3
- 4 原则:
- 5 K太小: 可能漏掉相关文档
- 6 N太大: 浪费LLM上下文窗口

3. 混合策略

代码块

- 1 第一阶段: 向量检索 (Top-100)
- 2 第二阶段: RRF融合 (Top-50)
- 3 第三阶段: Cohere Rerank (Top-10)
- 4 第四阶段: LLM生成
- 5
- 6 这样既保证召回, 又控制成本

6.3 CRAG: 纠错检索增强生成

什么是CRAG?

CRAG的全称是**Corrective Retrieval-Augmented Generation** (纠错检索增强生成)。简单来说, CRAG就是给RAG系统加了一个"质检员", 在把检索结果交给LLM之前, 先检查一下这些内容靠不靠谱!

核心理念:

代码块

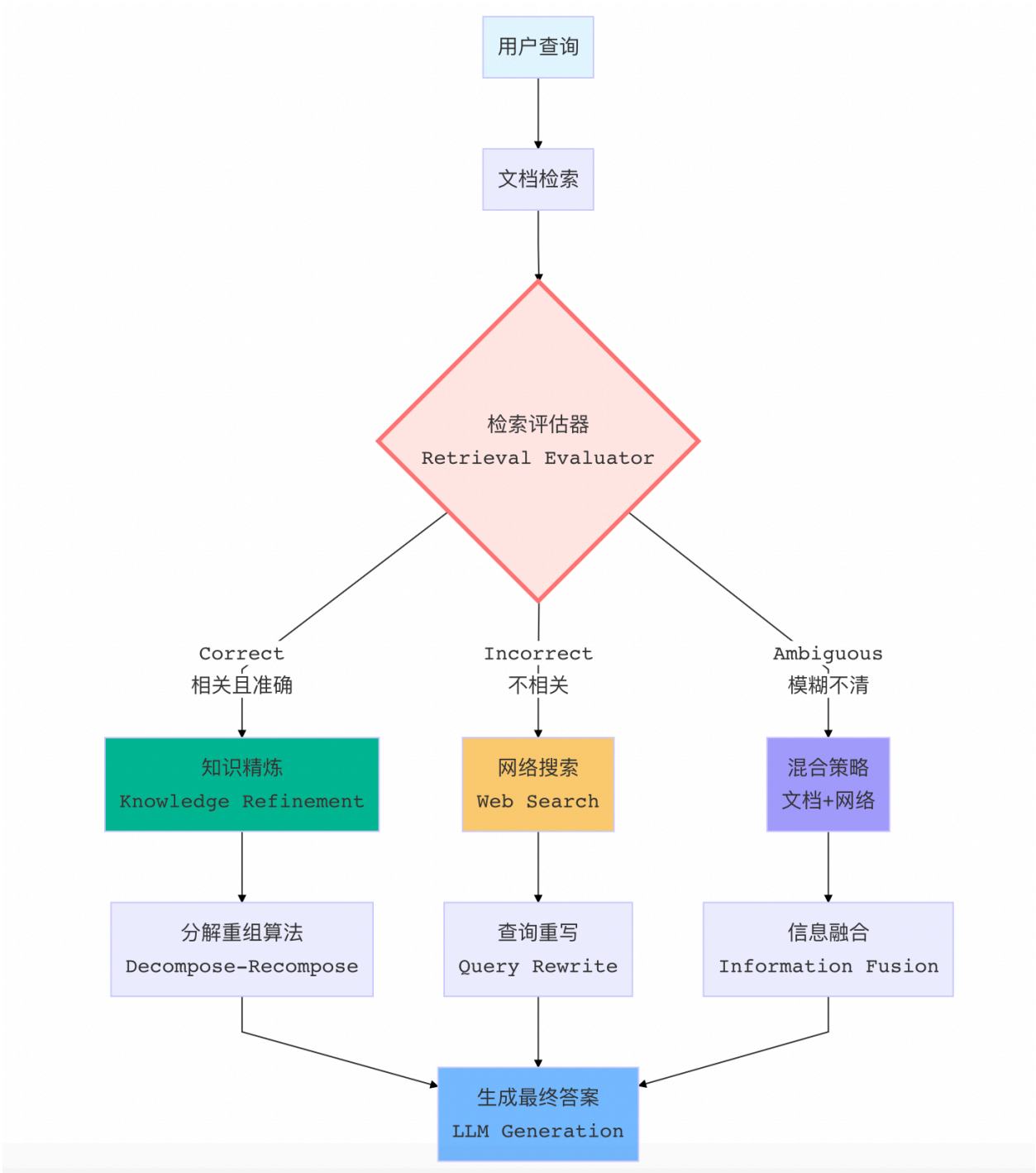
- 1 传统RAG: 检索 → 直接生成
- 2 CRAG: 检索 → 评估质量 → 纠错/补充 → 生成

CRAG架构图

![[image-20251027163118729]](img/ima...



用户669719 7月10日 11:40
资源失效



CRAG的三大核心组件

1. 检索评估器（Retrieval Evaluator）

作用：给每个检索到的文档打分，判断其质量和相关性。

评估维度：

- 相关性（Relevance）：文档是否回答了查询问题？
- 准确性（Accuracy）：信息是否准确可信？
- 完整性（Completeness）：信息是否完整充分？

评分结果分为三类：

```
代码块
class DocumentRelevance(Enum):
    1     CORRECT = "correct"           # 相关且准确, 置信度 > 0.8
    2     INCORRECT = "incorrect"       # 不相关或错误, 置信度 < 0.3
    3     AMBIGUOUS = "ambiguous"       # 模糊不清, 0.3 <= 置信度 <= 0.8
```

评估器实现方式:

方式A: 微调的小模型 (推荐)

```
代码块
1  from transformers import AutoTokenizer,
   AutoModelForSequenceClassification
2
3  #使用微调的T5模型
4  tokenizer = AutoTokenizer.from_pretrained("google/t5-large-crag-
   evaluator")
5  model = AutoModelForSequenceClassification.from_pretrained(
6      "google/t5-large-crag-evaluator",
7      num_labels=3 # Correct, Incorrect, Ambiguous
8  )
9
10 def evaluate_document(query, document):
11     """评估文档相关性"""
12     input_text = f"Query: {query}\nDocument:
   {document}\nRelevance:"
13     inputs = tokenizer(input_text, return_tensors="pt",
   max_length=512)
14     outputs = model(**inputs)
15     # 获取预测结果和置信度
16     probs = torch.softmax(outputs.logits, dim=-1)
17     predicted_class = torch.argmax(probs).item()
18     confidence = probs[0][predicted_class].item()
19     return {
20         "class": ["correct", "incorrect", "ambiguous"]
   [predicted_class],
21         "confidence": confidence
22     }
```

方式B: 使用大模型 (如GPT-4)

```
代码块
1  def gpt4_evaluate(query, document):
2      prompt = f"""
3      评估以下文档是否回答了查询问题。
4      查询: {query}
5      文档: {document}
6      请评估文档的相关性并选择一个类别:
7      - CORRECT: 文档直接回答了查询, 信息准确完整
8      - INCORRECT: 文档不相关或包含错误信息
9      - AMBIGUOUS: 文档部分相关但信息不够完整
10     只返回类别名称, 不需要解释。
11     """
12     response = openai.ChatCompletion.create(
13         model="gpt-4",
14         messages=[{"role": "user", "content": prompt}],
15         temperature=0
16     )
```



```
17         return response.choices[0].message.content.strip()
```

方式C：使用Cohere Rerank作为评估器

代码块

```
1  from langchain_cohere import CohereRerank
2
3  reranker = CohereRerank(
4      model="rerank-english-v3.0",
5      top_n=10
6  )
7
8  #Rerank会返回相关性分数
9  results = reranker.rerank(documents, query)
10
11 #根据分数分类
12 for doc in results:
13     if doc['relevance_score'] > 0.8:
14         doc['category'] = 'CORRECT'
15     elif doc['relevance_score'] < 0.3:
16         doc['category'] = 'INCORRECT'
17     else:
18         doc['category'] = 'AMBIGUOUS'
```

2. 知识精炼（Knowledge Refinement）

当文档被评为"CORRECT"时，我们不是直接使用整个文档，而是进行知识精炼。

分解-重组算法（Decompose-Recompose）：

![image-20251027163118729](img/image-20251027163118729.png)

实际案例：

代码块

```
1  原始文档（500字）：
2  "Python是一种广泛使用的高级编程语言，由Guido van Rossum在1989年创建。
3  Python的设计哲学强调代码可读性，其语法允许程序员用更少的代码表达概念。
4  Python支持多种编程范式，包括面向对象、命令式和函数式编程。
5  Python有一个庞大的标准库，被戏称为'自带电池'。
6  Python被广泛应用于Web开发、数据分析、人工智能、科学计算等领域。
7  （... 还有400字的详细历史和社区信息 ...）"
8
9  查询："Python的主要特点是什么？"
10
11 ↓ 分解成知识条 ↓
12
13 知识条1："Python强调代码可读性" ✓ 相关
14 知识条2："Python语法简洁，用更少代码表达概念" ✓ 相关
15 知识条3："支持多种编程范式" ✓ 相关
16 知识条4："有庞大的标准库" ✓ 相关
17 知识条5："由Guido van Rossum在1989年创建" ✗ 不太相关
18 知识条6："广泛应用于多个领域" ✓ 相关
19
20 ↓ 重组精炼 ↓
21
22 精炼后的上下文（150字）：
23 "Python的主要特点包括：
24 强调代码可读性
25 语法简洁，用更少代码表达概念"
```

- 26 支持多种编程范式（OOP、函数式等）
- 27 拥有庞大的标准库（'自带电池'）
- 28 广泛应用于Web开发、数据分析、AI等领域"

知识精炼的Python实现：

代码块

```
1  def decompose_document(document, query):
2      """将文档分解成知识条"""
3      # 按句子分割
4      sentences = sent_tokenize(document)
5      # 使用小模型评估每个句子的相关性
6      knowledge_strips = []
7      for sent in sentences:
8          relevance_score = compute_relevance(query, sent)
9          if relevance_score > 0.6: # 阈值可调
10             knowledge_strips.append({
11                 'content': sent,
12                 'score': relevance_score
13             })
14     return knowledge_strips
15
16 def recompose_knowledge(knowledge_strips, max_length=500):
17     """重组知识条"""
18     # 按相关性排序
19     sorted_strips = sorted(
20         knowledge_strips,
21         key=lambda x: x['score'],
22         reverse=True
23     )
24     # 组合直到达到最大长度
25     result = []
26     current_length = 0
27     for strip in sorted_strips:
28         strip_length = len(strip['content'])
29         if current_length + strip_length <= max_length:
30             result.append(strip['content'])
31             current_length += strip_length
32         else:
33             break
34     return ' '.join(result)
```

3. 网络搜索（Web Search）

当文档被评为"INCORRECT"时，系统会触发网络搜索来获取更可靠的信息。

查询重写（Query Rewriting）：

原始查询可能不适合网络搜索，需要重写为更适合搜索引擎的形式。

代码块

```
1  用户查询："我们公司去年的销售额增长了多少？"
2  ↓ 检索内部文档 → 评估为INCORRECT
3  ↓ 重写查询（移除公司特定上下文）
4  网络搜索查询："如何计算销售额增长率"
```

实现示例（使用Tavily Search API）：

```

1 from tavily import TavilyClient
2
3 def web_search_fallback(query, original_docs):
4     """当检索失败时触发网络搜索"""
5     # 1. 重写查询
6     rewritten_query = rewrite_query_for_web(query)
7     # 2. 执行网络搜索
8     tavily = TavilyClient(api_key="your_api_key")
9     search_results = tavily.search(
10         query=rewritten_query,
11         search_depth="advanced", # 深度搜索
12         max_results=5
13     )
14     # 3. 提取和格式化结果
15     web_docs = []
16     for result in search_results['results']:
17         web_docs.append({
18             'content': result['content'],
19             'url': result['url'],
20             'score': result['score']
21         })
22     return web_docs
23
24 def rewrite_query_for_web(query):
25     """使用LLM重写查询"""
26     prompt = f"""
27     将以下内部查询重写为适合网络搜索的形式：
28     原查询：{query}
29     重写时应该：
30     - 移除公司特定的上下文
31     - 使用更通用的术语
32     - 保持查询意图不变
33     重写后的查询：
34     """
35     response = llm.invoke(prompt)
36     return response.content.strip()

```

CRAG的完整工作流程

让我们通过一个完整的例子来理解CRAG的工作流程：

场景：医疗知识问答系统

代码块

```

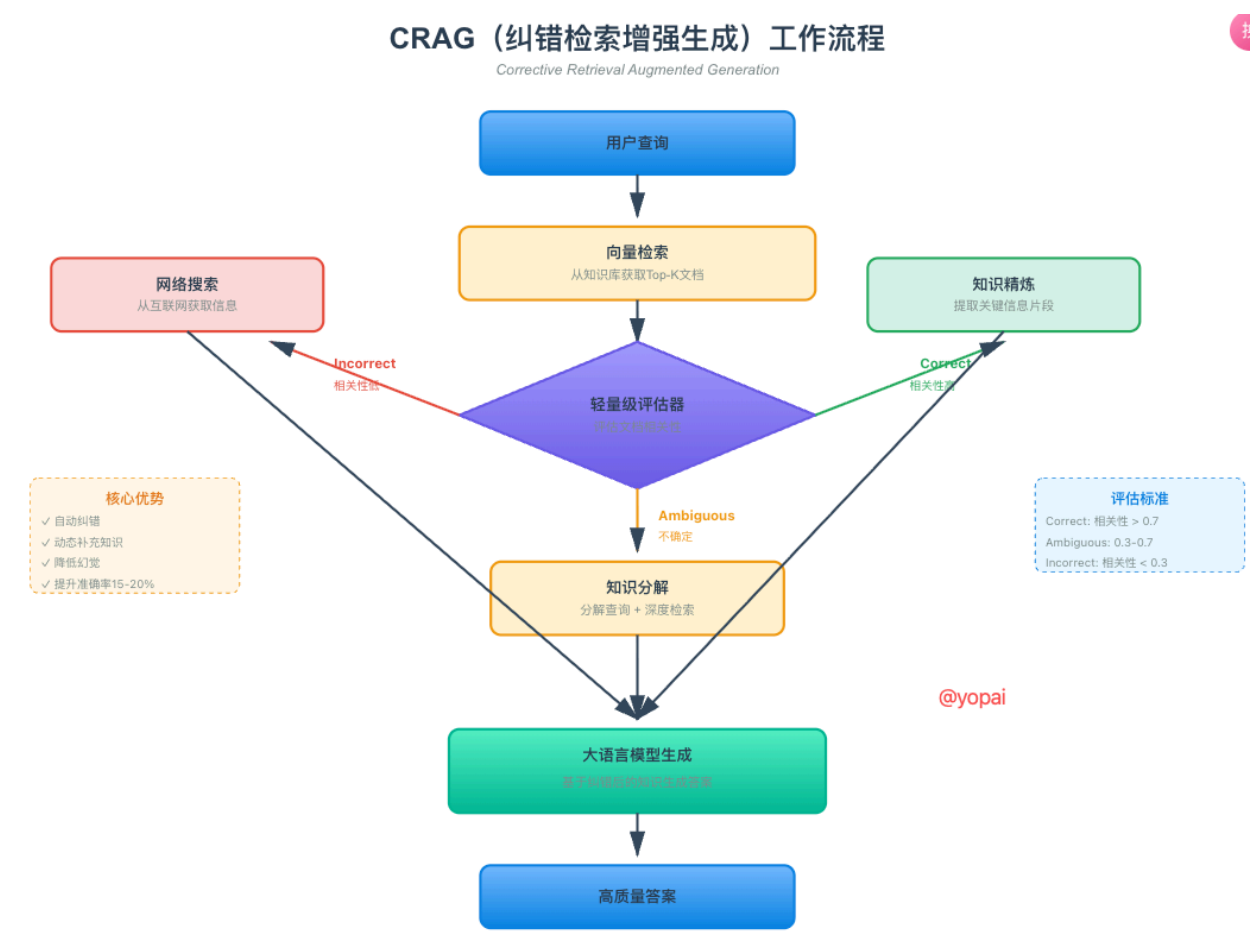
1 #用户查询
2 query = "阿司匹林的主要副作用是什么？"
3
4 #步骤1：初始检索
5 retrieved_docs = vector_store.similarity_search(query, k=5)
6 #返回5个文档
7
8 #步骤2：评估每个文档
9 evaluation_results = []
10 for doc in retrieved_docs:
11     eval_result = evaluate_document(query, doc)
12     evaluation_results.append(eval_result)
13
14 #评估结果示例：
15 Doc1: {"class": "correct", "confidence": 0.92}
16 Doc2: {"class": "correct", "confidence": 0.85}
17 Doc3: {"class": "ambiguous", "confidence": 0.65}

```

```
18 Doc4: {"class": "incorrect", "confidence": 0.25}
19 Doc5: {"class": "incorrect", "confidence": 0.18}
20
21 #步骤3: 根据评估结果采取行动
22 correct_docs = [d for d, e in zip(retrieved_docs,
23                                  evaluation_results)
24                  if e['class'] == 'correct']
25 ambiguous_docs = [d for d, e in zip(retrieved_docs,
26                                    evaluation_results)
27                             if e['class'] == 'ambiguous']
28 incorrect_count = sum(1 for e in evaluation_results if e['class']
29                       == 'incorrect')
30
31 final_context = []
32
33 #3a. 处理CORRECT文档 (知识精炼)
34 if correct_docs:
35     for doc in correct_docs:
36         refined = decompose_and_recompose(doc, query)
37         final_context.append({
38             'source': 'internal',
39             'content': refined
40         })
41
42 #3b. 处理AMBIGUOUS文档 (保留+网络搜索)
43 if ambiguous_docs:
44     # 保留部分内容
45     for doc in ambiguous_docs:
46         refined = decompose_and_recompose(doc, query,
47                                           threshold=0.7)
48         final_context.append({
49             'source': 'internal_ambiguous',
50             'content': refined
51         })
52     # 补充网络搜索
53     web_results = web_search_fallback(query, ambiguous_docs)
54     final_context.extend(web_results)
55
56 #3c. 处理INCORRECT情况 (纯网络搜索)
57 if incorrect_count >= 3 and not correct_docs:
58     # 如果大部分文档不相关, 直接网络搜索
59     web_results = web_search_fallback(query, retrieved_docs)
60     final_context = web_results
61
62 #步骤4: 生成最终答案
63 prompt = build_rag_prompt(query, final_context)
64 answer = llm.invoke(prompt)
65
66 #输出示例:
67 """
68 阿司匹林的主要副作用包括:
69
70 胃肠道反应 (最常见)
71 胃部不适、恶心
72 胃溃疡
73 胃出血
74
75 出血倾向
76 延长出血时间
77 增加出血风险
78
79 过敏反应
```

76 皮疹
77 哮喘（阿司匹林哮喘）
78
79 来源：
80 内部医疗知识库（置信度92%）
81 网络医学资源补充
82 ""

CRAG vs 传统RAG对比



性能对比（基于论文数据）：

代码块

1

测试集：PopQA（流行问题问答）

2

3

方法	准确率	召回率	幻觉率(%)
Vanilla LLM	42.3%	38.1%	28.6%
Standard RAG	56.8%	61.2%	15.3%
CRAG	71.4%	73.9%	4.2%

5

6

7

8

9

10

11

测试集：PubHealth（医疗健康事实验证）

12

13					
14	方法	准确率	F1	精确度	
15					
16	Standard RAG	68.5%	0.672	0.681	
17	CRAG	89.2%	0.887	0.893	
18					

19

20 关键发现：

21 CRAG在医疗领域将幻觉率降低到接近0%

22 准确率提升了25.8个百分点

23 特别适合高风险领域（医疗、法律、金融）

CRAG的LangChain实现

代码块

```
1  from langgraph.graph import StateGraph, END
2  from langchain_openai import ChatOpenAI
3  from langchain_core.prompts import ChatPromptTemplate
4
5  #定义状态
6  class GraphState(TypedDict):
7      query: str
8      documents: List[str]
9      evaluation: str
10     generation: str
11
12     #节点1: 检索
13     def retrieve(state):
14         query = state["query"]
15         documents = vector_store.similarity_search(query, k=5)
16         return {"documents": documents, "query": query}
17
18     #节点2: 评估文档相关性
19     def grade_documents(state):
20         query = state["query"]
21         documents = state["documents"]
22         # 使用LLM评估
23         grader_prompt = ChatPromptTemplate.from_template("""
24         评估文档是否与查询相关。
25         查询: {query}
26         文档: {document}
27         返回 'yes' 如果相关, 'no' 如果不相关。
28         """)
29         filtered_docs = []
30         for doc in documents:
31             grade = grader_prompt | llm
32             result = grade.invoke({"query": query, "document": doc})
33             if "yes" in result.content.lower():
34                 filtered_docs.append(doc)
35         # 判断是否需要网络搜索
36         if len(filtered_docs) == 0:
37             return {
38                 "documents": filtered_docs,
39                 "query": query,
40                 "evaluation": "incorrect"
41             }
42         elif len(filtered_docs) < len(documents) * 0.5:
```

```

43         return {
44             "documents": filtered_docs,
45             "query": query,
46             "evaluation": "ambiguous"
47         }
48     else:
49         return {
50             "documents": filtered_docs,
51             "query": query,
52             "evaluation": "correct"
53         }
54
55 #节点3: 网络搜索
56 def web_search(state):
57     query = state["query"]
58     # 重写查询
59     rewrite_prompt = ChatPromptTemplate.from_template("""
60     重写以下查询使其更适合网络搜索:
61     {query}
62     """)
63     rewritten = rewrite_prompt | llm
64     new_query = rewritten.invoke({"query": query}).content
65     # 执行搜索
66     tavily = TavilyClient(api_key=os.getenv("TAVILY_API_KEY"))
67     results = tavily.search(new_query, max_results=3)
68     web_docs = [r['content'] for r in results['results']]
69     return {"documents": web_docs, "query": query}
70
71 #节点4: 生成答案
72 def generate(state):
73     query = state["query"]
74     documents = state["documents"]
75     # 构建prompt
76     context = "\n\n".join(documents)
77     rag_prompt = ChatPromptTemplate.from_template("""
78     基于以下上下文回答问题:
79     上下文: {context}
80     问题: {query}
81     答案:
82     """)
83     chain = rag_prompt | llm
84     generation = chain.invoke({"context": context, "query": query})
85     return {"generation": generation.content}
86
87 #决策函数
88 def decide_to_generate(state):
89     evaluation = state["evaluation"]
90     if evaluation == "correct":
91         return "generate"
92     elif evaluation == "ambiguous":
93         return "web_search" # 补充网络搜索
94     else: # incorrect
95         return "web_search"
96
97 #构建图
98 workflow = StateGraph(GraphState)
99
100 #添加节点
101 workflow.add_node("retrieve", retrieve)
102 workflow.add_node("grade_documents", grade_documents)
103 workflow.add_node("web_search", web_search)
104 workflow.add_node("generate", generate)

```

| incorrect': 0.2



飞书用户0966LO 4月27日 18:45

这里对吗?



白杰 7月8日 21:05

@飞书用户 0966LO corrnnet
和 incorrent 加起来不一定是 1
吧


```
105
106  #设置入口
107  workflow.set_entry_point("retrieve")
108
109  #添加边
110  workflow.add_edge("retrieve", "grade_documents")
111  workflow.add_conditional_edges(
112      "grade_documents",
113      decide_to_generate,
114      {
115          "generate": "generate",
116          "web_search": "web_search"
117      }
118  )
119  workflow.add_edge("web_search", "generate")
120  workflow.add_edge("generate", END)
121
122  #编译并运行
123  app = workflow.compile()
124
125  #执行查询
126  result = app.invoke({"query": "阿司匹林的主要副作用是什么? "})
127  print(result["generation"])
```

CRAG的优缺点分析

✔ 优点：

- 1. 显著降低幻觉
 - 通过质量评估过滤不可靠信息
 - 医疗领域幻觉率从15%降至4%
- 2. 动态知识补充
 - 检索失败时自动触发网络搜索
 - 保证答案的时效性
- 3. 内容精炼
 - 分解-重组算法去除冗余
 - 优化LLM输入，提升生成质量
- 4. 适应性强
 - 根据文档质量自动调整策略
 - 适用于多种领域

✖ 缺点：

1. 复杂度增加

代码块

1

传统RAG：检索 → 生成（2步）

2

CRAG：检索 → 评估 → 精炼/搜索 → 生成（4步）

3

4

系统复杂度提升100%

5

维护成本增加

2. 延迟增加

- 1 传统RAG延迟：~500ms
- 2 CRAG延迟：~1200ms（增加140%）
- 3
- 4 主要耗时：
- 5 文档评估：200ms
- 6 知识精炼：300ms
- 7 网络搜索（如触发）：700ms

3. 成本上升

代码块

- 1 额外API调用：
- 2 评估器：每个文档1次LLM调用
- 3 查询重写：1次LLM调用
- 4 网络搜索：Tavily API费用
- 5
- 6 总成本增加约60-80%

4. 可扩展性挑战

代码块

- 1 大规模文档（10K+）时：
- 2 评估所有文档耗时长
- 3 需要分批处理
- 4 可能引入全局不一致

CRAG适用场景

强烈推荐：

- 🏥 医疗诊断辅助系统
- ⚖️ 法律文书分析
- 💰 金融风险评估
- 🔬 科研论文问答
- 📰 新闻事实核查

不太适合：

- 🎨 创意写作
- 💬 闲聊对话
- 🎮 娱乐应用
- ⚡ 超低延迟场景

CRAG实战优化技巧

1. 并行处理评估

代码块

- ```
1 from concurrent.futures import ThreadPoolExecutor
2
3 def parallel_evaluate(documents, query, max_workers=5):
4 with ThreadPoolExecutor(max_workers=max_workers) as executor:
5 results = list(executor.map(
6 lambda doc: evaluate_document(query, doc),
7 documents
```

```
8))
9 return results
10
11 #将评估时间从1000ms降至200ms (5个文档)
```

2. 缓存评估结果

代码块

```
1 from functools import lru_cache
2
3 @lru_cache(maxsize=1000)
4 def cached_evaluate(query_hash, doc_hash):
5 return evaluate_document(query, doc)
6
7 #相同查询-文档对无需重复评估
```

3. 智能阈值调整

代码块

```
1 def adaptive_threshold(domain, confidence):
2 """根据领域调整评估阈值"""
3 thresholds = {
4 'medical': {'correct': 0.9, 'incorrect': 0.2},
5 'general': {'correct': 0.7, 'incorrect': 0.3},
6 'creative': {'correct': 0.6, 'incorrect': 0.4}
7 }
8 return thresholds.get(domain, thresholds['general'])
```

4. 增量式网络搜索

代码块

```
1 def incremental_web_search(query, existing_docs, max_searches=3):
2 """逐步增加搜索结果，而非一次性搜索大量"""
3 results = []
4 for i in range(1, max_searches + 1):
5 new_results = tavily.search(query, max_results=i)
6 if is_sufficient(existing_docs + results, query):
7 break
8 results.extend(new_results)
9 return results
```

6.4 Self-RAG：自反思检索增强生成

什么是Self-RAG？

Self-RAG的全称是**Self-Reflective Retrieval-Augmented Generation**（自反思检索增强生成）。如果说CRAG是给RAG加了一个"质检员"，那Self-RAG就是让RAG系统学会了"自我反思"——它能在生成过程中不断问自己："我需要查资料吗？""这个资料靠谱吗？""我生成的内容对吗？"

核心理念：

代码块

```
1 传统RAG：固定检索 → 生成
2 CRAG：检索 → 评估 → 纠错 → 生成
```

Self-RAG的三大创新点

1. 按需检索（On-Demand Retrieval）

- 不是所有查询都需要检索
- 模型自己决定何时需要外部知识

2. 自我反思（Self-Reflection）

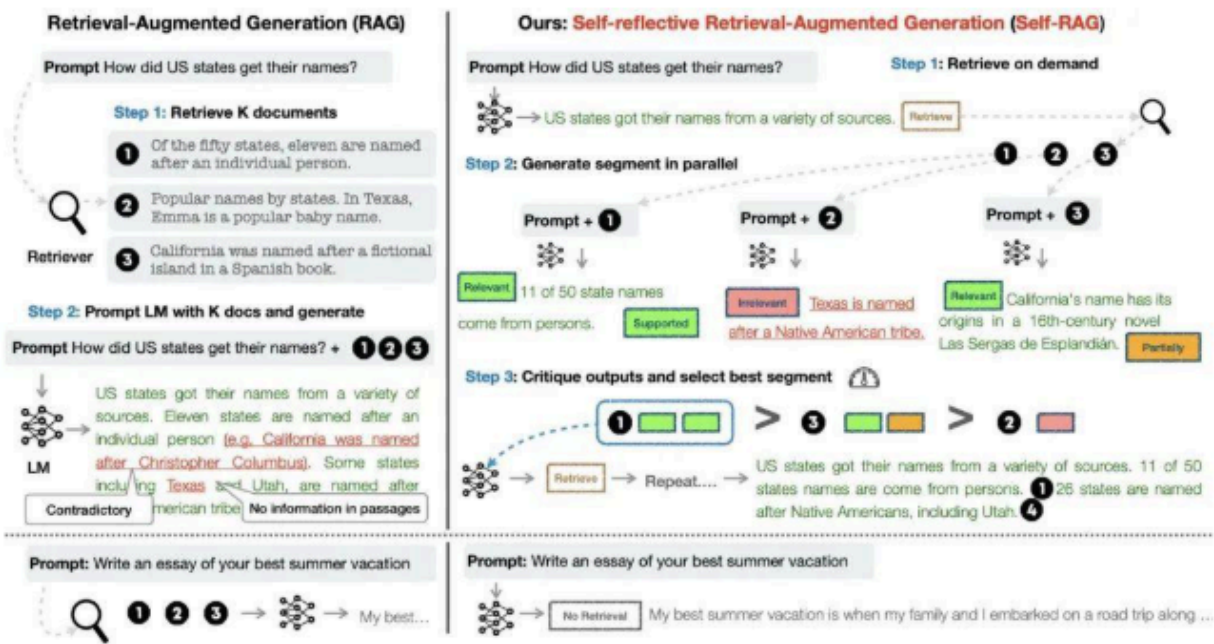
- 评估检索到的文档是否相关
- 评估生成的内容是否有事实支持
- 评估最终答案是否有用

3. 反思令牌（Reflection Tokens）

- 特殊的token指导模型行为
- 可在推理时控制模型的检索和生成策略

Self-RAG参考图

来自网络



反思令牌（Reflection Tokens）详解

Self-RAG通过在词表中添加特殊的反思令牌来控制模型行为。这些令牌分为两大类：

1. 检索令牌（Retrieval Tokens）

Retrieve Token：决定是否需要检索

代码块

```
1 class RetrieveDecision(Enum):
2 YES = "[Retrieve=Yes]" # 需要检索
3 NO = "[Retrieve=No]" # 不需要检索 CONTINUE = "[Retrieve=Continue]" # 继续使用之前检索的内容
```

什么时候生成 [Retrieve=Yes]?

- 查询涉及事实性信息
- 需要最新数据
- 超出模型训练知识范围

什么时候生成 [Retrieve=No]?

- 创意性任务（写诗、写故事）
- 常识性问题
- 数学计算

实际示例：

代码块

```
1 用户查询1: "写一首关于春天的诗"
2 模型输出: [Retrieve=No] 春风拂面暖如酥...
3 (不需要检索, 直接创作)
4
5 用户查询2: "2024年诺贝尔物理学奖得主是谁?"
6 模型输出: [Retrieve=Yes]
7 (需要检索最新信息)
8
9 用户查询3: "继续解释刚才提到的量子纠缠"
10 模型输出: [Retrieve=Continue]
11 (使用之前检索的文档)
```

2. 批评令牌 (Critique Tokens)

ISREL (Is Relevant): 文档相关性判断

代码块

```
1 class DocumentRelevance(Enum):
2 RELEVANT = "[ISREL=Relevant]"
3 IRRELEVANT = "[ISREL=Irrelevant]"
```

ISSUP (Is Supported): 生成内容是否有事实支持

代码块

```
1 class FactualSupport(Enum):
2 FULLY_SUPPORTED = "[ISSUP=FullySupported]"
3 PARTIALLY_SUPPORTED = "[ISSUP=PartiallySupported]"
4 NO_SUPPORT = "[ISSUP=NoSupport]"
```

ISUSE (Is Useful): 生成结果是否有用

代码块

```
1 class ResponseUtility(Enum):
2 USEFUL_5 = "[ISUSE=5]" # 非常有用
3 USEFUL_4 = "[ISUSE=4]"
4 USEFUL_3 = "[ISUSE=3]"
5 USEFUL_2 = "[ISUSE=2]"
6 USEFUL_1 = "[ISUSE=1]" # 完全无用
```

Self-RAG vs CRAG vs 传统RAG对比

RAG优化方法：性能与成本对比总表

| 方法            | 准确率   | 延迟     | 训练成本 | 推理成本 | 复杂度 |
|---------------|-------|--------|------|------|-----|
| 纯向量检索         | ☆☆    | 100ms  | 无    | 低    | 低   |
| RRF融合         | ☆☆☆   | 150ms  | 无    | 低    | 低   |
| Cohere Rerank | ☆☆☆☆  | 250ms  | 无    | 中    | 低   |
| CRAG          | ☆☆☆☆☆ | 800ms  | 低    | 中高   | 中   |
| Self-RAG      | ☆☆☆☆☆ | 1200ms | 极高   | 高    | 高   |

@yopai

评级说明

准确率: ☆越多越好 (Self-RAG最高)

延迟: 越短越好 (纯向量最快)

成本: 越低越好 (RRF最经济)

💡 推荐方案

✓ 快速上手: **RRF融合 + Cohere Rerank**

✓ 平衡选择: **CRAG** (准确+可控+成本合理)

✓ 高端应用: **Self-RAG** (医疗/法律/学术领域)

详细对比表：

|     |        |       |      |          |  |
|-----|--------|-------|------|----------|--|
| 代码块 |        |       |      |          |  |
| 1   |        |       |      |          |  |
| 2   | 特性     | 传统RAG | CRAG | Self-RAG |  |
| 3   |        |       |      |          |  |
| 4   | 是否总是检索 | 是     | 是    | 否        |  |
| 5   | 评估检索质量 | 否     | 是    | 是        |  |
| 6   | 评估生成质量 | 否     | 否    | 是        |  |
| 7   | 动态调整策略 | 否     | 部分   | 是        |  |
| 8   | 支持迭代优化 | 否     | 有限   | 是        |  |
| 9   | 模型可控性  | 低     | 中    | 高        |  |
| 10  | 系统复杂度  | 低     | 中    | 高        |  |
| 11  | 训练成本   | 无需    | 低    | 高        |  |
| 12  | 推理延迟   | 快     | 中    | 慢        |  |
| 13  | 准确率    | 中等    | 高    | 最高       |  |
| 14  |        |       |      |          |  |

性能对比（基于论文实验）：

|     |                  |       |          |        |  |
|-----|------------------|-------|----------|--------|--|
| 代码块 |                  |       |          |        |  |
| 1   | 测试集：PopQA（开放域问答） |       |          |        |  |
| 2   |                  |       |          |        |  |
| 3   |                  |       |          |        |  |
| 4   | 方法               | 准确率   | F1 Score | 延迟(ms) |  |
| 5   |                  |       |          |        |  |
| 6   | Llama2-7B        | 31.2% | 0.289    | 120    |  |
| 7   | RAG              | 49.3% | 0.467    | 450    |  |
| 8   | CRAG             | 58.7% | 0.556    | 980    |  |
| 9   | Self-RAG-7B      | 64.1% | 0.618    | 1200   |  |

|    |                     |       |           |        |  |
|----|---------------------|-------|-----------|--------|--|
| 10 | ChatGPT             | 52.4% | 0.501     | 800    |  |
| 11 | Self-RAG-13B        | 68.9% | 0.662     | 1450   |  |
| 12 |                     |       |           |        |  |
| 13 |                     |       |           |        |  |
| 14 | 测试集：PubHealth（事实验证） |       |           |        |  |
| 15 |                     |       |           |        |  |
| 16 |                     |       |           |        |  |
| 17 | 方法                  | 准确率   | Precision | Recall |  |
| 18 |                     |       |           |        |  |
| 19 | RAG                 | 78.3% | 0.761     | 0.792  |  |
| 20 | CRAG                | 85.6% | 0.841     | 0.869  |  |
| 21 | Self-RAG-7B         | 88.2% | 0.876     | 0.891  |  |
| 22 | Self-RAG-13B        | 91.4% | 0.908     | 0.919  |  |
| 23 |                     |       |           |        |  |

Self-RAG的推理时控制

Self-RAG最强大的特性之一是**推理时可控性**——你可以通过调整参数来控制模型行为，而无需重新训练！

1. 调整检索频率

代码块

```
1 #配置：更激进的检索
2 config_aggressive = {
3 'retrieve_threshold': 0.3, # 更低的阈值 = 更频繁检索
4 'retrieve_frequency': 'every_sentence' # 每句话都评估是否需要检索
5 }
6
7 #配置：保守的检索
8 config_conservative = {
9 'retrieve_threshold': 0.7, # 更高的阈值 = 较少检索
10 'retrieve_frequency': 'on_demand' # 仅在必要时检索
11 }
12
13 #配置：平衡模式
14 config_balanced = {
15 'retrieve_threshold': 0.5,
16 'retrieve_frequency': 'adaptive' # 根据查询复杂度动态调整
17 }
```

2. 段级束搜索（Segment-wise Beam Search）

Self-RAG使用特殊的束搜索策略，在生成每个段（sentence）时考虑反思令牌的影响。

代码块

```
1 def segment_wise_beam_search(
2 query,
3 model,
4 retriever,
5 beam_width=5,
6 preference_weights={
7 'relevance': 1.0,
8 'support': 1.5,
9 'utility': 2.0
```



```

10 }
11):
12 """
13 段级束搜索，优化多个维度
14 """
15 candidates = [{'text': '', 'score': 0.0, 'path': []}]
16 while not all_candidates_finished(candidates):
17 new_candidates = []
18 for candidate in candidates:
19 # 生成下一个段
20 next_segments = model.generate_next_segments(
21 query,
22 candidate['text'],
23 num_candidates=beam_width
24)
25 for segment, tokens in next_segments:
26 # 计算综合分数
27 score = 0.0
28 # 检索令牌权重
29 if '[Retrieve=Yes]' in tokens:
30 docs = retriever.get_relevant_documents(query)
31 relevance_score = evaluate_relevance(docs)
32 score += preference_weights['relevance'] *
relevance_score
33 # 支持度权重
34 if '[ISSUP]' in tokens:
35 support_level = extract_support_level(tokens)
36 score += preference_weights['support'] *
support_level
37 # 有用性权重
38 if '[ISUSE]' in tokens:
39 utility_score = extract_utility_score(tokens)
40 score += preference_weights['utility'] *
utility_score
41 new_candidates.append({
42 'text': candidate['text'] + segment,
43 'score': candidate['score'] + score,
44 'path': candidate['path'] + [tokens]
45 })
46 # 保留top-k候选
47 candidates = sorted(
48 new_candidates,
49 key=lambda x: x['score'],
50 reverse=True
51)[:beam_width]
52 return candidates[0] # 返回得分最高的候选

```

## 实际效果示例：

```

代码块

1 查询: "介绍一下量子计算的最新进展"
2
3 偏好1: 注重事实支持
4 preference_weights = {
5 'relevance': 1.0,
6 'support': 3.0, # 大幅提升支持度权重
7 'utility': 1.0
8 }
9
10 输出:

```

```
11 "量子计算在2024年取得了重大突破。[Retrieve=Yes]
12 根据Nature期刊2024年3月的报道，IBM宣布其127量子比特的Eagle处理器...
13 [ISSUP=FullySupported]
14 Google也在量子纠错方面取得进展... [ISSUP=FullySupported]"
15 （每个陈述都有充分的事实支持）
16
17 偏好2：注重实用性
18 preference_weights = {
19 'relevance': 1.0,
20 'support': 1.0,
21 'utility': 3.0 # 大幅提升实用性权重
22 }
23
24 输出：
25 "量子计算正在改变多个行业。[Retrieve=No]
26 在药物发现领域，量子计算可以模拟分子相互作用...
27 在金融领域，可以优化投资组合...
28 在密码学领域，既是机遇也是挑战..."
29 （更注重实际应用和对用户的价值）
```

## Self-RAG的LangGraph实现

代码块

```
1 from langgraph.graph import StateGraph, END
2 from langchain_openai import ChatOpenAI
3 from typing import TypedDict, Annotated, Sequence
4 import operator
5
6 #定义状态
7 class GraphState(TypedDict):
8 query: str
9 documents: Sequence[str]
10 generation: str
11 need_retrieve: bool
12 doc_relevance: Sequence[bool]
13 factual_support: str
14 utility_score: int
15 loop_count: int
16
17 #节点1：决定是否检索
18 def decide_to_retrieve(state):
19 """判断查询是否需要检索"""
20 query = state["query"]
21 # 使用LLM判断
22 prompt = f"""
23 判断以下查询是否需要检索外部知识：
24 查询：{query}
25 如果查询涉及：
26 - 最新信息或事实
27 - 超出常识的专业知识
28 - 需要引用来源
29 则返回 "yes"
30 如果查询涉及：
31 - 创意写作
32 - 常识问题
33 - 简单计算
34 则返回 "no"
35 只返回 yes 或 no。
36 """
```

```

37 llm = ChatOpenAI(model="gpt-4", temperature=0)
38 result = llm.invoke(prompt).content.lower()
39 return {"need_retrieve": "yes" in result, "loop_count": 0}
40
41 #节点2: 检索文档
42 def retrieve(state):
43 """执行检索"""
44 query = state["query"]
45 documents = vectorstore.similarity_search(query, k=5)
46 return {"documents": documents}
47
48 #节点3: 评估文档相关性
49 def grade_documents(state):
50 """评估每个文档的相关性"""
51 query = state["query"]
52 documents = state["documents"]
53 relevance = []
54 filtered_docs = []
55 for doc in documents:
56 prompt = f"""
57 评估文档是否与查询相关:
58 查询: {query}
59 文档: {doc.page_content[:500]}
60 如果文档包含回答查询所需的信息, 返回 "relevant"
61 否则返回 "irrelevant"
62 """
63 llm = ChatOpenAI(model="gpt-4", temperature=0)
64 result = llm.invoke(prompt).content.lower()
65 is_relevant = "relevant" in result
66 relevance.append(is_relevant)
67 if is_relevant:
68 filtered_docs.append(doc)
69 return {
70 "doc_relevance": relevance,
71 "documents": filtered_docs
72 }
73
74 #节点4: 生成答案
75 def generate(state):
76 """生成答案"""
77 query = state["query"]
78 documents = state["documents"]
79 need_retrieve = state.get("need_retrieve", True)
80 if need_retrieve and documents:
81 # 基于文档生成
82 context = "\n\n".join([doc.page_content for doc in
documents])
83 prompt = f"""
84 基于以下上下文回答问题:
85 上下文:
86 {context}
87 问题: {query}
88 答案:
89 """
90 else:
91 # 直接生成
92 prompt = f"""
93 回答以下问题:
94 问题: {query}
95 答案:
96 """
97 llm = ChatOpenAI(model="gpt-4", temperature=0.7)

```

```

98 generation = llm.invoke(prompt).content
99 return {"generation": generation}
100
101 #节点5: 评估事实支持度
102 def grade_generation_factual_support(state):
103 """评估生成内容是否有事实支持"""
104 generation = state["generation"]
105 documents = state["documents"]
106 if not documents:
107 return {"factual_support": "no_docs"}
108 context = "\n\n".join([doc.page_content for doc in documents])
109 prompt = f"""
110 评估生成的答案是否有文档支持：
111 文档：
112 {context}
113 生成的答案：
114 {generation}
115 评估答案中的每个事实陈述：
116 - 如果所有陈述都有文档支持，返回 "fully_supported"
117 - 如果部分陈述有支持，返回 "partially_supported"
118 - 如果没有支持或有明显错误，返回 "not_supported"
119 """
120 llm = ChatOpenAI(model="gpt-4", temperature=0)
121 result = llm.invoke(prompt).content.lower()
122 if "fully_supported" in result:
123 support = "fully_supported"
124 elif "partially_supported" in result:
125 support = "partially_supported"
126 else:
127 support = "not_supported"
128 return {"factual_support": support}
129
130 #节点6: 评估答案有用性
131 def grade_generation_utility(state):
132 """评估答案的有用性"""
133 query = state["query"]
134 generation = state["generation"]
135 prompt = f"""
136 评估答案对查询的有用性（1-5分）：
137 查询：{query}
138 答案：{generation}
139 评分标准：
140 5 - 完美回答，清晰、准确、全面
141 4 - 很好回答，略有不足
142 3 - 可以接受，但有明显缺陷
143 2 - 较差，大部分无用
144 1 - 完全无用
145 只返回数字1-5。
146 """
147 llm = ChatOpenAI(model="gpt-4", temperature=0)
148 result = llm.invoke(prompt).content.strip()
149 try:
150 score = int(result[0])
151 except:
152 score = 3 # 默认分数
153 return {"utility_score": score}
154
155 #决策函数
156 def decide_to_generate(state):
157 """决定是生成还是网络搜索"""
158 need_retrieve = state["need_retrieve"]
159 if need_retrieve:

```

```
160 return "retrieve"
161 else:
162 return "generate"
163
164 def check_doc_relevance(state):
165 """检查文档相关性"""
166 doc_relevance = state.get("doc_relevance", [])
167 if any(doc_relevance):
168 return "generate"
169 else:
170 return "web_search"
171
172 def check_factual_support(state):
173 """检查事实支持度"""
174 factual_support = state["factual_support"]
175 if factual_support == "fully_supported":
176 return "grade_utility"
177 else:
178 return "regenerate"
179
180 def check_utility(state):
181 """检查有用性"""
182 utility_score = state["utility_score"]
183 loop_count = state["loop_count"]
184 if utility_score >= 4:
185 return "end"
186 elif loop_count < 2:
187 return "retry"
188 else:
189 return "end" # 最多重试2次
190
191 #构建图
192 workflow = StateGraph(GraphState)
193
194 #添加节点
195 workflow.add_node("decide_retrieve", decide_to_retrieve)
196 workflow.add_node("retrieve", retrieve)
197 workflow.add_node("grade_documents", grade_documents)
198 workflow.add_node("generate", generate)
199 workflow.add_node("grade_support",
200 grade_generation_factual_support)
201 workflow.add_node("grade_utility", grade_generation_utility)
202
203 #设置入口
204 workflow.set_entry_point("decide_retrieve")
205
206 #添加边
207 workflow.add_conditional_edges(
208 "decide_retrieve",
209 decide_to_generate,
210 {
211 "retrieve": "retrieve",
212 "generate": "generate"
213 }
214)
215 workflow.add_edge("retrieve", "grade_documents")
216
217 workflow.add_conditional_edges(
218 "grade_documents",
219 check_doc_relevance,
220 {
```

```
221 "generate": "generate",
222 "web_search": "retrieve" # 简化: 重新检索
223 }
224)
225
226 workflow.add_edge("generate", "grade_support")
227
228 workflow.add_conditional_edges(
229 "grade_support",
230 check_factual_support,
231 {
232 "grade_utility": "grade_utility",
233 "regenerate": "generate"
234 }
235)
236
237 workflow.add_conditional_edges(
238 "grade_utility",
239 check_utility,
240 {
241 "end": END,
242 "retry": "retrieve"
243 }
244)
245
246 #编译
247 app = workflow.compile()
248
249 #执行
250 result = app.invoke({
251 "query": "什么是Self-RAG? ",
252 "documents": [],
253 "generation": "",
254 "need_retrieve": True,
255 "doc_relevance": [],
256 "factual_support": "",
257 "utility_score": 0,
258 "loop_count": 0
259 })
260
261 print(result["generation"])
```

Self-RAG的优缺点

✔ 优点：

1. 最高准确率

代码块

- 1 在多个benchmark上超越ChatGPT和其他RAG方法
- 2 事实准确率提升20-30%

2. 按需检索

代码块

- 1 不是所有查询都检索 → 节省成本和时间
- 2 创意任务：0次检索
- 3 事实查询：1-3次检索

3. 推理时可控

代码块

- 1 无需重新训练即可调整行为
- 2 根据不同应用场景调整偏好权重

4. 多维度评估

代码块

- 1 同时考虑：相关性、支持度、有用性
- 2 全面保证答案质量

5. 自我纠错能力

代码块

- 1 检测到答案不佳时自动重试
- 2 迭代优化直到满意

❌ 缺点：

1. 训练成本极高

代码块

- 1 需要两阶段训练：
- 2 Critic Model训练
- 3 Generator Model训练
- 4
- 5 数据需求：10-15万条高质量样本
- 6 GPU需求：A100 × 8，训练时间2-3周
- 7 总成本：\$50,000 - \$100,000

2. 推理速度最慢

代码块

- 1 每个段都需要多次评估
- 2 复杂查询可能检索多次
- 3
- 4 平均延迟：
- 5 传统RAG：500ms
- 6 CRAG：1000ms
- 7 Self-RAG：1500ms

3. 系统复杂度最高

代码块



- 1 需要维护：
- 2 Generator模型
- 3 Critic模型
- 4 检索系统
- 5 束搜索逻辑
- 6
- 7 调试和优化难度大

4. 模型大小限制

- 代码块
- 1 目前只有7B和13B的开源模型
  - 2 更大模型（70B+）训练成本过高

5. token开销大

- 代码块
- 1 反思令牌占用额外空间
  - 2 例如: "[Retrieve=Yes] [ISREL=Relevant] [ISSUP=FullySupported]"
  - 3 增加10-15%的token使用

Self-RAG适用场景

强烈推荐：

- 🎓 教育和研究助手
- 📖 学术论文问答
- 📁 企业知识库（预算充足）
- 🏥 医疗文献检索
- ⚖️ 法律案例分析

不适合：

- 🚀 实时对话系统（延迟要求<500ms）
- 📱 移动端应用（模型太大）
- 💰 预算有限的项目（训练/推理成本高）
- 🎨 创意类应用（过度检索可能限制创造力）

这就引出了下一个核心话题... 看另一个文档

 李新运 5月11日 16:31  
另一个文档链接呢

 用户980025 5月26日 22:09  
我估计是下面的 3.评估 RAG 部分



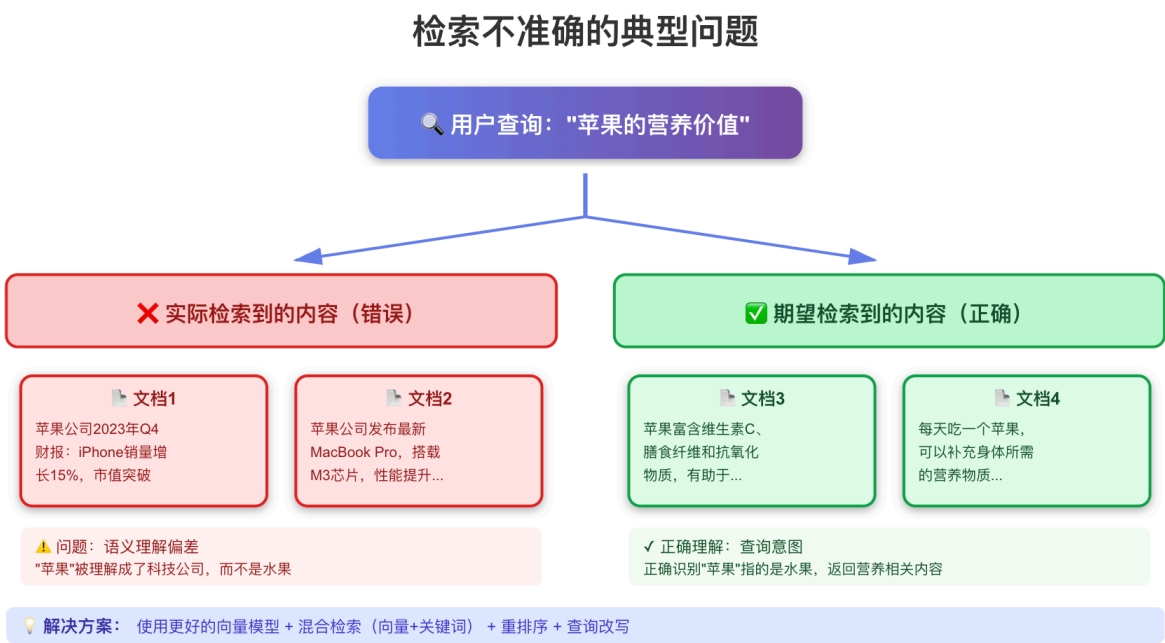


## 2. RAG系统最头疼的3大难题

### 🔥 难题1：检索不准确（最核心的痛点）

问题描述：

- 你问"苹果的营养价值"，系统却返回"苹果公司的股价"
- 检索结果偏差，导致生成的答案驴唇不对马嘴



为什么会这样？

#### 1. 语义理解不足

代码块

```
1 # 问题：同一个词，不同含义
```

```
2 query1 = "苹果好吃吗？" # 指水果
3 query2 = "苹果股票怎么样？" # 指公司
4
5 # 如果向量模型不够好，可能会混淆
```

2. 分块策略不当

代码块

```
1 # ❌ 错误的切分
2 chunk1 = "我们的产品销售额在2023年"
3 chunk2 = "达到了5000万元，增长..."
4
5 # 用户查询："2023年销售额"
6 # 结果：chunk1被检索到，但信息不完整！
7
8 # ✅ 正确的切分（保持上下文完整）
9 chunk = "我们的产品销售额在2023年达到了5000万元，增长25%"
```

3. 向量检索的局限性

代码块

```
1 # 纯向量检索的问题
2 query = "2023年10月15日的会议记录"
3
4 # 向量相似度可能检索到：
5 # - "2023年10月16日的会议记录" ✓ （相似但不对）
6 # - "2023年9月的会议总结" ✓ （相似但不对）
7 # - "2023年10月15日的会议记录" ✓✓✓ （精确匹配）
```

解决方案：混合检索

代码块

```
1 from langchain.retrievers import EnsembleRetriever
2 from langchain.retrievers import BM25Retriever
3
4 # 方案1：向量检索（语义相似）
5 vector_retriever = vectorstore.as_retriever(search_kwargs={"k": 5})
6
7 # 方案2：BM25检索（关键词匹配）
8 bm25_retriever = BM25Retriever.from_texts(chunks)
9 bm25_retriever.k = 5
10
11 # 混合检索：结合两种方法的优点
12 ensemble_retriever = EnsembleRetriever(
13 retrievers=[vector_retriever, bm25_retriever],
14 weights=[0.6, 0.4] # 向量60%权重，BM25 40%权重
15)
16
17 # 检索效果对比
18 query = "2023年10月15日的销售数据"
19 results = ensemble_retriever.get_relevant_documents(query)
```

进阶方案：重排序（Reranking）

代码块

```
1 from sentence_transformers import CrossEncoder
2
3 # 第一步：粗筛（检索Top 20）
4 initial_docs = vectorstore.similarity_search(query, k=20)
5
6 # 第二步：精排（用更强的模型重新打分）
7 reranker = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')
8
9 # 计算每个文档与query的相关性分数
10 scores = reranker.predict([(query, doc.page_content) for doc in
 initial_docs])
11
12 # 按分数排序，取Top 3
13 sorted_docs = [doc for _, doc in sorted(zip(scores, initial_docs),
 reverse=True)]
14 final_docs = sorted_docs[:3]
```

## 🔥 难题2：上下文窗口限制

### 问题描述：

- 检索到10篇相关文档，但AI一次只能看3篇
- 有用的信息可能在第7篇，AI看不到

[https://claude.zhangsan.shop/chat/context\\_window\\_limit.svg](https://claude.zhangsan.shop/chat/context_window_limit.svg)

### 解决方案：智能压缩

代码块

```
1 from langchain.retrievers.document_compressors import
 LLMChainExtractor
2 from langchain.retrievers import ContextualCompressionRetriever
3
4 # 创建压缩器
5 compressor = LLMChainExtractor.from_llm(llm)
6
7 # 压缩检索器
8 compression_retriever = ContextualCompressionRetriever(
9 base_compressor=compressor,
10 base_retriever=vector_retriever
11)
12
13 # 使用示例
14 query = "2023年销售额"
15 compressed_docs =
 compression_retriever.get_relevant_documents(query)
16
17 # 对比：
18 # 原始文档：5000字 → 压缩后：500字（只保留与query相关的部分）
```

## 🔥 难题3：如何评估RAG效果（没有标准答案）

### 问题描述：

- 生成的答案看起来不错，但真的对吗？
- 不同的RAG配置，哪个更好？

这就引出了下一个核心话题... 看另一个文档

🔥 难题4：文档，表格怎么解析，这个我们到时候重新更新一篇文章解析

### 3. 评估RAG：如何知道你的系统靠不靠谱？

📊 评估体系全景图

[https://claude.zhongsan.shop/chat/rag\\_evaluation\\_framework.svg](https://claude.zhongsan.shop/chat/rag_evaluation_framework.svg)

#### 🎯 评估维度1：检索质量

指标1：召回率（Recall）

通俗解释：该找到的文档，你找到了多少？

代码块

```
1 # 示例场景
2 真实相关文档 = [doc1, doc2, doc3, doc4, doc5] # 5个相关文档
3 系统检索结果 = [doc1, doc2, doc6] # 检索到3个，其中2个正确
4
5 召回率 = 检索到的正确文档数 / 总的相关文档数
6 = 2 / 5
7 = 0.4 (40%)
```

代码实现：

代码块

```
1 def calculate_recall(retrieved_docs, relevant_docs):
2 """
3 计算召回率
4
5 Args:
6 retrieved_docs: 系统检索到的文档ID列表
7 relevant_docs: 真实相关的文档ID列表
8 """
9 retrieved_set = set(retrieved_docs)
10 relevant_set = set(relevant_docs)
11
12 # 检索到的正确文档
13 correct = retrieved_set.intersection(relevant_set)
14
15 recall = len(correct) / len(relevant_set) if relevant_set else
0
16
17 return recall
18
19 # 测试
20 retrieved = ['doc1', 'doc2', 'doc6']
21 relevant = ['doc1', 'doc2', 'doc3', 'doc4', 'doc5']
22
23 print(f"召回率: {calculate_recall(retrieved, relevant):.2%}") # 40%
```

## 指标2：准确率（Precision）

通俗解释：检索到的文档中，有多少是真正相关的？

代码块

```
1 准确率 = 检索到的正确文档数 / 检索到的总文档数
2 = 2 / 3
3 = 0.67 (67%)
```

## 指标3：MRR（Mean Reciprocal Rank）

通俗解释：第一个正确答案排在第几位？

代码块

```
1 # 场景1：第一个就是正确答案
2 检索结果 = [doc1✓, doc2, doc3]
3 MRR = 1/1 = 1.0
4
5 # 场景2：第三个才是正确答案
6 检索结果 = [doc5, doc7, doc1✓]
7 MRR = 1/3 = 0.33
8
9 # 场景3：没有正确答案
10 检索结果 = [doc5, doc7, doc9]
11 MRR = 0
```

代码实现：

代码块

```
1 def calculate_mrr(retrieved_docs, relevant_docs):
2 """计算MRR"""
3 for i, doc in enumerate(retrieved_docs, 1):
4 if doc in relevant_docs:
5 return 1.0 / i
6 return 0.0
7
8 # 测试
9 retrieved = ['doc5', 'doc7', 'doc1']
10 relevant = ['doc1', 'doc2']
11
12 print(f"MRR: {calculate_mrr(retrieved, relevant):.3f}") # 0.333
```

## 评估维度2：生成质量

### 指标1：事实准确性

人工评估标准：

代码块

```
1 评分标准 = {
2 5: "完全正确，信息准确",
3 4: "基本正确，有小瑕疵",
4 3: "部分正确",
5 2: "大部分错误",
6 1: "完全错误"
7 }
```

自动评估（使用LLM评分）：

```
代码块

1 from langchain.evaluation import load_evaluator
2
3 # 创建评估器
4 evaluator = load_evaluator("labeled_criteria",
5 criteria="correctness")
6
7 # 评估示例
8 eval_result = evaluator.evaluate_strings(
9 prediction="2023年Q4销售额为5000万元", # AI生成的答案
10 reference="根据财报，2023年Q4销售额为5000万元", # 标准答案
11 input="2023年Q4销售额是多少？" # 原始问题
12)
13 print(f"准确性得分: {eval_result['score']}")
14 print(f"评估理由: {eval_result['reasoning']}")
```

指标2：忠实度（Faithfulness）

通俗解释：生成的答案是否忠于检索到的文档？有没有瞎编？

```
代码块

1 # ❌ 低忠实度示例
2 检索文档: "2023年Q4销售额为5000万元"
3 AI回答: "2023年Q4销售额为5000万元，预计2024年将达到8000万元"
4 问题: 后半句是AI瞎编的！
5
6 # ✅ 高忠实度示例
7 检索文档: "2023年Q4销售额为5000万元"
8 AI回答: "根据财报，2023年Q4销售额为5000万元"
```

代码评估：

```
代码块

1 def evaluate_faithfulness(generated_answer, retrieved_docs, llm):
2 """
3 评估答案忠实度
4 """
5 prompt = f"""
6 检索到的文档:
7 {retrieved_docs}
8
9 生成的答案:
10 {generated_answer}
11
12 请评估：生成的答案中的信息是否都能在检索文档中找到依据？
13 评分：1-5分（5分=完全忠实，1分=大量虚构）
```

```
14
15 输出格式：
16 得分： X/5
17 理由： ...
18 """
19
20 result = llm.predict(prompt)
21 return result
22
23 # 使用示例
24 docs = "2023年Q4销售额为5000万元，同比增长25%"
25 answer = "2023年Q4销售额为5000万元，预计2024年将达到8000万元"
26
27 faithfulness_score = evaluate_faithfulness(answer, docs, llm)
28 print(faithfulness_score)
```

指标3：相关性（Relevance）

通俗解释：回答是否切题？

代码块

```
1 # 用户问题
2 query = "如何提高团队效率？"
3
4 # ❌ 低相关性
5 answer = "团队建设很重要，沟通是关键..." # 太泛泛
6
7 # ✅ 高相关性
8 answer = "提高团队效率的3个方法：1)使用敏捷开发 2)每日站会 3)OKR目标管理"
```

🎯 评估维度3：端到端评估

使用RAGAS框架

代码块

```
1 from ragas import evaluate
2 from ragas.metrics import (
3 faithfulness,
4 answer_relevancy,
5 context_relevancy,
6 context_recall
7)
8
9 # 准备评估数据
10 eval_data = {
11 "question": ["2023年Q4销售额是多少？", "产品有哪些核心功能？"],
12 "answer": ["5000万元", "包括AI助手、数据分析、自动化流程"],
13 "contexts": [
14 ["2023年Q4销售额为5000万元，同比增长25%"],
15 ["核心功能包括：AI智能助手、实时数据分析、工作流自动化"]
16],
17 "ground_truths": ["5000万元", "AI助手、数据分析、自动化"]
18 }
19
20 # 执行评估
21 result = evaluate(
22 eval_data,
23 metrics=[
```



```
24 faithfulness, # 忠实度
25 answer_relevancy, # 答案相关性
26 context_relevancy, # 上下文相关性
27 context_recall # 上下文召回
28]
29)
30
31 # 查看结果
32 print(result)
33 # 输出示例:
34 # {
35 # 'faithfulness': 0.95,
36 # 'answer_relevancy': 0.92,
37 # 'context_relevancy': 0.88,
38 # 'context_recall': 0.90
39 # }
```

## 完整评估流程

代码块

```
1 class RAGEvaluator:
2 """RAG系统评估器"""
3
4 def __init__(self, rag_system, test_dataset):
5 self.rag_system = rag_system
6 self.test_dataset = test_dataset # 测试数据集
7
8 def evaluate_retrieval(self):
9 """评估检索效果"""
10 recalls = []
11 precisions = []
12 mrrs = []
13
14 for test_case in self.test_dataset:
15 query = test_case['query']
16 relevant_docs = test_case['relevant_docs']
17
18 # 执行检索
19 retrieved_docs = self.rag_system.retrieve(query, k=5)
20
21 # 计算指标
22 recall = self._calculate_recall(retrieved_docs,
23 relevant_docs)
24 precision = self._calculate_precision(retrieved_docs,
25 relevant_docs)
26 mrr = self._calculate_mrr(retrieved_docs,
27 relevant_docs)
28
29 recalls.append(recall)
30 precisions.append(precision)
31 mrrs.append(mrr)
32
33 return {
34 'avg_recall': sum(recalls) / len(recalls),
35 'avg_precision': sum(precisions) / len(precisions),
36 'avg_mrr': sum(mrrs) / len(mrrs)
37 }
38
39 def evaluate_generation(self):
40 """评估生成效果"""
```



```

38 faithfulness_scores = []
39 relevancy_scores = []
40
41 for test_case in self.test_dataset:
42 query = test_case['query']
43 ground_truth = test_case['answer']
44
45 # 执行RAG
46 answer, contexts = self.rag_system.query(query)
47
48 # 评估
49 faith_score = self._evaluate_faithfulness(answer,
50 contexts)
51 rel_score = self._evaluate_relevancy(answer, query)
52
53 faithfulness_scores.append(faith_score)
54 relevancy_scores.append(rel_score)
55
56 return {
57 'avg_faithfulness': sum(faithfulness_scores) /
58 len(faithfulness_scores),
59 'avg_relevancy': sum(relevancy_scores) /
60 len(relevancy_scores)
61 }
62
63 def run_full_evaluation(self):
64 """运行完整评估"""
65 retrieval_metrics = self.evaluate_retrieval()
66 generation_metrics = self.evaluate_generation()
67
68 print("=" * 50)
69 print("RAG系统评估报告")
70 print("=" * 50)
71 print("\n📊 检索效果:")
72 print(f" 召回率: {retrieval_metrics['avg_recall']:.2%}")
73 print(f" 准确率: {retrieval_metrics['avg_precision']:.2%}")
74 print(f" MRR: {retrieval_metrics['avg_mrr']:.3f}")
75 print("\n✍️ 生成效果:")
76 print(f" 忠实度: {generation_metrics['avg_faithfulness']:.2%}")
77 print(f" 相关性: {generation_metrics['avg_relevancy']:.2%}")
78 print("=" * 50)
79
80 return {**retrieval_metrics, **generation_metrics}
81
82 # 使用示例
83 test_data = [
84 {
85 'query': '2023年Q4销售额是多少? ',
86 'relevant_docs': ['doc1', 'doc3'],
87 'answer': '5000万元'
88 },
89 # ... 更多测试用例
90]
91
92 evaluator = RAGEvaluator(rag_system, test_data)
93 results = evaluator.run_full_evaluation()

```

## 4. 实战代码：手把手搭建你的第一个RAG

### 🔧 完整代码实现

代码块

```
1 """
2 完整的RAG系统实现
3 支持：文档加载、向量化、检索、生成
4 """
5
6 import os
7 from typing import List, Dict
8 from langchain.document_loaders import TextLoader, PyPDFLoader
9 from langchain.text_splitter import RecursiveCharacterTextSplitter
10 from langchain.embeddings import HuggingFaceEmbeddings
11 from langchain.vectorstores import Chroma
12 from langchain.chat_models import ChatOpenAI
13 from langchain.chains import RetrievalQA
14 from langchain.prompts import PromptTemplate
15
16 class SimpleRAG:
17 """简单的RAG系统"""
18
19 def __init__(self,
20 embedding_model: str = "paraphrase-multilingual-
21 MiniLM-L12-v2",
22 llm_model: str = "gpt-3.5-turbo",
23 chunk_size: int = 500,
24 chunk_overlap: int = 50):
25 """
26 初始化RAG系统
27
28 Args:
29 embedding_model: 向量化模型名称
30 llm_model: 大语言模型名称
31 chunk_size: 文本块大小
32 chunk_overlap: 文本块重叠大小
33 """
34 print("🚀 初始化RAG系统...")
35
36 # 1. 初始化文本分割器
37 self.text_splitter = RecursiveCharacterTextSplitter(
38 chunk_size=chunk_size,
39 chunk_overlap=chunk_overlap,
40 separators=["\n\n", "\n", ". ", "! ", "? ", " ", ""])
41
42 # 2. 初始化向量化模型
43 print(f"📦 加载向量化模型: {embedding_model}")
44 self.embeddings = HuggingFaceEmbeddings(
45 model_name=embedding_model,
46 model_kwargs={'device': 'cpu'})
47
48 # 3. 初始化LLM
49 print(f"🤖 加载大语言模型: {llm_model}")
50 self.llm = ChatOpenAI(
51 model_name=llm_model,
52 temperature=0.1 # 降低随机性, 提高准确性
53)
```

```
55
56 # 4. 向量数据库
57 self.vectorstore = None
58 self.qa_chain = None
59
60 print("✅ RAG系统初始化完成! \n")
61
62 def load_documents(self, file_paths: List[str]) -> List[str]:
63 """
64 加载文档
65
66 Args:
67 file_paths: 文件路径列表
68
69 Returns:
70 文档内容列表
71 """
72 print("📁 开始加载文档...")
73 all_docs = []
74
75 for file_path in file_paths:
76 print(f" - 加载: {file_path}")
77
78 # 根据文件类型选择加载器
79 if file_path.endswith('.pdf'):
80 loader = PyPDFLoader(file_path)
81 else:
82 loader = TextLoader(file_path, encoding='utf-8')
83
84 docs = loader.load()
85 all_docs.extend(docs)
86
87 print(f"✅ 成功加载 {len(all_docs)} 个文档\n")
88 return all_docs
89
90 def build_knowledge_base(self, documents: List[str]):
91 """
92 构建知识库
93
94 Args:
95 documents: 文档列表
96 """
97 print("🔪 开始构建知识库...")
98
99 # 1. 文档切分
100 print(" 1️⃣ 切分文档...")
101 chunks = self.text_splitter.split_documents(documents)
102 print(f" 切分为 {len(chunks)} 个文本块")
103
104 # 2. 向量化并存储
105 print(" 2️⃣ 向量化并存储到数据库...")
106 self.vectorstore = Chroma.from_documents(
107 documents=chunks,
108 embedding=self.embeddings,
109 persist_directory="./chroma_db"
110)
111 print(f" ✅ 已存储到向量数据库")
112
113 # 3. 创建检索QA链
114 print(" 3️⃣ 创建问答链...")
115 self._create_qa_chain()
116
```

```
117 print("✅ 知识库构建完成! \n")
118
119 def _create_qa_chain(self):
120 """创建问答链"""
121 # 自定义提示模板
122 template = """你是一个专业的AI助手。请基于以下检索到的上下文信息来
回答问题。
123
124 【重要规则】
125 1. 只使用提供的上下文信息回答
126 2. 如果上下文中没有相关信息，明确说"根据提供的信息无法回答"
127 3. 回答要准确、简洁
128 4. 如果可能，引用具体的数据或事实
129
130 上下文信息：
131 {context}
132
133 问题：{question}
134
135 答案："""
136
137 prompt = PromptTemplate(
138 template=template,
139 input_variables=["context", "question"]
140)
141
142 self.qa_chain = RetrievalQA.from_chain_type(
143 llm=self.llm,
144 chain_type="stuff", # 将所有文档一次性传给LLM
145 retriever=self.vectorstore.as_retriever(
146 search_kwargs={"k": 3} # 检索Top 3相关文档
147),
148 chain_type_kwargs={"prompt": prompt},
149 return_source_documents=True # 返回源文档
150)
151
152 def query(self, question: str) -> Dict:
153 """
154 查询问题
155
156 Args:
157 question: 用户问题
158
159 Returns:
160 包含答案和来源文档的字典
161 """
162 if not self.qa_chain:
163 raise ValueError("请先构建知识库! ")
164
165 print(f"🔍 查询: {question}")
166 print("-" * 50)
167
168 # 执行查询
169 result = self.qa_chain({"query": question})
170
171 # 提取答案和来源
172 answer = result['result']
173 source_docs = result['source_documents']
174
175 print(f"\n💡 答案:\n{answer}\n")
176 print(f"📄 来源文档 ({len(source_docs)}个):")
177 for i, doc in enumerate(source_docs, 1):
```

```
178 print(f" [{i}] {doc.page_content[:100]}...")
179 print("-" * 50 + "\n")
180
181 return {
182 'answer': answer,
183 'sources': [doc.page_content for doc in source_docs]
184 }
185
186 def search_similar(self, query: str, k: int = 5) -> List[Dict]:
187 """
188 搜索相似文档
189
190 Args:
191 query: 查询文本
192 k: 返回数量
193
194 Returns:
195 相似文档列表
196 """
197 if not self.vectorstore:
198 raise ValueError("请先构建知识库！")
199
200 results =
self.vectorstore.similarity_search_with_score(query, k=k)
201
202 print(f"🔍 搜索: {query}")
203 print(f"📄 找到 {len(results)} 个相关文档:\n")
204
205 similar_docs = []
206 for i, (doc, score) in enumerate(results, 1):
207 print(f" [{i}] 相似度: {score:.3f}")
208 print(f" 内容: {doc.page_content[:100]}...\n")
209 similar_docs.append({
210 'content': doc.page_content,
211 'score': float(score)
212 })
213
214 return similar_docs
215
216
217 # ===== 使用示例 =====
218
219 def main():
220 """主函数：演示RAG系统的完整使用流程"""
221
222 # 1. 创建RAG系统
223 rag = SimpleRAG(
224 embedding_model="paraphrase-multilingual-MiniLM-L12-v2",
225 llm_model="gpt-3.5-turbo",
226 chunk_size=500,
227 chunk_overlap=50
228)
229
230 # 2. 准备测试文档
231 print(f"📝 创建测试文档...")
232
233 # 创建示例文档
234 with open("company_report.txt", "w", encoding="utf-8") as f:
235 f.write("""
236 2023年度公司财务报告
237
238 一、业绩概况
```

```
239 2023年全年，公司实现营业收入2.5亿元，同比增长35%。
240 其中，第四季度单季度营收达到5000万元，创历史新高。
241
242 二、产品线表现
243 - AI助手产品：年销售额1.2亿元，占比48%
244 - 数据分析平台：年销售额8000万元，占比32%
245 - 企业服务：年销售额5000万元，占比20%
246
247 三、市场展望
248 预计2024年公司营收将突破3亿元，继续保持高速增长态势。
249
250
251 with open("product_manual.txt", "w", encoding="utf-8") as f:
252 f.write("""
253 产品使用手册
254
255 产品名称：智能AI助手
256 版本：v2.0
257
258 核心功能：
259 1. 自然语言对话：支持多轮对话，理解上下文
260 2. 知识问答：基于企业知识库回答问题
261 3. 任务自动化：自动执行重复性任务
262 4. 数据分析：实时分析业务数据
263
264 使用场景：
265 - 客户服务：24小时在线客服
266 - 内部协作：团队知识管理
267 - 业务分析：销售数据洞察
268 """)
269
270 # 3. 加载文档
271 documents = rag.load_documents([
272 "company_report.txt",
273 "product_manual.txt"
274])
275
276 # 4. 构建知识库
277 rag.build_knowledge_base(documents)
278
279 # 5. 测试查询
280 print("=" * 70)
281 print("开始测试查询")
282 print("=" * 70 + "\n")
283
284 # 测试1: 财务问题
285 rag.query("2023年第四季度的营收是多少？")
286
287 # 测试2: 产品问题
288 rag.query("AI助手有哪些核心功能？")
289
290 # 测试3: 搜索相似文档
291 rag.search_similar("产品功能介绍", k=3)
292
293 print("✅ 测试完成！")
294
295
296 if __name__ == "__main__":
297 main()
```

代码块

```
1 🚀 初始化RAG系统...
2 📦 加载向量化模型: paraphrase-multilingual-MiniLM-L12-v2
3 🤖 加载大语言模型: gpt-3.5-turbo
4 ✅ RAG系统初始化完成!
5
6 📁 开始加载文档...
7 - 加载: company_report.txt
8 - 加载: product_manual.txt
9 ✅ 成功加载 2 个文档
10
11 🛠️ 开始构建知识库...
12 1️⃣ 切分文档...
13 切分为 8 个文本块
14 2️⃣ 向量化并存储到数据库...
15 ✅ 已存储到向量数据库
16 3️⃣ 创建问答链...
17 ✅ 知识库构建完成!
18
19 =====
20 开始测试查询
21 =====
22
23 🔍 查询: 2023年第四季度的营收是多少?
24 -----
25
26 💡 答案:
27 根据2023年度公司财务报告, 2023年第四季度的营收达到5000万元, 创历史新高。
28
29 📄 来源文档 (3个):
30 [1] 一、业绩概况
31 2023年全年, 公司实现营业收入2.5亿元, 同比增长35%。
32 其中, 第四季度单季度营收达到5000万元, 创历史新高...
33 [2] 三、市场展望
34 预计2024年公司营收将突破3亿元, 继续保持高速增长态势...
35 [3] 二、产品线表现
36 - AI助手产品: 年销售额1.2亿元, 占比48%...
37 -----
```

5. 进阶技巧：让你的RAG系统脱颖而出

🚀 技巧1：多路召回策略

代码块

```
1 class AdvancedRetriever:
2 """高级检索器: 结合多种检索方法"""
3
4 def __init__(self, vectorstore, bm25_index, reranker):
5 self.vectorstore = vectorstore
6 self.bm25_index = bm25_index
7 self.reranker = reranker
8
```



```

9 def retrieve(self, query: str, k: int = 5) -> List[str]:
10 """多路召回 + 重排序"""
11
12 # 路径1: 向量检索 (Top 20)
13 vector_docs = self.vectorstore.similarity_search(query,
14 k=20)
15
16 # 路径2: BM25关键词检索 (Top 20)
17 bm25_docs = self.bm25_index.search(query, k=20)
18
19 # 路径3: 混合检索 (去重)
20 all_docs = list(set(vector_docs + bm25_docs))
21
22 # 重排序: 用更强的模型打分
23 reranked_docs = self.reranker.rerank(query, all_docs)
24
25 # 返回Top K
26 return reranked_docs[:k]

```

## 技巧2：查询改写

代码块

```

1 def query_rewrite(original_query: str, llm) -> List[str]:
2 """
3 查询改写：生成多个变体查询
4 """
5 prompt = f"""
6 原始查询: {original_query}
7
8 请生成3个语义相同但表达不同的查询变体，用于提高检索召回率。
9
10 输出格式（每行一个）：
11 1. ...
12 2. ...
13 3. ...
14 """
15
16 response = llm.predict(prompt)
17 variants = [line.strip() for line in response.split('\n') if
18 line.strip()]
19
20 return [original_query] + variants
21
22 # 使用示例
23 query = "公司去年赚了多少钱？"
24 variants = query_rewrite(query, llm)
25
26 # 输出：
27 # ['公司去年赚了多少钱？',
28 # '2023年公司的营收是多少？',
29 # '去年公司财务表现如何？',
30 # '上一年度的销售额统计']
31
32 # 用所有变体检索，合并结果
33 all_results = []
34 for q in variants:
35 results = vectorstore.similarity_search(q, k=5)
36 all_results.extend(results)
37
38 # 去重并重排序

```



```
37 final_results = rerank(all_results)
```

### 技巧3：分层检索

代码块

```
1 class HierarchicalRAG:
2 """分层RAG：先粗筛，再精筛"""
3
4 def retrieve(self, query: str):
5 # 第1层：快速粗筛 (Top 100)
6 coarse_results = self.fast_retriever.search(query, k=100)
7
8 # 第2层：精细重排 (Top 20)
9 fine_results = self.reranker.rerank(query, coarse_results,
10 k=20)
11
12 # 第3层：LLM筛选 (Top 5)
13 final_results = self.llm_filter(query, fine_results, k=5)
14
15 return final_results
```

### 技巧4：实时更新知识库

代码块

```
1 class IncrementalRAG:
2 """支持增量更新的RAG"""
3
4 def add_document(self, new_doc: str):
5 """添加新文档"""
6 # 切分
7 chunks = self.text_splitter.split_text(new_doc)
8
9 # 向量化
10 vectors = self.embeddings.embed_documents(chunks)
11
12 # 增量添加到向量库 (不需要重建整个库)
13 self.vectorstore.add_texts(chunks, vectors)
14
15 print(f"✅ 已添加 {len(chunks)} 个新文档块")
16
17 def update_document(self, doc_id: str, new_content: str):
18 """更新文档"""
19 # 删除旧版本
20 self.vectorstore.delete(doc_id)
21
22 # 添加新版本
23 self.add_document(new_content)
```



6. 评估RAG：如何知道你的系统靠不靠谱？

评估体系全景图



# 🎯 评估维度1：检索质量

## 指标1：召回率（Recall）

通俗解释：该找到的文档，你找到了多少？

代码块

```
1 # 示例场景
2 真实相关文档 = [doc1, doc2, doc3, doc4, doc5] # 5个相关文档
3 系统检索结果 = [doc1, doc2, doc6] # 检索到3个，其中2个正确
4
5 召回率 = 检索到的正确文档数 / 总的相关文档数
6 = 2 / 5
7 = 0.4 (40%)
```

### 代码实现：

代码块

```
1 def calculate_recall(retrieved_docs, relevant_docs):
2 """
3 计算召回率
4
5 Args:
6 retrieved_docs: 系统检索到的文档ID列表
7 relevant_docs: 真实相关的文档ID列表
8 """
9 retrieved_set = set(retrieved_docs)
10 relevant_set = set(relevant_docs)
11
12 # 检索到的正确文档
13 correct = retrieved_set.intersection(relevant_set)
14
15 recall = len(correct) / len(relevant_set) if relevant_set else
16 0
17 return recall
18
19 # 测试
20 retrieved = ['doc1', 'doc2', 'doc6']
21 relevant = ['doc1', 'doc2', 'doc3', 'doc4', 'doc5']
22
23 print(f"召回率: {calculate_recall(retrieved, relevant):.2%}") # 40%
```

## 指标2：准确率（Precision）

通俗解释：检索到的文档中，有多少是真正相关的？

代码块

```
1 准确率 = 检索到的正确文档数 / 检索到的总文档数
2 = 2 / 3
3 = 0.67 (67%)
```

## 指标3：MRR（Mean Reciprocal Rank）

通俗解释：第一个正确答案排在第几位？

代码块

```
1 # 场景1: 第一个就是正确答案
2 检索结果 = [doc1✓, doc2, doc3]
3 MRR = 1/1 = 1.0
4
5 # 场景2: 第三个才是正确答案
6 检索结果 = [doc5, doc7, doc1✓]
7 MRR = 1/3 = 0.33
8
9 # 场景3: 没有正确答案
10 检索结果 = [doc5, doc7, doc9]
11 MRR = 0
```

代码实现：

代码块

```
1 def calculate_mrr(retrieved_docs, relevant_docs):
2 """计算MRR"""
3 for i, doc in enumerate(retrieved_docs, 1):
4 if doc in relevant_docs:
5 return 1.0 / i
6 return 0.0
7
8 # 测试
9 retrieved = ['doc5', 'doc7', 'doc1']
10 relevant = ['doc1', 'doc2']
11
12 print(f"MRR: {calculate_mrr(retrieved, relevant):.3f}") # 0.333
```

 评估维度2：生成质量

指标1：事实准确性

人工评估标准：

代码块

```
1 评分标准 = {
2 5: "完全正确，信息准确",
3 4: "基本正确，有小瑕疵",
4 3: "部分正确",
5 2: "大部分错误",
6 1: "完全错误"
7 }
```

自动评估（使用LLM评分）：

代码块

```
1 from langchain.evaluation import load_evaluator
2
3 # 创建评估器
4 evaluator = load_evaluator("labeled_criteria",
5 criteria="correctness")
6
7 # 评估示例
8 eval_result = evaluator.evaluate_strings(
9 prediction="2023年Q4销售额为5000万元", # AI生成的答案
10 reference="根据财报，2023年Q4销售额为5000万元", # 标准答案
11 input="2023年Q4销售额是多少？" # 原始问题
```

```
11)
12
13 print(f"准确性得分: {eval_result['score']}")
14 print(f"评估理由: {eval_result['reasoning']}")
```

指标2：忠实度（Faithfulness）

通俗解释：生成的答案是否忠于检索到的文档？有没有瞎编？

代码块

```
1 # ❌ 低忠实度示例
2 检索文档: "2023年Q4销售额为5000万元"
3 AI回答: "2023年Q4销售额为5000万元，预计2024年将达到8000万元"
4 问题: 后半句是AI瞎编的！
5
6 # ✅ 高忠实度示例
7 检索文档: "2023年Q4销售额为5000万元"
8 AI回答: "根据财报，2023年Q4销售额为5000万元"
```

代码评估：

代码块

```
1 def evaluate_faithfulness(generated_answer, retrieved_docs, llm):
2 """
3 评估答案忠实度
4 """
5 prompt = f"""
6 检索到的文档:
7 {retrieved_docs}
8
9 生成的答案:
10 {generated_answer}
11
12 请评估：生成的答案中的信息是否都能在检索文档中找到依据？
13 评分：1-5分（5分=完全忠实，1分=大量虚构）
14
15 输出格式：
16 得分： X/5
17 理由： ...
18 """
19
20 result = llm.predict(prompt)
21 return result
22
23 # 使用示例
24 docs = "2023年Q4销售额为5000万元，同比增长25%"
25 answer = "2023年Q4销售额为5000万元，预计2024年将达到8000万元"
26
27 faithfulness_score = evaluate_faithfulness(answer, docs, llm)
28 print(faithfulness_score)
```

指标3：相关性（Relevance）

通俗解释：回答是否切题？

代码块

```
1 # 用户问题
```

```
2 query = "如何提高团队效率？"
3
4 # ❌ 低相关性
5 answer = "团队建设很重要，沟通是关键..." # 太泛泛
6
7 # ✅ 高相关性
8 answer = "提高团队效率的3个方法：1)使用敏捷开发 2)每日站会 3)OKR目标管理"
```

### 🎯 评估维度3：端到端评估

#### 使用RAGAS框架

```
代码块

1 from ragas import evaluate
2 from ragas.metrics import (
3 faithfulness,
4 answer_relevancy,
5 context_relevancy,
6 context_recall
7)
8
9 # 准备评估数据
10 eval_data = {
11 "question": ["2023年Q4销售额是多少？", "产品有哪些核心功能？"],
12 "answer": ["5000万元", "包括AI助手、数据分析、自动化流程"],
13 "contexts": [
14 ["2023年Q4销售额为5000万元，同比增长25%"],
15 ["核心功能包括：AI智能助手、实时数据分析、工作流自动化"]
16],
17 "ground_truths": ["5000万元", "AI助手、数据分析、自动化"]
18 }
19
20 # 执行评估
21 result = evaluate(
22 eval_data,
23 metrics=[
24 faithfulness, # 忠实度
25 answer_relevancy, # 答案相关性
26 context_relevancy, # 上下文相关性
27 context_recall # 上下文召回
28]
29)
30
31 # 查看结果
32 print(result)
33 # 输出示例：
34 # {
35 # 'faithfulness': 0.95,
36 # 'answer_relevancy': 0.92,
37 # 'context_relevancy': 0.88,
38 # 'context_recall': 0.90
39 # }
```

### 📈 完整评估流程

```
代码块

1 class RAGEvaluator:
2 """RAG系统评估器"""
```

```
3
4 def __init__(self, rag_system, test_dataset):
5 self.rag_system = rag_system
6 self.test_dataset = test_dataset # 测试数据集
7
8 def evaluate_retrieval(self):
9 """评估检索效果"""
10 recalls = []
11 precisions = []
12 mrrs = []
13
14 for test_case in self.test_dataset:
15 query = test_case['query']
16 relevant_docs = test_case['relevant_docs']
17
18 # 执行检索
19 retrieved_docs = self.rag_system.retrieve(query, k=5)
20
21 # 计算指标
22 recall = self._calculate_recall(retrieved_docs,
relevant_docs)
23 precision = self._calculate_precision(retrieved_docs,
relevant_docs)
24 mrr = self._calculate_mrr(retrieved_docs,
relevant_docs)
25
26 recalls.append(recall)
27 precisions.append(precision)
28 mrrs.append(mrr)
29
30 return {
31 'avg_recall': sum(recalls) / len(recalls),
32 'avg_precision': sum(precisions) / len(precisions),
33 'avg_mrr': sum(mrrs) / len(mrrs)
34 }
35
36 def evaluate_generation(self):
37 """评估生成效果"""
38 faithfulness_scores = []
39 relevancy_scores = []
40
41 for test_case in self.test_dataset:
42 query = test_case['query']
43 ground_truth = test_case['answer']
44
45 # 执行RAG
46 answer, contexts = self.rag_system.query(query)
47
48 # 评估
49 faith_score = self._evaluate_faithfulness(answer,
contexts)
50 rel_score = self._evaluate_relevancy(answer, query)
51
52 faithfulness_scores.append(faith_score)
53 relevancy_scores.append(rel_score)
54
55 return {
56 'avg_faithfulness': sum(faithfulness_scores) /
len(faithfulness_scores),
57 'avg_relevancy': sum(relevancy_scores) /
len(relevancy_scores)
58 }
```



```
59
60 def run_full_evaluation(self):
61 """运行完整评估"""
62 retrieval_metrics = self.evaluate_retrieval()
63 generation_metrics = self.evaluate_generation()
64
65 print("=" * 50)
66 print("RAG系统评估报告")
67 print("=" * 50)
68 print("\n📊 检索效果:")
69 print(f" 召回率: {retrieval_metrics['avg_recall']:.2%}")
70 print(f" 准确率: {retrieval_metrics['avg_precision']:.2%}")
71 print(f" MRR: {retrieval_metrics['avg_mrr']:.3f}")
72 print("\n✍️ 生成效果:")
73 print(f" 忠实度: {generation_metrics['avg_faithfulness']:.2%}")
74 print(f" 相关性: {generation_metrics['avg_relevancy']:.2%}")
75 print("=" * 50)
76
77 return {**retrieval_metrics, **generation_metrics}
78
79 # 使用示例
80 test_data = [
81 {
82 'query': '2023年Q4销售额是多少? ',
83 'relevant_docs': ['doc1', 'doc3'],
84 'answer': '5000万元'
85 },
86 # ... 更多测试用例
87]
88
89 evaluator = RAGEvaluator(rag_system, test_data)
90 results = evaluator.run_full_evaluation()
```

## 7. 实战代码：手把手搭建你的第一个RAG

### 🔧 完整代码实现

代码块

```
1 """
2 完整的RAG系统实现
3 支持：文档加载、向量化、检索、生成
4 """
5
6 import os
7 from typing import List, Dict
8 from langchain.document_loaders import TextLoader, PyPDFLoader
9 from langchain.text_splitter import RecursiveCharacterTextSplitter
10 from langchain.embeddings import HuggingFaceEmbeddings
11 from langchain.vectorstores import Chroma
12 from langchain.chat_models import ChatOpenAI
13 from langchain.chains import RetrievalQA
14 from langchain.prompts import PromptTemplate
15
16 class SimpleRAG:
17 """简单的RAG系统"""
```



```

18
19 def __init__(self,
20 embedding_model: str = "paraphrase-multilingual-
MiniLM-L12-v2",
21 llm_model: str = "gpt-3.5-turbo",
22 chunk_size: int = 500,
23 chunk_overlap: int = 50):
24 """
25 初始化RAG系统
26
27 Args:
28 embedding_model: 向量化模型名称
29 llm_model: 大语言模型名称
30 chunk_size: 文本块大小
31 chunk_overlap: 文本块重叠大小
32 """
33 print("🚀 初始化RAG系统...")
34
35 # 1. 初始化文本分割器
36 self.text_splitter = RecursiveCharacterTextSplitter(
37 chunk_size=chunk_size,
38 chunk_overlap=chunk_overlap,
39 separators=["\n\n", "\n", "。 ", "! ", "? ", " ", ""])
40
41
42 # 2. 初始化向量化模型
43 print(f"📦 加载向量化模型: {embedding_model}")
44 self.embeddings = HuggingFaceEmbeddings(
45 model_name=embedding_model,
46 model_kwargs={'device': 'cpu'})
47
48
49 # 3. 初始化LLM
50 print(f"🤖 加载大语言模型: {llm_model}")
51 self.llm = ChatOpenAI(
52 model_name=llm_model,
53 temperature=0.1 # 降低随机性, 提高准确性
54)
55
56 # 4. 向量数据库
57 self.vectorstore = None
58 self.qa_chain = None
59
60 print("✅ RAG系统初始化完成! \n")
61
62 def load_documents(self, file_paths: List[str]) -> List[str]:
63 """
64 加载文档
65
66 Args:
67 file_paths: 文件路径列表
68
69 Returns:
70 文档内容列表
71 """
72 print("📁 开始加载文档...")
73 all_docs = []
74
75 for file_path in file_paths:
76 print(f" - 加载: {file_path}")
77
78 # 根据文件类型选择加载器

```

```
79 if file_path.endswith('.pdf'):
80 loader = PyPDFLoader(file_path)
81 else:
82 loader = TextLoader(file_path, encoding='utf-8')
83
84 docs = loader.load()
85 all_docs.extend(docs)
86
87 print(f"✅ 成功加载 {len(all_docs)} 个文档\n")
88 return all_docs
89
90 def build_knowledge_base(self, documents: List[str]):
91 """
92 构建知识库
93
94 Args:
95 documents: 文档列表
96 """
97 print("🔪 开始构建知识库...")
98
99 # 1. 文档切分
100 print(" 1️⃣ 切分文档...")
101 chunks = self.text_splitter.split_documents(documents)
102 print(f" 切分为 {len(chunks)} 个文本块")
103
104 # 2. 向量化并存储
105 print(" 2️⃣ 向量化并存储到数据库...")
106 self.vectorstore = Chroma.from_documents(
107 documents=chunks,
108 embedding=self.embeddings,
109 persist_directory="./chroma_db"
110)
111 print(f" ✅ 已存储到向量数据库")
112
113 # 3. 创建检索QA链
114 print(" 3️⃣ 创建问答链...")
115 self._create_qa_chain()
116
117 print(f"✅ 知识库构建完成! \n")
118
119 def _create_qa_chain(self):
120 """创建问答链"""
121 # 自定义提示模板
122 template = """你是一个专业的AI助手。请基于以下检索到的上下文信息来
回答问题。
123
124 【重要规则】
125 1. 只使用提供的上下文信息回答
126 2. 如果上下文中没有相关信息，明确说"根据提供的信息无法回答"
127 3. 回答要准确、简洁
128 4. 如果可能，引用具体的数据或事实
129
130 上下文信息：
131 {context}
132
133 问题: {question}
134
135 答案: """
136
137 prompt = PromptTemplate(
138 template=template,
139 input_variables=["context", "question"]
```

```

140)
141
142 self.qa_chain = RetrievalQA.from_chain_type(
143 llm=self.llm,
144 chain_type="stuff", # 将所有文档一次性传给LLM
145 retriever=self.vectorstore.as_retriever(
146 search_kwargs={"k": 3} # 检索Top 3相关文档
147),
148 chain_type_kwargs={"prompt": prompt},
149 return_source_documents=True # 返回源文档
150)
151
152 def query(self, question: str) -> Dict:
153 """
154 查询问题
155
156 Args:
157 question: 用户问题
158
159 Returns:
160 包含答案和来源文档的字典
161 """
162 if not self.qa_chain:
163 raise ValueError("请先构建知识库! ")
164
165 print(f"🔍 查询: {question}")
166 print("-" * 50)
167
168 # 执行查询
169 result = self.qa_chain({"query": question})
170
171 # 提取答案和来源
172 answer = result['result']
173 source_docs = result['source_documents']
174
175 print(f"\n💡 答案:\n{answer}\n")
176 print(f"📄 来源文档 ({len(source_docs)}个):")
177 for i, doc in enumerate(source_docs, 1):
178 print(f" [{i}] {doc.page_content[:100]}...")
179 print("-" * 50 + "\n")
180
181 return {
182 'answer': answer,
183 'sources': [doc.page_content for doc in source_docs]
184 }
185
186 def search_similar(self, query: str, k: int = 5) -> List[Dict]:
187 """
188 搜索相似文档
189
190 Args:
191 query: 查询文本
192 k: 返回数量
193
194 Returns:
195 相似文档列表
196 """
197 if not self.vectorstore:
198 raise ValueError("请先构建知识库! ")
199
200 results =
self.vectorstore.similarity_search_with_score(query, k=k)

```

```
201
202 print(f"🔍 搜索: {query}")
203 print(f"📄 找到 {len(results)} 个相关文档:\n")
204
205 similar_docs = []
206 for i, (doc, score) in enumerate(results, 1):
207 print(f" [{i}] 相似度: {score:.3f}")
208 print(f" 内容: {doc.page_content[:100]}...\n")
209 similar_docs.append({
210 'content': doc.page_content,
211 'score': float(score)
212 })
213
214 return similar_docs
215
216
217 # ===== 使用示例 =====
218
219 def main():
220 """主函数：演示RAG系统的完整使用流程"""
221
222 # 1. 创建RAG系统
223 rag = SimpleRAG(
224 embedding_model="paraphrase-multilingual-MiniLM-L12-v2",
225 llm_model="gpt-3.5-turbo",
226 chunk_size=500,
227 chunk_overlap=50
228)
229
230 # 2. 准备测试文档
231 print("✍️ 创建测试文档...")
232
233 # 创建示例文档
234 with open("company_report.txt", "w", encoding="utf-8") as f:
235 f.write("""
236 2023年度公司财务报告
237
238 一、业绩概况
239 2023年全年，公司实现营业收入2.5亿元，同比增长35%。
240 其中，第四季度单季度营收达到5000万元，创历史新高。
241
242 二、产品线表现
243 - AI助手产品：年销售额1.2亿元，占比48%
244 - 数据分析平台：年销售额8000万元，占比32%
245 - 企业服务：年销售额5000万元，占比20%
246
247 三、市场展望
248 预计2024年公司营收将突破3亿元，继续保持高速增长态势。
249 """)
250
251 with open("product_manual.txt", "w", encoding="utf-8") as f:
252 f.write("""
253 产品使用手册
254
255 产品名称：智能AI助手
256 版本：v2.0
257
258 核心功能：
259 1. 自然语言对话：支持多轮对话，理解上下文
260 2. 知识问答：基于企业知识库回答问题
261 3. 任务自动化：自动执行重复性任务
262 4. 数据分析：实时分析业务数据
```

```
263
264 使用场景：
265 - 客户服务：24小时在线客服
266 - 内部协作：团队知识管理
267 - 业务分析：销售数据洞察
268 """
269
270 # 3. 加载文档
271 documents = rag.load_documents([
272 "company_report.txt",
273 "product_manual.txt"
274])
275
276 # 4. 构建知识库
277 rag.build_knowledge_base(documents)
278
279 # 5. 测试查询
280 print("=" * 70)
281 print("开始测试查询")
282 print("=" * 70 + "\n")
283
284 # 测试1: 财务问题
285 rag.query("2023年第四季度的营收是多少? ")
286
287 # 测试2: 产品问题
288 rag.query("AI助手有哪些核心功能? ")
289
290 # 测试3: 搜索相似文档
291 rag.search_similar("产品功能介绍", k=3)
292
293 print("✅ 测试完成! ")
294
295
296 if __name__ == "__main__":
297 main()
```

🎯 运行效果示例

```
代码块

1 🚀 初始化RAG系统...
2 📦 加载向量化模型: paraphrase-multilingual-MiniLM-L12-v2
3 🤖 加载大语言模型: gpt-3.5-turbo
4 ✅ RAG系统初始化完成!
5
6 📁 开始加载文档...
7 - 加载: company_report.txt
8 - 加载: product_manual.txt
9 ✅ 成功加载 2 个文档
10
11 🛠️ 开始构建知识库...
12 1️⃣ 切分文档...
13 切分为 8 个文本块
14 2️⃣ 向量化并存储到数据库...
15 ✅ 已存储到向量数据库
16 3️⃣ 创建问答链...
17 ✅ 知识库构建完成!
18
19 =====
20 ===
21 开始测试查询
```

21 =====  
22 ===  
23 🔍 查询：2023年第四季度的营收是多少？  
24 -----  
25  
26 💡 答案：  
27 根据2023年度公司财务报告，2023年第四季度的营收达到5000万元，创历史新高。  
28  
29 📄 来源文档（3个）：  
30 [1] 一、业绩概况  
31 2023年全年，公司实现营业收入2.5亿元，同比增长35%。  
32 其中，第四季度单季度营收达到5000万元，创历史新高...  
33 [2] 三、市场展望  
34 预计2024年公司营收将突破3亿元，继续保持高速增长态势...  
35 [3] 二、产品线表现  
36 - AI助手产品：年销售额1.2亿元，占比48%...  
37 -----

## 8. 进阶技巧：让你的RAG系统脱颖而出

### 🚀 技巧1：多路召回策略

代码块

```
1 class AdvancedRetriever:
2 """高级检索器：结合多种检索方法"""
3
4 def __init__(self, vectorstore, bm25_index, reranker):
5 self.vectorstore = vectorstore
6 self.bm25_index = bm25_index
7 self.reranker = reranker
8
9 def retrieve(self, query: str, k: int = 5) -> List[str]:
10 """多路召回 + 重排序"""
11
12 # 路径1: 向量检索 (Top 20)
13 vector_docs = self.vectorstore.similarity_search(query,
14 k=20)
15
16 # 路径2: BM25关键词检索 (Top 20)
17 bm25_docs = self.bm25_index.search(query, k=20)
18
19 # 路径3: 混合检索 (去重)
20 all_docs = list(set(vector_docs + bm25_docs))
21
22 # 重排序: 用更强的模型打分
23 reranked_docs = self.reranker.rerank(query, all_docs)
24
25 # 返回Top K
26 return reranked_docs[:k]
```

### 🚀 技巧2：查询改写

代码块

```
1 def query_rewrite(original_query: str, llm) -> List[str]:
```

```

2 """
3 查询改写：生成多个变体查询
4 """
5 prompt = f"""
6 原始查询：{original_query}
7
8 请生成3个语义相同但表达不同的查询变体，用于提高检索召回率。
9
10 输出格式（每行一个）：
11 1. ...
12 2. ...
13 3. ...
14 """
15
16 response = llm.predict(prompt)
17 variants = [line.strip() for line in response.split('\n') if
18 line.strip()]
19
20 return [original_query] + variants
21
22 # 使用示例
23 query = "公司去年赚了多少钱？"
24 variants = query_rewrite(query, llm)
25 # 输出：
26 # ['公司去年赚了多少钱？',
27 # '2023年公司的营收是多少？',
28 # '去年公司财务表现如何？',
29 # '上一年度的销售额统计']
30
31 # 用所有变体检索，合并结果
32 all_results = []
33 for q in variants:
34 results = vectorstore.similarity_search(q, k=5)
35 all_results.extend(results)
36
37 # 去重并重排序
38 final_results = rerank(all_results)

```

### 技巧3：分层检索

代码块

```

1 class HierarchicalRAG:
2 """分层RAG：先粗筛，再精筛"""
3
4 def retrieve(self, query: str):
5 # 第1层：快速粗筛 (Top 100)
6 coarse_results = self.fast_retriever.search(query, k=100)
7
8 # 第2层：精细重排 (Top 20)
9 fine_results = self.reranker.rerank(query, coarse_results,
10 k=20)
11
12 # 第3层：LLM筛选 (Top 5)
13 final_results = self.llm_filter(query, fine_results, k=5)
14
15 return final_results

```

### 技巧4：实时更新知识库



代码块

```
1 class IncrementalRAG:
2 """支持增量更新的RAG"""
3
4 def add_document(self, new_doc: str):
5 """添加新文档"""
6 # 切分
7 chunks = self.text_splitter.split_text(new_doc)
8
9 # 向量化
10 vectors = self.embeddings.embed_documents(chunks)
11
12 # 增量添加到向量库 (不需要重建整个库)
13 self.vectorstore.add_texts(chunks, vectors)
14
15 print(f"✅ 已添加 {len(chunks)} 个新文档块")
16
17 def update_document(self, doc_id: str, new_content: str):
18 """更新文档"""
19 # 删除旧版本
20 self.vectorstore.delete(doc_id)
21
22 # 添加新版本
23 self.add_document(new_content)
```



# 全文评论



用户329087 2025年12月28日

为什么最后一节是重复的



何东 4月15日 15:10

这么多要看到什么时候？



用户541651 7月11日 11:15

不赖